

Edvard Kjesbu Bahr

Neural Posterior Estimation for Discrete-Time Stochastic Volatility Models

Master's thesis in Industrial Mathematics

Supervisor: Jarle Tufto

July 2026

Edvard Kjesbu Bahr

Neural Posterior Estimation for Discrete-Time Stochastic Volatility Models

Master's thesis in Industrial Mathematics

Supervisor: Jarle Tufto

July 2026

Norwegian University of Science and Technology

Faculty of Information Technology and Electrical Engineering

Department of Mathematical Sciences



Norwegian University of
Science and Technology

Abstract

Stochastic-volatility models describe financial returns through an unobserved volatility process, which makes repeated Bayesian parameter inference computationally demanding. This thesis studies whether neural posterior estimation can amortize this task by learning the first two moments of transformed marginal parameter posteriors from simulated parameter–data pairs. Two estimators are considered for the standard stochastic-volatility model: a summary-statistic neural network and a temporal convolutional network (TCN) that learns directly from transformed return sequences.

The estimators are evaluated on 5 000 simulated sequences of length 253 against moment-matched Gaussian marginals constructed from 20 000 retained MCMC draws per sequence. The TCN gives the closest neural agreement with MCMC. Under the marginal Gaussian loss, its performance is similar to MCMC conditioned on 85% of the observations under a broad default prior and 90% under a more concentrated finance prior. Its mean evaluation time is 0.0028 seconds per sequence, compared with approximately 3.5 seconds for the benchmark MCMC configuration.

The TCN is subsequently extended to a time-irreversible GHST-SV model with skewed and heavy-tailed log-volatility innovations. Relative to a transformed-prior reference, it reduces the overall loss by 81.6% and continues to estimate the three standard-SV parameters effectively. The improvements are only 5.9% for the parameter governing skewness and 9.6% for the parameter governing tail weight. Credible-interval sweeps and simulation-based calibration show that these two marginal estimates remain strongly prior-dependent and are not accurately represented.

The results show that TCN-based neural posterior estimation can provide fast and accurate marginal posterior-moment estimates for the standard stochastic-volatility model in a fixed amortized setting. The architecture is straightforward to adapt to richer simulatable models, but the GHST-SV experiment shows that successful adaptation also depends on how strongly the new parameters affect the observed sequence and on the adequacy of the chosen posterior family.

Sammendrag

Stokastiske volatilitetsmodeller beskriver finansielle avkastninger ved hjelp av en uobservert volatilitetsprosess, noe som gjør gjentatt bayesiansk parameterinferens beregningskrevende. Denne oppgaven undersøker om nevralt posterior-estimering kan amortisere denne oppgaven ved å lære de to første momentene til transformerte marginale parameterposteriorer fra simulerte parameter-datapar. To estimatorene vurderes for den standard stokastiske volatilitetsmodellen: et nevralt nettverk basert på oppsummeringsstatistikker og et temporalt konvolusjonsnettverk (TCN) som lærer direkte fra transformerte avkastningssekvenser.

Estimatorene evalueres på 5 000 simulerte sekvenser med lengde 253, mot momenttilpassede gaussiske marginalfordelinger konstruert fra 20 000 beholdte MCMC-trekk per sekvens. Av de to nevrale estimatorene gir TCN-estimatoren resultater som ligger nærmest MCMC. Målt ved den marginale gaussiske loss-funksjonen er ytelsen sammenlignbar med MCMC betinget på 85% av observasjonene under en bred standardprior og 90% under en mer konsentrert finansprior. Den gjennomsnittlige evalueringstiden er 0.0028 sekunder per sekvens, sammenlignet med omtrent 3.5 sekunder for MCMC-konfigurasjonen som brukes som referanse.

TCN-estimatoren utvides deretter til en tidsirreversibel GHST-SV-modell med skjeve og tykkhalede log-volatilitetsinnovasjoner. Sammenlignet med en referanse basert på den transformerte prioren reduserer den den samlede lossverdien med 81.6%, samtidig som den fortsatt estimerer de tre standard-SV-parametrene effektivt. Forbedringene er imidlertid bare 5.9% for parameteren som styrer skjevhet og 9.6% for parameteren som styrer haletyngde. Parametersveip med kredibilitetsintervaller og simuleringsbasert kalibrering viser at disse to marginale estimatene fortsatt er sterkt påvirket av prioren og ikke representeres nøyaktig.

Resultatene viser at TCN-basert nevralt posterior-estimering kan gi raske og nøyaktige estimater av momentene til de marginale posteriorfordelingene for den standard stokastiske volatilitetsmodellen i en fast amortisert setting. Arkitekturen er enkel å tilpasse til rikere modeller som det er mulig å simulere fra, men GHST-SV-eksperimentet viser at en vellykket tilpasning også avhenger av hvor sterkt de nye parametrene påvirker den observerte sekvensen, og av hvor egnet den valgte posteriorfamilien er.

Contents

Abstract	iii
Sammendrag	v
Contents	vii
Figures	xi
Tables	xiii
1 Introduction	1
1.1 Motivation and problem formulation	1
1.2 Approach and scope	2
1.3 Sustainability relevance	4
1.4 Thesis structure	4
2 Theory	5
2.1 Notation and conventions	5
2.1.1 Indexed collections, finite ranges and subscript conventions	6
2.1.2 Random variables, time series, and sequences	7
2.1.3 Latent variables and the likelihood function	8
2.1.4 Equality in distribution	8
2.2 Stationarity	9
2.2.1 Mean, autocovariance and autocorrelation	9
2.2.2 Weak stationarity	9
2.2.3 Strict stationarity	9
2.3 ARMA processes	10
2.3.1 White noise innovations	10
2.3.2 Autoregressive processes	10
2.3.3 Moving-average processes	12
2.3.4 Autoregressive moving-average processes	12
2.4 Stochastic volatility models	12
2.4.1 Model definition and properties	13
2.4.2 MLE of SV parameters	14
2.4.3 Approximate estimation methods for SV parameters	15
2.4.4 Bayesian inference, MCMC for SV parameters and ESS	17
2.5 Neural posterior estimation	20
2.5.1 Neural posterior estimation as amortized inference	20
2.5.2 From simulated pairs to a posterior log-score objective	21
2.5.3 Choice of posterior family and parameter transformation	23

2.5.4	Marginal Gaussian loss and shared representation	24
2.6	Neural network architectures	26
2.6.1	Encoder-head structure and multilayer perceptron heads . .	26
2.6.2	Summary-statistic MLP encoder	29
2.6.3	Transformed log-return TCN encoder	32
2.7	Stochastic optimization with Adam	37
2.7.1	Population loss, mini-batches and stochastic gradients . . .	38
2.7.2	Adam update rule	40
2.7.3	Population loss and posterior approximation	42
3	Model Implementation	47
3.1	Data simulation and numerical implementation	48
3.1.1	Prior configurations and the units of μ	48
3.1.2	Construction of training inputs and targets	50
3.1.3	Numerical safeguards and reproducibility	52
3.2	MCMC configuration and choice of sequence length	54
3.2.1	Burn-in, retained chain length and effective sample size . .	54
3.2.2	Empirical Bernstein-von Mises diagnostic and choice of se- quence length	56
3.2.3	Fixed MCMC and sequence-length configuration across model simulations	58
3.3	Neural-network implementation with PyTorch	59
3.3.1	Gaussian negative log-likelihood and graph-based automatic differentiation	60
3.3.2	CUDA acceleration, automatic mixed precision and gradient diagnostics	61
3.3.3	Deterministic execution and reproducibility	62
3.4	Early stopping and benchmark evaluation	62
3.4.1	Simulation-based validation and early stopping	63
3.4.2	Metrics for comparison with the MCMC reference	65
3.5	Tuning-parameter selection	68
3.5.1	Tuning the summary-statistic network	69
3.5.2	Tuning the temporal convolutional network	70
3.5.3	Adam configuration and comparison of the two networks . .	71
4	Results	73
4.1	Validation loss and Monte Carlo uncertainty	73
4.1.1	Validation loss during training	73
4.1.2	Relative standard deviations of the loss estimates	75
4.2	Comparison with the MCMC reference	76
4.2.1	Posterior-mean RMSE	76
4.2.2	Posterior variance	77
4.2.3	Marginal Gaussian loss differences	78
4.2.4	Comparison with reduced-data MCMC references	78
4.2.5	Empirical coverage	79
4.3	Marginal credible intervals across parameter sweeps	81

4.4	Runtime of posterior-moment estimation	84
4.5	Discussion of results	85
4.5.1	Separating moment accuracy from posterior shape	86
4.5.2	Sequence representation and model extension	86
4.5.3	Speed, flexibility and repeated inference	87
4.5.4	Overall comparison of the inference methods	88
4.5.5	Extensions and further evaluation	89
5	Time Irreversible SV models	91
5.1	The GHST-SV model	92
5.1.1	Time irreversibility	92
5.1.2	Selecting the log-volatility innovation distribution	93
5.1.3	GHST-SV model definition	96
5.1.4	Priors and unconstrained parameterization	98
5.2	Model implementation	99
5.2.1	Target and input construction and tuning parameters	99
5.2.2	MCMC and transformed-prior reference	101
5.2.3	Simulation-based calibration and the Kolmogorov–Smirnov test	102
5.2.4	Approximating the initial distribution	104
5.3	Results	106
5.3.1	Validation loss history	106
5.3.2	Loss comparison among MCMC, TCN and prior	107
5.3.3	Credible intervals and simulation-based calibration	108
6	Discussion	113
6.1	Overall assessment	113
6.2	Consequences of the marginal Gaussian approximation	114
6.3	Interpretation of the GHST-SV extension	114
6.4	Further work	116
	Conclusion	117
	Bibliography	119
A	Additional Material	125
A.1	GitHub project link	125
A.2	Moments of the log-squared observations	125
A.3	ARIMA initialization	127
A.4	Gaussian limit as $\nu \rightarrow \infty$	127

Figures

- 3.1 Marginal prior densities under the default and finance prior configurations. The finance prior is more concentrated for μ and ϕ , whereas both configurations assign the same prior to σ^2 49
- 3.2 Distribution of the minimum ESS across the transformed parameter chains and their squared centered values. For each prior configuration, 2000 independently simulated SV sequences of length $n = 253$ were analyzed using $B = 500$ discarded iterations followed by $M = 20\,000$ retained draws without thinning. 55
- 3.3 Gaussian QQ plots for standardized draws of the transformed SV parameters. The panels compare the prior, denoted by $n = 0$, with posteriors conditioned on sequences of length $n = 253$ and $n = 1012$. The MCMC chains retain 20 000 draws following a burn-in of 500 iterations. 57
- 4.1 Validation-loss histories for the TCN and Summary NN under the default and finance priors. The vertical axis is logarithmic, and the crosses mark the epochs at which training terminated. All four runs were terminated by the early-stopping rule before $E_{\max} = 2000$. The restored weights correspond to the best qualifying checkpoint and therefore need not be those from the marked terminal epoch. . 74
- 4.2 Empirical coverage of the central Gaussian credible intervals, estimated from $K = 5\,000$ benchmark pairs. Rows correspond to μ , ψ , and ρ , while columns correspond to the default and finance priors. The dashed diagonal represents exact nominal coverage. The `stochvol` curves use Gaussian marginals whose means and variances are estimated from the retained `stochvol` MCMC draws. . . . 80

4.3	Posterior medians and 95% marginal credible intervals across the parameter sweeps. Rows correspond to the parameter being varied, and columns correspond to the default and finance priors. Each point is based on one simulated sequence of length $n = 253$, with the remaining parameters fixed at their baseline values. The dashed line represents equality between the posterior median and the generating parameter. The <code>stochvol</code> intervals are empirical quantile intervals calculated from $M = 20\,000$ retained MCMC draws, whereas the NPE intervals are constructed in the transformed space and mapped back using (4.2).	83
5.1	Standardized GHST (left) and SHASH (right) densities. The blue curves show the Gaussian case, while the green and red curves show positive skewness with increasing tail weight.	95
5.2	Simulated paths from the standard SV model with Gaussian log-volatility innovations (left) and the GHST-SV model with positively skewed and heavy-tailed innovations (right). The upper panels show the latent log volatilities h_t , while the lower panels show the corresponding observed log returns y_t	97
5.3	Validation-loss history for the TCN under the GHST-SV model. The vertical axis is logarithmic, and the cross marks the terminal epoch. The restored weights correspond to the best qualifying checkpoint.	107
5.4	Posterior medians and 95% marginal credible intervals across the parameter sweeps (left), and TCN SBC histograms calculated from $K = 5\,000$ benchmark pairs (right). The rows correspond to μ , ϕ , and σ . In the left column, the dashed lines mark equality between the generating parameter and posterior median. In the right column, the dashed lines show the density of the uniform distribution.	109
5.5	TCN posterior medians and 95% marginal credible intervals across the r and ν sweeps (left), and the corresponding SBC histograms calculated from $K = 5\,000$ benchmark pairs (right). In the left column, the dashed lines mark equality between the generating parameter and posterior median. In the right column, the dashed lines show the density of the uniform distribution.	111

Tables

3.1	Prior configurations used when simulating data from the SV model.	48
3.2	Numerical safeguards used in simulation, feature construction and neural-network training.	53
3.3	MCMC and sequence-length configuration used in the benchmark experiments.	59
3.4	Training and early-stopping configurations used for the two neural estimators.	64
3.5	Post-training validation losses under the default prior for the summary-statistic network. The smallest loss is shown in bold.	69
3.6	Selected architectural and optimization settings and trainable parameter counts for the two neural estimators.	71
4.1	Relative standard deviations of the parameter-wise mean losses, overall mean losses, and their paired differences, calculated from $K = 5\,000$ benchmark pairs. In each column heading, the dot in the superscript denotes the method or method difference given in the first column.	75
4.2	Posterior-mean RMSEs relative to the generating transformed parameters, estimated from $K = 5\,000$ benchmark pairs.	77
4.3	Geometric mean ratios of NPE marginal posterior variance to MCMC posterior variance, estimated from $K = 5\,000$ benchmark pairs and $M = 20\,000$ retained MCMC draws per pair.	77
4.4	Marginal Gaussian loss differences relative to the moment-matched MCMC reference, estimated from $K = 5\,000$ benchmark pairs and $M = 20\,000$ retained MCMC draws per pair.	78
4.5	Raw marginal Gaussian losses for the full-sequence TCN and the closest MCMC reference among the considered truncated sequence lengths. Results are based on $K = 5\,000$ benchmark pairs and $M = 20\,000$ retained MCMC draws per pair.	79
4.6	Baseline values and ranges used in the credible-interval sweeps. Each parameter was varied over ten equally spaced values, including the endpoints, while the remaining parameters were held at their baseline values.	82

4.7	Mean and standard deviation of the runtime required to estimate the first two moments of the transformed marginal posteriors for one sequence of length $n = 253$, calculated from $K = 5\,000$ benchmark pairs.	85
4.8	Main practical trade-offs among the posterior estimators considered in this thesis.	88
5.1	Selected tuning-parameter configurations used to train the standard SV and GHST-SV TCN posterior estimators, together with the resulting receptive fields and trainable parameter counts.	100
5.2	Means and variances of the transformed prior distributions.	102
5.3	Marginal Gaussian losses for MCMC, TCN and prior, calculated from $K = 5\,000$ benchmark pairs with sequence length $n = 2\,530$. The MCMC moments are calculated from $M = 20\,000$ retained draws per pair after $B = 500$ burn-in iterations.	107
5.4	Baseline values and ranges used in the GHST-SV credible-interval sweeps.	108

Chapter 1

Introduction

1.1 Motivation and problem formulation

Volatility is an important property of financial returns. It describes the variation in returns and is therefore relevant when assessing the uncertainty associated with an investment. This volatility is not observed directly and is rarely constant over time. Periods of small return variation are often followed by other calm periods, while periods of large variation tend to occur together. Stochastic-volatility models account for this behavior by allowing the variance of the returns to depend on an unobserved process.

The potential scale of volatility analysis can be illustrated by Norges Bank Investment Management (NBIM), which manages the Government Pension Fund Global. Norges Bank Investment Management [1] reports that the fund holds equity investments in more than 7 200 companies across different countries, industries and currencies. Its market-risk framework includes measures such as volatility, expected shortfall and stress tests [2].

This provides a useful thought experiment. Suppose that an SV model were estimated separately from the return sequence of each equity holding. Bayesian inference would then give a posterior distribution for the parameters describing the corresponding volatility process. In the standard SV model, these parameters describe the average log-volatility, its persistence and the size of the log-volatility innovations. Their posterior distributions would also describe the uncertainty associated with these properties.

Carrying out this analysis for a large number of return sequences would require the same posterior-inference problem to be solved repeatedly. The NBIM example is only used to illustrate this computational setting. This thesis neither analyzes data from NBIM nor proposes a portfolio-risk system. Instead, it considers the more limited methodological question of whether posterior information about SV parameters can be estimated accurately and efficiently for new return sequences after an initial training stage.

Bayesian inference for SV models is complicated by the latent log-volatility process. Markov chain Monte Carlo methods can approximate the parameter pos-

terior, but a new Markov chain must be generated and analyzed for each observed sequence. This may be manageable for a small number of datasets, but the total cost increases when the same model is estimated repeatedly across many assets, time periods or simulation studies. Furthermore, extending the SV model may require corresponding changes to the posterior-sampling algorithm.

Neural posterior estimation provides a possible alternative. NPE is a form of amortized simulation-based inference in which a neural network is trained on parameter–data pairs simulated from the assumed model. The trained network learns a mapping from an observed dataset to a parametric approximation of its parameter posterior. The main computational cost is therefore incurred during training, while posterior estimates for subsequent datasets can be obtained through network evaluation.

NPE methods are often motivated by models whose likelihoods are unavailable or particularly difficult to evaluate. The same amortization principle may also be useful when established inference methods exist but become costly when applied repeatedly. Maceda et al. [3] study this idea by letting neural networks parameterize approximations to posterior distributions and training them on simulated parameter–data pairs. Their framework provides an important motivation for the approach considered in this thesis.

Constructing a flexible approximation of the complete joint posterior is a difficult problem. As an initial working model, we instead approximate each transformed marginal parameter posterior separately by a Gaussian distribution. The neural network estimates its mean and variance. This restriction does not represent posterior dependence or all possible non-Gaussian marginal shapes. It nevertheless provides estimates of the first two marginal posterior moments, which summarize the location and uncertainty of each transformed parameter.

The main question considered in this thesis is consequently:

Can neural posterior estimation provide accurate and computationally efficient estimates of the first two moments of the marginal parameter posteriors in stochastic-volatility models, and can the resulting approach be adapted to richer SV models?

The purpose is not to establish that the method is ready for large-scale financial applications. Rather, the thesis examines whether this restricted form of amortized posterior inference provides a useful starting point. If successful, the same general construction could later be studied for other quantities, such as latent volatilities, posterior predictions or volatility forecasts.

1.2 Approach and scope

The theoretical part of the thesis first introduces the standard log-Gaussian SV model and the concepts needed to describe it. Particular attention is given to stationarity, temporal dependence and the latent log-volatility process. Bayesian parameter inference and MCMC are then presented to explain how the parameter

posterior can be approximated using conventional sampling methods and why repeated inference can be computationally demanding.

The NPE objective is subsequently derived from parameter–data pairs generated under the prior and the SV model. The constrained model parameters are transformed to an unconstrained space, after which their marginal posteriors are approximated by Gaussian distributions. The resulting loss trains the neural networks to estimate the marginal posterior means and variances. The network architectures and stochastic optimization using Adam are also introduced.

Two NPE estimators are considered. The first is a summary-statistic neural network that receives a fixed collection of features calculated from each return sequence. The second uses a temporal convolutional network and learns directly from a transformed version of the complete sequence. This comparison examines whether a predefined summary map retains sufficient information for posterior estimation or whether a sequence-based representation provides more accurate estimates.

The model-implementation chapter describes how the theoretical construction is translated into a training procedure. This includes the transformations of the model parameters, the construction of the target and input matrices, the implementation of the two neural estimators in PyTorch and the selection of their tuning parameters. We use the term *simulation-based training* for the procedure in which new parameter–data pairs are repeatedly simulated during training and validation. This avoids reliance on a single fixed training sample and allows the estimators to be evaluated throughout optimization on newly simulated sequences.

The resulting estimators are first tested for the standard SV model. Their posterior means and variances are compared with corresponding estimates obtained using MCMC. The comparison considers their accuracy, their representation of posterior uncertainty and the time required to evaluate a new return sequence. These results are used to select the more suitable NPE estimator for the subsequent model extension.

The final part of the thesis considers an SV model that can represent a temporally asymmetric pattern in volatility more directly. Under the standard model, the Gaussian log-volatility innovations are symmetric. In the extended model, they are replaced by skewed and heavy-tailed innovations. This allows large upward volatility innovations to occur with a different probability than corresponding downward innovations. Together with the persistence of the latent process, this can represent a sudden increase in volatility followed by a more gradual return toward its typical level. The resulting process can therefore be time irreversible.

The selected NPE construction is adapted to the extended model by changing the simulation procedure and adding outputs for the parameters governing skewness and tail weight. The estimator is then retrained and tested using simulated data from the extended model. This examines one of the proposed advantages of simulation-based inference: once a new model can be simulated, the existing neural construction may be retained without first deriving a complete model-specific posterior-sampling method.

1.3 Sustainability relevance

The possible sustainability relevance of the thesis is indirect. If amortized posterior inference can reduce the time required for repeated statistical analyses and can be adapted to richer models, it may support the technological upgrading and productivity improvements described by United Nations Sustainable Development Goal 8.2 United Nations [4]. The thesis does not measure economic productivity or demonstrate this effect in a financial application, but it examines a methodological step that could support such future uses.

1.4 Thesis structure

Chapter 2 presents the theoretical framework. It introduces the standard SV model, Bayesian inference using MCMC, neural posterior estimation, the neural-network architectures and stochastic optimization using Adam.

Chapter 3 describes the implementation of the Summary NN and TCN estimators. It specifies the parameter transformations, input and target construction, simulation-based training procedure, validation scheme and tuning parameters used in the experiments.

Chapter 4 compares the two NPE estimators with MCMC for the standard SV model. The comparison considers posterior-moment estimates, uncertainty, computational time and the effect of the chosen sequence representation.

Chapter 5 extends the preferred NPE estimator to an SV model with skewed and heavy-tailed log-volatility innovations. It introduces the extended model, describes the required implementation changes and evaluates the resulting posterior estimates using simulation-based tests.

Chapter 6 discusses the findings across both SV models. It considers the strengths and limitations of the neural estimators, the weaker estimation of the GHST-SV shape parameters and possible directions for further work.

Chapter 2

Theory

This chapter presents the theoretical basis for neural posterior estimation in stochastic volatility models. We first introduce the notation and conventions used throughout the thesis. We then review stationarity and ARMA processes, which are needed to describe the latent log-volatility process and the dependence in log-squared returns. Next, we define the standard log-normal SV model and discuss the difficulties caused by its latent state process. This leads to a brief review of approximate likelihood methods and Bayesian inference using MCMC.

We then introduce NPE as an amortized simulation-based inference method. Its training objective is constructed from simulated parameter–data pairs. Since some SV parameters are constrained, they are transformed to an unconstrained space before their marginal posteriors are approximated by Gaussian distributions. This gives the marginal Gaussian loss used throughout the thesis.

The neural networks used to estimate the posterior means and variances are then presented. These are a summary-statistic MLP and a TCN that learns directly from transformed return sequences. Finally, training is described as stochastic optimization of a population loss using newly simulated mini-batches and Adam. We also show that this population loss corresponds to a prior-predictive average of forward Kullback–Leibler divergences, and that its unrestricted minimum matches the first two moments of the transformed marginal posteriors. These results provide the theoretical basis for the model implementations and empirical analyses in the following chapters.

2.1 Notation and conventions

This section states the notation and terminology used throughout the thesis. Its purpose is to clarify the conventions for indexed collections, finite samples, collections of finite samples, random variables, observed values, latent variables and equality in distribution.

2.1.1 Indexed collections, finite ranges and subscript conventions

We use curly brackets to denote indexed collections. Although curly brackets usually denote unordered sets, unordered sets do not play an important role in this thesis. A collection written using curly brackets should therefore be understood as retaining its indexing, ordering and any repeated elements.

For a two-sided time-indexed collection, we write

$$\{z_t\} = \{z_t : t \in \mathbb{Z}\}.$$

The notation $\{z_t\}$ without an explicitly stated index range is reserved for such a two-sided collection.

For an inclusive finite index range, we write

$$z_{m:n} = (z_m, \dots, z_n).$$

Thus, a colon within a subscript selects the finite vector of entries indexed from m to n .

Curly brackets with an explicitly stated finite index range instead collect objects as that index varies. For example, simulated parameter–sample pairs are written as

$$\{(\theta^{(i)}, y_{1:n}^{(i)})\}_{i=1}^N.$$

This denotes the finite indexed collection

$$(\theta^{(1)}, y_{1:n}^{(1)}), \dots, (\theta^{(N)}, y_{1:n}^{(N)}),$$

where $y_{1:n}^{(i)}$ is the sample generated conditionally on $\theta^{(i)}$. The superscript (i) labels the corresponding pair and should not be interpreted as a power. The absence or presence of an explicit index range therefore distinguishes the two-sided collection $\{z_t\}$ from a finite indexed collection such as $\{(\theta^{(i)}, y_{1:n}^{(i)})\}_{i=1}^N$.

When an object has several integer-valued indices, commas separate the index positions, whose order agrees with the order of the corresponding dimensions. For example, if

$$M \in \mathbb{R}^{p \times q \times m},$$

then $M_{i,j,\ell}$ denotes the scalar entry at position (i, j, ℓ) , where

$$i \in \{1, \dots, p\}, \quad j \in \{1, \dots, q\}, \quad \ell \in \{1, \dots, m\}.$$

A colon may replace an integer index in any position to select an inclusive range. For a two-index object z , for example,

$$z_{a,1:B} = (z_{a,1}, \dots, z_{a,B}),$$

and the complete A -by- B indexed array may be written as

$$z_{1:A,1:B} = \{z_{a,1:B}\}_{a=1}^A.$$

When z is interpreted as a matrix, its first index identifies the row and its second index identifies the column.

Not every symbol appearing in a subscript is an integer-valued index. A symbol introduced as a parameter or descriptive label retains that meaning, and a subscript may contain both descriptive components and integer-valued indices. For example, suppose that

$$T_W(y) = \{h_a(y; W)\}_{a=1}^A.$$

Here W denotes the network weights and is not an index, whereas a indexes the components of the output. Consequently, for $j \in \{1, \dots, A\}$,

$$T_{W,j}(y) = h_j(y; W).$$

In the mixed subscript W, j , the symbol W identifies the network weights, while j identifies an output component.

The semicolon in $h_j(y; W)$ separates arguments according to their roles: y is the input at which the function is evaluated, whereas W parameterizes the function. Replacing the semicolon by a comma would not change the mathematical function, but the semicolon makes this distinction explicit. It should not be confused with $|$, which is reserved in probabilistic expressions for conditioning, as in the posterior density

$$p(\theta | y).$$

2.1.2 Random variables, time series, and sequences

Throughout this thesis, upper-case letters denote random variables, while lower-case letters denote their realized or fixed values. For example, Y_t denotes a random variable, whereas y_t denotes a possible or observed realization of Y_t .

The same convention is used for random vectors. In particular,

$$Y_{1:n} = (Y_1, \dots, Y_n)$$

denotes a random vector, while

$$y_{1:n} = (y_1, \dots, y_n)$$

denotes one of its possible realizations.

A discrete-time process is a two-sided collection of random variables

$$\{X_t\}.$$

For integers $m \leq n$, the finite truncation

$$X_{m:n} = (X_m, \dots, X_n)$$

is a random vector consisting of the variables indexed from m to n . Throughout this thesis, *temporal* is simply used as the adjective corresponding to time. Temporal ordering therefore means ordering in time, while *temporal dependence* means

dependence over time, that is, statistical dependence between random variables at different time indices. In this thesis, both the two-sided collection $\{X_t\}$ and its finite truncations $X_{m:n}$ are referred to as *time series*.

Once a finite segment has been observed or simulated, its realization

$$x_{m:n} = (x_m, \dots, x_n)$$

is referred to as a *sequence*. Hence, in this thesis, a time series consists of random variables, whereas a sequence consists of their realized values.

Since the thesis uses a Bayesian framework, model parameters may themselves be treated as random variables. A random parameter vector is denoted by Θ , while θ denotes a realized or otherwise fixed parameter value. Let π denote the prior distribution of Θ . When this distribution admits a density, its value at θ is written $\pi(\theta)$. In simulation-based inference, the random prior draws are therefore written as

$$\Theta^{(i)} \sim \pi(\theta),$$

while $\theta^{(i)}$ denotes the realized value of the i -th draw. The superscript i consequently indexes repeated draws or simulated pairs.

2.1.3 Latent variables and the likelihood function

A *latent variable* is a random variable that is included in the statistical model but is not observed. If Y denotes an observed random variable and H denotes a latent random variable, then a model parameterized by θ may be specified through a joint density

$$p(y, h | \theta),$$

while the observed data consist only of y . Here h denotes a possible value of the latent variable, not an observed value. The density of the observed data is then the marginal density

$$f(y; \theta) = \int p(y, h | \theta) dh.$$

For fixed observed data y , the same expression may be viewed as a function of θ and in that case is referred to as the *likelihood function*. This convention will be used when distinguishing between *complete-data* expressions, which involve both y and h , and *observed-data* expressions, which involve only y .

2.1.4 Equality in distribution

Let X and Y be random variables taking values in $\mathbb{R}^d$. We say that X and Y are *equal in distribution* if they induce the same probability law. In this case we write

$$X \stackrel{d}{=} Y.$$

When X and Y have distribution functions F_X and F_Y , this is equivalent to

$$F_X(x) = F_Y(x) \quad \text{for all } x \in \mathbb{R}^d.$$

2.2 Stationarity

Stationarity describes when the probabilistic behaviour of a time series is stable under shifts in time. There are several notions of stationarity, depending on whether one requires stability only of moments or of the full distribution. The definitions of the mean function, autocovariance function, autocorrelation function and stationarity used in this section follow [5, Chapter 1]. In this thesis, unless otherwise specified, *stationarity* will refer to weak stationarity.

2.2.1 Mean, autocovariance and autocorrelation

Let $\{X_t\}$ be a real-valued time series with $\mathbb{E}[X_t^2] < \infty$ for all t . Its *mean function* is

$$m_X(t) = \mathbb{E}[X_t],$$

and its *autocovariance function* is

$$\gamma_X(s, t) = \text{Cov}(X_s, X_t) = \mathbb{E}[(X_s - m_X(s))(X_t - m_X(t))].$$

If the autocovariance depends only on the lag h , we write

$$\gamma_X(h) = \text{Cov}(X_{t+h}, X_t), \quad h \in \mathbb{Z}.$$

If $\gamma_X(0) > 0$, the corresponding *autocorrelation function* is

$$\rho_X(h) = \frac{\gamma_X(h)}{\gamma_X(0)}. \quad (2.1)$$

2.2.2 Weak stationarity

The process $\{X_t\}$ is *weakly stationary* if its mean is constant in time,

$$\mathbb{E}[X_t] = C \quad \text{for all } t \in \mathbb{Z},$$

for some $C \in \mathbb{R}$, and its autocovariance depends only on the lag,

$$\text{Cov}(X_{t+h}, X_t) = \gamma_X(h) \quad \text{for all } t, h \in \mathbb{Z}.$$

Thus, weak stationarity only concerns the first two moments of the process.

2.2.3 Strict stationarity

The process $\{X_t\}$ is *strictly stationary*, if all finite-dimensional distributions are invariant under time shifts. That is, for every $k \in \mathbb{N}$, every $t_1, \dots, t_k \in \mathbb{Z}$, and every $h \in \mathbb{Z}$,

$$(X_{t_1}, \dots, X_{t_k}) \stackrel{d}{=} (X_{t_1+h}, \dots, X_{t_k+h}).$$

If a strictly stationary process has finite second moments, then it is weakly stationary. The converse is not true in general.

2.3 ARMA processes

Autoregressive Moving Average (ARMA) models are a standard class of models for stationary time series with serial dependence. The definitions and basic properties in this section follow [5, Chapter 3]. We first introduce white noise, autoregressive (AR) and moving average (MA) processes before defining the general ARMA(p, q) model. This section is included because the latent log-volatility process in the stochastic volatility model follows an AR(1) process, and because ARMA-type structure will later be used to motivate summary statistics based on the transformed observations $\log(Y_t^2)$.

In [5], AR, MA and ARMA processes are defined as zero-mean processes. In this thesis, we use a slightly broader convention. A stationary process $\{X_t\}$ with mean $\mu_X = \mathbb{E}[X_t]$ will be called an AR, MA or ARMA process if the centered process

$$\{X_t - \mu_X\}$$

satisfies the corresponding zero-mean definition in [5, Definition 3.1.1]. Hence, AR, MA and ARMA processes in this thesis are allowed to have nonzero mean.

2.3.1 White noise innovations

Before defining AR and MA processes, we introduce the driving noise process. A real-valued time series $\{\eta_t\}$ is called *white noise* with mean zero and variance σ^2 , written

$$\{\eta_t\} \sim \text{WN}(0, \sigma^2),$$

if

$$\mathbb{E}[\eta_t] = 0, \quad \text{Var}(\eta_t) = \sigma^2, \quad \text{Cov}(\eta_t, \eta_s) = 0 \quad \text{for } t \neq s.$$

White noise is therefore uncorrelated over time, but not necessarily independent. The symbol η_t is used here for a generic white-noise innovation. Later, in the stochastic volatility model, the same notation is used for the innovation in the latent log-volatility process, which should be read as a particular instance of this generic convention.

2.3.2 Autoregressive processes

A stationary process $\{X_t\}$ with mean μ_X is called an *autoregressive process of order p* , abbreviated AR(p), if

$$X_t - \mu_X = \sum_{i=1}^p \phi_i (X_{t-i} - \mu_X) + \eta_t, \quad (2.2)$$

where $\{\eta_t\} \sim \text{WN}(0, \sigma^2)$. The coefficients $\phi_1, \dots, \phi_p$ are the *autoregressive coefficients*. The centered value $X_t - \mu_X$ therefore depends linearly on its previous p centered values, up to the new innovation η_t .

For $p = 1$, the model becomes

$$X_t - \mu_X = \phi(X_{t-1} - \mu_X) + \eta_t. \quad (2.3)$$

When $|\phi| < 1$, this equation has a stationary solution that can be written in terms of the current and past innovations. To see why, repeated substitution gives, for $m \geq 0$,

$$X_t - \mu_X = \sum_{j=0}^m \phi^j \eta_{t-j} + \phi^{m+1}(X_{t-m-1} - \mu_X).$$

If $\{X_t\}$ is stationary with finite variance, the final term converges to zero in mean square, since

$$\mathbb{E}\left[\{\phi^{m+1}(X_{t-m-1} - \mu_X)\}^2\right] = \phi^{2(m+1)} \text{Var}(X_t) \rightarrow 0.$$

This yields the infinite-order representation

$$X_t = \mu_X + \sum_{j=0}^{\infty} \phi^j \eta_{t-j}. \quad (2.4)$$

The condition $|\phi| < 1$ ensures that the effect of past innovations decays geometrically. In particular, using the uncorrelatedness of the white-noise innovations,

$$\text{Var}\left(\sum_{j=0}^m \phi^j \eta_{t-j}\right) = \sigma^2 \sum_{j=0}^m \phi^{2j},$$

and the right-hand side converges to $\sigma^2/(1 - \phi^2)$ as $m \rightarrow \infty$. The infinite sum in (2.4) is therefore well behaved when $|\phi| < 1$.

Using this representation, we obtain

$$\mathbb{E}[X_t] = \mu_X, \quad \text{Var}(X_t) = \frac{\sigma^2}{1 - \phi^2}. \quad (2.5)$$

Moreover, if $\sigma^2 > 0$, the autocovariance function is

$$\text{Cov}(X_t, X_{t-h}) = \frac{\sigma^2}{1 - \phi^2} \phi^{|h|},$$

which gives the autocorrelation function

$$\rho_X(h) = \phi^{|h|}. \quad (2.6)$$

The parameter ϕ therefore controls the persistence of the process: when $|\phi|$ is close to one, shocks decay slowly.

2.3.3 Moving-average processes

A stationary process $\{X_t\}$ with mean μ_X is called a *moving-average process of order q* , abbreviated MA(q), if

$$X_t - \mu_X = \eta_t + \sum_{j=1}^q \vartheta_j \eta_{t-j},$$

where $\{\eta_t\} \sim \text{WN}(0, \sigma^2)$. The coefficients $\vartheta_1, \dots, \vartheta_q$ are the *moving-average coefficients*. The value of X_t is therefore a linear combination of the current innovation and the previous q innovations.

Finite-order moving-average processes are stationary whenever the white noise process has finite variance, since they are finite linear combinations of uncorrelated random variables with constant variance.

2.3.4 Autoregressive moving-average processes

A stationary process $\{X_t\}$ with mean μ_X is called an *autoregressive moving-average process of order (p, q)* , abbreviated ARMA(p, q), if

$$X_t - \mu_X = \sum_{i=1}^p \phi_i (X_{t-i} - \mu_X) + \eta_t + \sum_{j=1}^q \vartheta_j \eta_{t-j},$$

where $\{\eta_t\} \sim \text{WN}(0, \sigma^2)$. The autoregressive coefficients $\phi_1, \dots, \phi_p$ describe dependence on previous values of the process, while the moving-average coefficients $\vartheta_1, \dots, \vartheta_q$ describe dependence on current and previous innovations.

The corresponding autoregressive and moving-average polynomials are

$$P_\phi(z) = 1 - \phi_1 z - \dots - \phi_p z^p, \quad P_\vartheta(z) = 1 + \vartheta_1 z + \dots + \vartheta_q z^q.$$

The polynomials P_ϕ and P_ϑ are assumed to have no common factors, so that the representation is not redundant.

The AR(p) and MA(q) models are obtained as the special cases $q = 0$ and $p = 0$, respectively.

2.4 Stochastic volatility models

SV models are used for time series whose conditional variance changes over time. In financial applications, the observed series is often a sequence of log returns, where volatility clustering is a central feature. The basic idea goes back to the product-process formulation of Taylor [6], originally published in 1982, which laid the foundation for discrete-time models with a latent stochastic volatility process. The standard log-normal SV model used in this thesis follows the vanilla SV formulation in Hosszejni and Kastner [7, Section 2.1], specialized to centered returns with no regression component. In this model, the observations are conditionally

Gaussian given a latent log-volatility process, while the log-volatility itself follows an AR(1) process.

This section first defines the model and derives the stationary distribution of the latent log-volatility. It then explains why likelihood-based inference is challenging, since the latent path must be integrated out. Finally, it summarizes three approaches to inference for SV parameters: an ARMA(1,1)-based approximation for log-squared returns, a Laplace approximation to the latent-variable likelihood, and Bayesian inference by MCMC.

2.4.1 Model definition and properties

Let Y_t denote the log return at time t . The standard discrete-time log-normal stochastic volatility model is given by

$$Y_t = \exp(H_t/2)\varepsilon_t, \quad (2.7)$$

where $\{\varepsilon_t\}$ is an i.i.d. sequence with

$$\varepsilon_t \sim \mathcal{N}(0, 1).$$

The latent process $\{H_t\}$ is called the *log-volatility* process. Conditional on H_t , the observation Y_t satisfies

$$Y_t | H_t \sim \mathcal{N}(0, \exp(H_t)). \quad (2.8)$$

Thus, H_t controls the conditional variance of the log return.

In the standard log-normal stochastic volatility model, the log-volatility is specified as

$$H_t - \mu = \phi(H_{t-1} - \mu) + \sigma\eta_t, \quad (2.9)$$

where

$$\eta_t \sim \mathcal{N}(0, 1), \quad |\phi| < 1, \quad \sigma > 0.$$

The sequences $\{\varepsilon_t\}$ and $\{\eta_t\}$ are assumed to be independent. The parameter μ is the mean level of the log-volatility process, ϕ controls its persistence, and σ controls the size of the log-volatility innovations.

The equation in (2.9) is the AR(1) model in (2.3), with $X_t = H_t$ and $\mu_X = \mu$. From (2.4), the stationary solution is

$$H_t = \mu + \sigma \sum_{j=0}^{\infty} \phi^j \eta_{t-j}. \quad (2.10)$$

Since the innovations are Gaussian, this implies

$$H_t \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{1 - \phi^2}\right). \quad (2.11)$$

For a finite sample $Y_{1:n} = (Y_1, \dots, Y_n)$, the corresponding log-volatility path is denoted by

$$H_{1:n} = (H_1, \dots, H_n).$$

The model can then be written hierarchically as

$$\begin{aligned} H_1 &\sim \mathcal{N}\left(\mu, \frac{\sigma^2}{1-\phi^2}\right), \\ Y_t | H_t &\sim \mathcal{N}(0, \exp(H_t)), \quad t = 1, \dots, n, \\ H_{t+1} | H_t &\sim \mathcal{N}(\mu + \phi(H_t - \mu), \sigma^2), \quad t = 1, \dots, n-1. \end{aligned} \quad (2.12)$$

Thus, the observations are conditionally independent given the log-volatility path, while temporal dependence is caused by $\{H_t\}$ being an AR(1) process.

In what follows, we collect the model parameters in the vector

$$\theta = (\mu, \phi, \sigma), \quad (2.13)$$

and refer to these as the *SV parameters*.

2.4.2 MLE of SV parameters

Given an observed log return series $y_{1:n}$, ordinary frequentist parameter estimation would typically be based on the maximum likelihood estimator (MLE)

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} f(y_{1:n}; \theta).$$

For SV models, evaluating this likelihood is difficult because the log-volatility path $H_{1:n}$ is latent and must therefore be integrated out of the joint density:

$$f(y_{1:n}; \theta) = \int_{\mathbb{R}^n} p(h_1 | \theta) \prod_{t=2}^n p(h_t | h_{t-1}, \theta) \prod_{t=1}^n p(y_t | h_t, \theta) dh_{1:n}. \quad (2.14)$$

This is an n -dimensional integral, where n is the length of the observed time series.

The difficulty is not caused by the latent process alone, since $H_{1:n}$ follows a Gaussian autoregressive structure. The main issue is the observation equation

$$Y_t | H_t \sim \mathcal{N}(0, \exp(H_t)).$$

The observations are conditionally Gaussian given H_t , but they are not related to H_t through a linear Gaussian measurement equation. Therefore, the standard Kalman filter likelihood for linear Gaussian state-space models cannot be applied directly. Equivalently, using

$$Y_t = \exp(H_t/2)\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1),$$

we obtain

$$\log(Y_t^2) = H_t + \log(\varepsilon_t^2). \quad (2.15)$$

Thus, the log-volatility is observed through an additive non-Gaussian noise term. This makes likelihood-based estimation challenging compared to ordinary Gaussian ARMA models.

2.4.3 Approximate estimation methods for SV parameters

Given $y_{1:n}$ the SV parameters can be approximated in several ways. Two useful alternatives are to transform the observations into a simpler time-series representation, or to approximate (2.14) directly.

A classical approach is to work with the log-squared observations. This idea goes back to the quasi-maximum likelihood approach of Ruiz [8]. Motivated by (2.15), define

$$X_t = \log(Y_t^2). \quad (2.16)$$

Then

$$X_t = H_t + \log(\varepsilon_t^2).$$

The observation noise satisfies

$$\mathbb{E}[\log(\varepsilon_t^2)] = -\gamma - \log 2, \quad \text{Var}(\log(\varepsilon_t^2)) = \frac{\pi^2}{2},$$

where γ denotes the Euler–Mascheroni constant. In particular,

$$\mathbb{E}[X_t] = \mu - \gamma - \log 2. \quad (2.17)$$

Using the AR(1) equation for the log-volatility process in (2.9), and substituting $H_{t-1} = X_{t-1} - \log(\varepsilon_{t-1}^2)$, gives

$$X_t = \mu(1 - \phi) + \phi X_{t-1} + \sigma \eta_t + \log(\varepsilon_t^2) - \phi \log(\varepsilon_{t-1}^2).$$

This equation shows why the log-squared observations are naturally related to an ARMA(1, 1) model. The autoregressive coefficient is the persistence parameter ϕ , while the remaining terms form a noise process with dependence at lag one caused by the measurement noise $\log(\varepsilon_t^2)$. The autocovariance function of $\{X_t\}$ then is

$$\gamma_X(0) = \frac{\sigma^2}{1 - \phi^2} + \frac{\pi^2}{2}, \quad \gamma_X(k) = \frac{\sigma^2}{1 - \phi^2} \phi^k, \quad k \geq 1. \quad (2.18)$$

Hence the autocovariances decay geometrically with rate ϕ , just as in an ARMA(1, 1) process.

If the auxiliary model

$$X_t - m_X = \phi_A(X_{t-1} - m_X) + \eta_t + \vartheta_A \eta_{t-1}, \quad \{\eta_t\} \sim \text{WN}(0, \sigma_A^2), \quad (2.19)$$

is fitted to $\log(Y_t^2)$, then ϕ_A should be interpreted as an estimate of the SV persistence parameter ϕ . The fitted mean gives the corresponding estimate

$$\hat{\mu} \approx \hat{m}_X + \gamma + \log 2.$$

The fitted quantities ϑ_A and σ_A^2 do not have a direct relation to the SV parameters, but they can still be informative about them. The moving-average parameter ϑ_A helps the auxiliary ARMA(1, 1) model match the short-lag autocovariance structure of $\log(Y_t^2)$, which depends on both the $\log\chi_1^2$ measurement noise and the

latent log-volatility process. In particular, the contribution of the measurement noise must be interpreted relative to the latent variance $\sigma^2/(1 - \phi^2)$, so ϑ_A can contain indirect information about ϕ and σ . Similarly, σ_A^2 reflects both latent log-volatility variation and observation noise, and can therefore also be informative about the SV parameters, even though it is not equal to the latent innovation variance σ^2 .

A second approach is to approximate the latent-variable integral directly. Let $u = H_{1:n}$ denote the latent log-volatility path and define the negative joint log-density

$$\mathcal{J}(u; \theta) = -\log p(y_{1:n}, u \mid \theta). \quad (2.20)$$

Then

$$f(y_{1:n}; \theta) = \int_{\mathbb{R}^n} \exp\{-\mathcal{J}(u; \theta)\} du. \quad (2.21)$$

The main idea is to approximate the conditional density of u given $y_{1:n}$ and θ by a Gaussian density centered at its mode. Let

$$\hat{u}(\theta) = \arg \min_u \mathcal{J}(u; \theta), \quad Q(\theta) = \nabla_{uu}^2 \mathcal{J}(\hat{u}(\theta); \theta). \quad (2.22)$$

Approximating $\mathcal{J}(u; \theta)$ by its second-order Taylor expansion around $\hat{u}(\theta)$ gives the Laplace approximation of the likelihood

$$\hat{f}(y_{1:n}; \theta) = (2\pi)^{n/2} |Q(\theta)|^{-1/2} \exp\{-\mathcal{J}(\hat{u}(\theta); \theta)\}. \quad (2.23)$$

The approximate maximum likelihood estimator is obtained by maximizing $\hat{f}(y_{1:n}; \theta)$, or equivalently by minimizing the corresponding approximate negative log-likelihood. This approach is implemented efficiently in Template Model Builder (TMB), which combines automatic differentiation with Laplace approximation for latent-variable models [9]. The R package `stochvolTMB` applies this TMB-based approach to likelihood estimation of stochastic volatility models [10].

The Laplace approximation works well when the conditional density

$$p(u \mid y_{1:n}, \theta) \propto p(y_{1:n} \mid u, \theta) p(u \mid \theta)$$

is close to Gaussian. This is encouraged when the latent prior $p(u \mid \theta)$ is Gaussian and the log-likelihood $\log p(y_{1:n} \mid u, \theta)$ is approximately quadratic in u . For the standard log-normal SV model, the first condition is satisfied because $H_{1:n}$ follows a Gaussian AR(1) process. The second condition is not exact, since

$$\log p(y_t \mid h_t, \theta) = \text{const} - \frac{1}{2} h_t - \frac{1}{2} y_t^2 \exp(-h_t)$$

is not quadratic in h_t . Nevertheless, the Gaussian latent structure makes the approximation useful in the standard model. However, the approximation becomes less reliable for extensions with skewed or heavy-tailed log-volatility innovations, because $p(u \mid \theta)$ then moves the conditional density $p(u \mid y_{1:n}, \theta)$ further away from a Gaussian shape.

2.4.4 Bayesian inference, MCMC for SV parameters and ESS

The approximate estimation methods in Section 2.4.3 are frequentist methods that return point estimates of the SV parameters. In contrast, Bayesian inference assigns a prior distribution to the random parameter vector Θ , whose realized value is $\theta = (\mu, \phi, \sigma)$. Given an observed return series $y_{1:n}$, inference is based on the posterior density

$$\pi(\theta, h_{1:n} \mid y_{1:n}) \propto \left\{ \prod_{t=1}^n p(y_t \mid h_t, \theta) \right\} p(h_{1:n} \mid \theta) \pi(\theta). \quad (2.24)$$

The marginal posterior distribution of Θ is obtained by marginalizing over the log-volatility path,

$$\pi(\theta \mid y_{1:n}) = \int_{\mathbb{R}^n} \pi(\theta, h_{1:n} \mid y_{1:n}) dh_{1:n}. \quad (2.25)$$

MCMC avoids evaluating this integral directly. Instead, it constructs a Markov chain whose stationary distribution is the joint posterior in (2.24). If

$$\{\theta^{(m)}, h_{1:n}^{(m)}\}_{m=1}^M$$

denotes the retained draws from this chain, then

$$\{\theta^{(m)}\}_{m=1}^M$$

are treated as approximate draws from the marginal posterior distribution $\pi(\theta \mid y_{1:n})$, once the distribution of the chain is sufficiently close to its stationary distribution. Hence, posterior summaries for the SV parameters are obtained by retaining the θ -draws and discarding the sampled log-volatility paths. For general background on MCMC, see Robert and Casella [11].

A generic MCMC implementation is based on update rules, often built from proposal distributions and accept–reject steps, that move the chain through the joint space of θ and $h_{1:n}$. These rules are chosen so that repeated updates make the chain visit regions with high posterior probability more often than regions with low posterior probability. A schematic version is shown in Algorithm 1.

Algorithm 1 Schematic MCMC sampler for SV posterior inference

Require: Observed returns $y_{1:n}$, prior density $\pi(\theta)$, number of retained draws M , burn-in B , and update rules for $h_{1:n}$ and θ

- 1: Initialize $\theta^{(-B-1)}$ and $h_{1:n}^{(-B-1)}$
- 2: **for** $m = -B, \dots, M$ **do**
- 3: Update the log-volatility path using the current values of $y_{1:n}$, $h_{1:n}^{(m-1)}$, and $\theta^{(m-1)}$
- 4: Update the SV parameters using the current values of $y_{1:n}$, $h_{1:n}^{(m)}$, and $\theta^{(m-1)}$
- 5: **if** $m > 0$ **then**
- 6: Store $\theta^{(m)}$
- 7: **end if**
- 8: **end for**
- 9: Use the stored draws as samples from $\pi(\theta \mid y_{1:n})$

Algorithm 1 highlights the implementation choices that must be fixed when using MCMC: the prior distribution, the number of iterations, the burn-in period, and the update rules used to move through the posterior. A naive implementation may update the latent log-volatilities one component at a time, but this can mix slowly because the components of $H_{1:n}$ are strongly dependent through the AR(1) structure.

In this thesis, the MCMC baseline is instead computed using the R package `stochvol`, described by Kastner [12]. The package implements Bayesian inference for SV models using MCMC and is preferable to a naive sampler because it uses update rules tailored to the SV state-space structure. In particular, the latent log-volatility path is updated in a block rather than one component at a time, which typically reduces correlation between successive draws and improves sampling efficiency.

The prior form used in the Bayesian SV models in this thesis follows the usual vanilla SV prior structure in `stochvol`. The components of Θ are assumed a priori independent, so that

$$\pi(\theta) = \pi_\mu(\mu)\pi_\phi(\phi)\pi_{\sigma^2}(\sigma^2).$$

We assume

$$\mu \sim \mathcal{N}(m_\mu, s_\mu^2), \quad \frac{\phi + 1}{2} \sim \text{Beta}(a_\phi, b_\phi), \quad \sigma^2 \sim B_\sigma \chi_1^2, \quad (2.26)$$

where $B_\sigma > 0$ is a scale hyperparameter. The stationary initial distribution is used for the log-volatility. The hyperparameters

$$m_\mu, s_\mu, a_\phi, b_\phi, B_\sigma$$

are chosen according to the data setting, while the prior families are kept fixed across the Bayesian SV models considered in this thesis.

Since MCMC draws are generally autocorrelated, M post-burn-in draws need not contain as much information as M independent draws. For a scalar stationary

chain $Z_{1:M}$, the effective sample size (ESS) is commonly expressed as

$$\text{ESS}(Z_{1:M}) = \frac{M}{1 + 2 \sum_{h=1}^{\infty} \rho_Z(h)}, \quad (2.27)$$

where $\rho_Z(h)$ is the autocorrelation at lag h , defined in (2.1). In practice, the infinite sum is estimated and truncated. Persistent positive autocorrelation increases the denominator and reduces the effective number of independent draws.

The Monte Carlo standard error (MCSE) is the standard deviation of an MCMC estimator across repeated finite chains. Let

$$\mu_Z = \mathbb{E}(Z), \quad v_Z = \text{Var}(Z),$$

and define

$$\hat{\mu}_Z = \frac{1}{M} \sum_{m=1}^M Z_m, \quad U_m = (Z_m - \mu_Z)^2, \quad \hat{v}_Z = \frac{1}{M} \sum_{m=1}^M U_m.$$

In practice, μ_Z in the variance estimator is replaced by $\hat{\mu}_Z$, which gives the same large-sample interpretation. The corresponding MCSEs may be approximated by

$$\text{SD}(\hat{\mu}_Z) \approx \sqrt{\frac{v_Z}{\text{ESS}(Z_{1:M})}}, \quad \text{SD}(\hat{v}_Z) \approx \sqrt{\frac{\text{Var}((Z - \mu_Z)^2)}{\text{ESS}(U_{1:M})}}.$$

See Flegel and Jones [13] for a general treatment of MCSE estimation for MCMC averages.

Dividing $\text{SD}(\hat{\mu}_Z)$ by $|\mu_Z|$ would give a relative MCSE, but since μ_Z might be unknown we instead use the posterior standard deviation as a reference scale, which gives

$$\frac{\text{SD}(\hat{\mu}_Z)}{\sqrt{v_Z}} \approx \frac{1}{\sqrt{\text{ESS}(Z_{1:M})}}. \quad (2.28)$$

For the variance estimator, if Z is approximately Gaussian, then

$$\text{Var}((Z - \mu_Z)^2) = 2v_Z^2.$$

Scaling the MCSE by the true variance v_Z therefore gives

$$\frac{\text{SD}(\hat{v}_Z)}{v_Z} \approx \sqrt{\frac{2}{\text{ESS}(U_{1:M})}}. \quad (2.29)$$

Equations (2.28) and (2.29) therefore allow the standardized MCSE of the estimated mean and the relative MCSE of the estimated variance to be approximated directly from their corresponding ESS values.

2.5 Neural posterior estimation

For the stochastic volatility (SV) model considered in this thesis, direct likelihood-based inference is difficult because the log-volatility is latent and must be integrated out. At the same time, simulation from the model is straightforward once the parameter vector θ is fixed. This motivates the use of simulation-based inference methods, which replace direct likelihood evaluation by repeated simulation from the model; see, for example, Cranmer et al. [14] for a general introduction.

In this thesis, we use neural posterior estimation (NPE) to estimate the SV parameters. NPE is a Bayesian simulation-based inference method where a machine learning model is trained on simulated parameter-data pairs to approximate the posterior distribution. Early neural posterior methods were developed by Papamakarios and Murray [15], with later sequential extensions such as Lueckmann et al. [16] and Greenberg et al. [17]. The approach used here is closest to the amortized variational formulation of Maceda et al. [3]. More specifically, the model learns how the posterior distribution over a transformation of θ from (2.13) changes as a function of the observed time series. Instead of returning a single point estimate, the method returns a distribution over plausible parameter values given the observed data.

The rest of this section makes this procedure concrete. We first formulate NPE as an amortized posterior approximation problem. We then derive the objective function used to fit the network weights W from simulated parameter-data pairs. Finally, we specify the parametric posterior family, describe the machine learning model T_W , and give an interpretation of the resulting objective function.

2.5.1 Neural posterior estimation as amortized inference

A central feature of NPE is that inference is *amortized*. This means that the computational cost is mainly paid during training, where many simulated data sets are used to learn a general mapping from data to posterior approximations. Once this mapping has been learned, posterior inference for a new observed time series only requires evaluating the trained network. In this way the same trained model can be reused for new observations from the same model class.

In the model considered here, the parameter vector θ takes values in a parameter space Ω . Rather than approximating the posterior directly on Ω , we first transform the parameter vector to a space where standard parametric distribution families can be used more conveniently. Let

$$G : \Omega \rightarrow \tilde{\Omega}$$

be a bijective differentiable transformation with differentiable inverse, and define

$$\tilde{\theta} = G(\theta).$$

The transformed parameter $\tilde{\theta}$ takes values in $\tilde{\Omega}$, which is chosen to be an unconstrained Euclidean parameter space. Posterior statements about the original parameter θ can then be obtained by mapping back through G^{-1} .

Following the variational amortized NPE approach of Maceda et al. [3], we approximate the posterior distribution of the transformed parameter by a chosen parametric family. Let

$$q(\tilde{\theta}; \lambda), \quad \lambda \in \Lambda,$$

denote this family, where Λ is the parameter space for the distributional parameters of q . For an observed time series $y_{1:n} \in \mathbb{R}^n$, the aim is to approximate the transformed posterior density by a member of this family,

$$p(\tilde{\theta} \mid y_{1:n}) \approx q(\tilde{\theta}; \lambda), \quad \tilde{\theta} \in \tilde{\Omega}.$$

The distributional parameters λ should depend on the observed data, since different time series generally induce different posterior distributions.

In NPE, this dependence is represented directly by a neural network. Let

$$T_W : \mathbb{R}^n \rightarrow \Lambda$$

be a neural network with weights W , where $T_W(y_{1:n})$ denotes the posterior-family parameters predicted from the observed time series. The amortized posterior approximation is then written as

$$p(\tilde{\theta} \mid y_{1:n}) \approx q(\tilde{\theta}; T_W(y_{1:n})), \quad \tilde{\theta} \in \tilde{\Omega}. \quad (2.30)$$

Thus, the network does not output a point estimate of θ , but the parameters of an approximate posterior distribution over the transformed parameter $\tilde{\theta}$.

The goal is to choose the weights W and a suitable parametric family q such that the density $q(\tilde{\theta}; T_W(y_{1:n}))$ gives a good approximation to the true transformed posterior $p(\tilde{\theta} \mid y_{1:n})$ for data sets generated from the model. The remaining questions are therefore how the weights W should be chosen from simulated parameter-data pairs, and which parametric family q should be used for the posterior approximation. This leads to the empirical posterior log-score objective derived in the next subsection, followed by a discussion of the chosen posterior family.

2.5.2 From simulated pairs to a posterior log-score objective

Let $\pi(\theta)$ denote the prior density of the SV parameters. Although the marginal sampling density $f(y_{1:n}; \theta)$ is generally difficult to evaluate, samples from it can be generated by first simulating the latent states $H_{1:n}$ and then simulating $Y_{1:n}$ conditional on those states.

Let G denote the one-to-one parameter transformation specified in the next subsection, and write

$$\tilde{\Theta} = G(\Theta).$$

We let $\tilde{\pi}(\tilde{\theta})$ denote the density induced by the prior $\pi(\theta)$ under this transformation. Training pairs are generated according to

$$\Theta^{(i)} \sim \pi(\theta), \quad Y_{1:n}^{(i)} \mid \Theta^{(i)} \sim f(\cdot; \Theta^{(i)}), \quad \tilde{\Theta}^{(i)} = G(\Theta^{(i)}),$$

for $i = 1, \dots, N$.

Equivalently, the transformed parameter–data pairs are independent draws from the joint simulation density

$$p_{\text{sim}}(\tilde{\theta}, y_{1:n}) = \tilde{\pi}(\tilde{\theta})f(y_{1:n}; G^{-1}(\tilde{\theta})). \quad (2.31)$$

After simulation, their realized values form the training set

$$\mathcal{D}_N = \left\{ \left(\tilde{\theta}^{(i)}, y_{1:n}^{(i)} \right) \right\}_{i=1}^N. \quad (2.32)$$

Integrating the simulation density over the transformed parameters gives the *prior predictive density*

$$\begin{aligned} m(y_{1:n}) &= \int p_{\text{sim}}(\tilde{\theta}, y_{1:n}) d\tilde{\theta} \\ &= \int f(y_{1:n}; \theta) \pi(\theta) d\theta. \end{aligned} \quad (2.33)$$

The simulation density can therefore also be factorized as

$$p_{\text{sim}}(\tilde{\theta}, y_{1:n}) = p(\tilde{\theta} \mid y_{1:n})m(y_{1:n}),$$

where $p(\tilde{\theta} \mid y_{1:n})$ is the posterior density on the transformed parameter space. The prior predictive density is generally intractable, but it does not depend on the network weights W .

For fixed weights W , the transformed posterior density is approximated by (2.30). Substituting this approximation into the factorization above gives

$$p_{\text{sim}}(\tilde{\theta}, y_{1:n}) \approx q(\tilde{\theta}; T_W(y_{1:n}))m(y_{1:n}).$$

Treating the simulated pairs in $\mathcal{D}_N$ as fixed observations gives the approximate joint likelihood

$$Q_N(W) = \prod_{i=1}^N q(\tilde{\theta}^{(i)}; T_W(y_{1:n}^{(i)}))m(y_{1:n}^{(i)}).$$

Taking logarithms and dividing by N yields

$$\frac{1}{N} \log Q_N(W) = \hat{J}_N(W) + \frac{1}{N} \sum_{i=1}^N \log m(y_{1:n}^{(i)}),$$

where

$$\hat{J}_N(W) = \frac{1}{N} \sum_{i=1}^N \log q(\tilde{\theta}^{(i)}; T_W(y_{1:n}^{(i)})). \quad (2.34)$$

Since the second term does not depend on W , maximizing $Q_N(W)$ is equivalent to maximizing $\hat{J}_N(W)$. This empirical posterior log-score rewards networks that assign high posterior density to the transformed parameter values that generated the corresponding simulated data sets.

2.5.3 Choice of posterior family and parameter transformation

The objective in (2.34) does not yet specify the posterior family q . In this thesis, we approximate the marginal posterior distribution of each SV parameter rather than the full joint posterior distribution. A marginal posterior is obtained by integrating the remaining parameters out of the joint posterior. For example, $p(\mu \mid y_{1:n})$ is obtained by integrating out ϕ and σ .

This removes the need to estimate the posterior covariance structure. The network instead estimates one marginal posterior approximation for each model parameter. An additional parameter in a richer SV model can then be handled by adding another marginal output rather than extending a full covariance parameterization.

We use Gaussian marginal approximations because they are determined by a mean and a variance and therefore give a simple parametric output. They also have a large-sample motivation: under suitable regularity conditions, the Bernstein–von Mises theorem suggests that posterior distributions become approximately Gaussian. Since some SV parameters are constrained, the Gaussian approximations are defined after transforming all parameters to an unconstrained space.

Recall that

$$\theta = (\mu, \phi, \sigma),$$

with

$$\mu \in \mathbb{R}, \quad \phi \in (-1, 1), \quad \sigma > 0.$$

The natural parameter space is therefore

$$\Omega = \mathbb{R} \times (-1, 1) \times (0, \infty).$$

We take the transformed parameter space to be

$$\tilde{\Omega} = \mathbb{R}^3$$

and specify the transformation introduced in the preceding subsection as

$$G(\mu, \phi, \sigma) = (\mu, 2 \operatorname{arctanh}(\phi), \log \sigma) = (\mu, \psi, \rho), \quad (2.35)$$

where

$$\psi = 2 \operatorname{arctanh}(\phi), \quad \rho = \log \sigma.$$

The transformation of ϕ can equivalently be written as

$$2 \operatorname{arctanh}(\phi) = \log \left(\frac{1 + \phi}{1 - \phi} \right) = \operatorname{logit} \left(\frac{\phi + 1}{2} \right).$$

It is therefore a logit transformation after mapping $(-1, 1)$ to $(0, 1)$. The logarithmic transformation maps $\sigma \in (0, \infty)$ to the real line, while μ is already unconstrained. The inverse transformation is

$$G^{-1}(\mu, \psi, \rho) = \left(\mu, \tanh \left(\frac{\psi}{2} \right), \exp(\rho) \right). \quad (2.36)$$

For an observed time series $y_{1:n}$, the transformed marginal posteriors are approximated by

$$\mu \mid y_{1:n} \approx \mathcal{N}(m_1, v_1), \quad \psi \mid y_{1:n} \approx \mathcal{N}(m_2, v_2), \quad \rho \mid y_{1:n} \approx \mathcal{N}(m_3, v_3),$$

where

$$m_{1:3} = (m_1, m_2, m_3) \in \mathbb{R}^3, \quad v_{1:3} = (v_1, v_2, v_3) \in (0, \infty)^3.$$

The dependence of these posterior parameters on $y_{1:n}$ is suppressed here for notational convenience.

Let

$$\tilde{\theta} = (\tilde{\theta}_1, \tilde{\theta}_2, \tilde{\theta}_3) = (\mu, \psi, \rho),$$

and let

$$\varphi(x; m, v) = \frac{1}{\sqrt{2\pi v}} \exp \left\{ -\frac{(x - m)^2}{2v} \right\}$$

denote the univariate Gaussian density with mean m and variance v . The three marginal approximations can then be collected in the product density

$$q(\tilde{\theta}; m_{1:3}, v_{1:3}) = \prod_{j=1}^3 \varphi(\tilde{\theta}_j; m_j, v_j).$$

Equivalently,

$$q(\tilde{\theta}; m_{1:3}, v_{1:3}) = \mathcal{N}_3(\tilde{\theta}; m_{1:3}, \text{diag}(v_1, v_2, v_3)).$$

The product form is used to collect the three marginal approximations into one posterior score. It does not model posterior dependence between the transformed parameters and should therefore be interpreted as a collection of marginal Gaussian approximations rather than as an approximation of the full joint posterior dependence structure.

2.5.4 Marginal Gaussian loss and shared representation

The chosen posterior family reduces the NPE problem to estimating the data-dependent mean and variance of each transformed marginal posterior. For an observed time series $y_{1:n} \in \mathbb{R}^n$, the network output is

$$\begin{aligned} T_W(y_{1:n}) &= (T_{W,1}(y_{1:n}), T_{W,2}(y_{1:n}), T_{W,3}(y_{1:n})) \in (\mathbb{R} \times (0, \infty))^3, \\ T_{W,j}(y_{1:n}) &= (\widehat{m}_{W,j}(y_{1:n}), \widehat{v}_{W,j}(y_{1:n})) \in \mathbb{R} \times (0, \infty), \quad j = 1, 2, 3. \end{aligned} \quad (2.37)$$

Here, $\widehat{m}_{W,j}(y_{1:n})$ and $\widehat{v}_{W,j}(y_{1:n})$ are the predicted mean and variance of the j th transformed marginal posterior.

These outputs define the approximate posterior density

$$q_W(\tilde{\theta}; y_{1:n}) = \prod_{j=1}^3 \varphi(\tilde{\theta}_j; \widehat{m}_{W,j}(y_{1:n}), \widehat{v}_{W,j}(y_{1:n})). \quad (2.38)$$

Substituting this density into (2.34) gives

$$\hat{J}_N(W) = \sum_{j=1}^3 \hat{J}_{N,j}(W),$$

where

$$\hat{J}_{N,j}(W) = \frac{1}{N} \sum_{i=1}^N \log \varphi \left(\tilde{\theta}_j^{(i)}; \hat{m}_{W,j}(y_{1:n}^{(i)}), \hat{v}_{W,j}(y_{1:n}^{(i)}) \right).$$

Inserting the Gaussian density yields

$$\hat{J}_{N,j}(W) = -\frac{1}{2} \log(2\pi) - L_{N,j}(W),$$

where the j th marginal loss is

$$L_{N,j}(W) = \frac{1}{2N} \sum_{i=1}^N \left[\log \hat{v}_{W,j}(y_{1:n}^{(i)}) + \frac{\left(\tilde{\theta}_j^{(i)} - \hat{m}_{W,j}(y_{1:n}^{(i)}) \right)^2}{\hat{v}_{W,j}(y_{1:n}^{(i)})} \right]. \quad (2.39)$$

The overall empirical loss is the mean of the three marginal losses:

$$\begin{aligned} L_N(W) &= \frac{1}{3} \sum_{j=1}^3 L_{N,j}(W) \\ &= \frac{1}{6N} \sum_{i=1}^N \sum_{j=1}^3 \left[\log \hat{v}_{W,j}(y_{1:n}^{(i)}) + \frac{\left(\tilde{\theta}_j^{(i)} - \hat{m}_{W,j}(y_{1:n}^{(i)}) \right)^2}{\hat{v}_{W,j}(y_{1:n}^{(i)})} \right]. \end{aligned} \quad (2.40)$$

The empirical log-score and the loss are related by

$$\hat{J}_N(W) = -\frac{3}{2} \log(2\pi) - 3L_N(W).$$

Minimizing $L_N(W)$ is therefore equivalent to maximizing the empirical NPE objective.

The two terms in the loss balance posterior location and posterior uncertainty. A smaller predicted variance gives a sharper posterior approximation, but it also increases the penalty for errors in the predicted mean. A larger predicted variance reduces the standardized squared-error term but is penalized through the log-variance term. The loss therefore rewards posterior approximations whose predicted locations and uncertainties agree with the generating parameter–data pairs.

Although $L_N(W)$ is constructed from three marginal losses, the marginal approximations are not fitted by three separate networks. Each parameter has its own output head, while all heads use the same learned representation of $y_{1:n}$. The shared representation is therefore updated using information from all three parameters. This avoids learning a separate feature extractor for each marginal

posterior and allows features that are informative about several parameters to be reused. For example, ϕ and σ both affect the temporal dependence and variation of the latent log-volatility process, so some features of the observed series may be informative about both parameters.

2.6 Neural network architectures

The previous section formulated NPE in terms of a neural network T_W , which maps an observed time series to the parameters of approximate marginal posteriors. We now describe how this map is represented in practice. Neural networks are flexible parametric function approximators whose weights are learned by minimizing a loss function over training data; see, for example, Goodfellow et al. [18] for a general introduction. In the present setting, the training data are the simulated parameter–data pairs $\mathcal{D}_N$ from (2.32), and the network weights are learned by minimizing the posterior loss $L_N(W)$ defined in (2.40). Unless otherwise stated, all hidden layers in the neural networks used in this thesis use the rectified linear unit activation function.

We consider two neural-network architectures for the map T_W . The first uses summary statistics of the observed time series as input to a multilayer perceptron encoder. The second uses the transformed sequence

$$\text{logsq}(y_{1:n}) = (\log(y_1^2 + \kappa), \dots, \log(y_n^2 + \kappa)) \quad (2.41)$$

as input to a temporal convolutional encoder with residual connections. The second architecture is motivated by residual networks, introduced by He et al. [19], and by temporal convolutional networks for sequence modelling, as studied by Bai et al. [20].

The two architectures differ in how they extract information from the observed time series, but they share the same output structure. In both cases, the encoder maps the chosen input representation of $y_{1:n}$ to a feature vector z . This feature vector is then passed to three parameter-specific prediction heads, one for each transformed stochastic volatility (SV) parameter. Each head returns the mean and of one marginal Gaussian posterior approximation.

The rest of this section is organized as follows. We first describe the shared encoder–head structure and the multilayer perceptron heads used by both architectures. We then describe the summary-statistic multilayer perceptron and the temporal convolutional residual architecture in detail.

2.6.1 Encoder–head structure and multilayer perceptron heads

All neural network architectures used in this thesis implement T_W through an encoder–head structure. Let

$$x(y_{1:n}) \in \mathcal{X} \quad (2.42)$$

denote the *input representation* of the observed time series passed to the network. The *encoder* is a parametric map

$$E_{w_0} : \mathcal{X} \rightarrow \mathbb{R}^{d_E},$$

where w_0 denotes the encoder weights. Its output is the *feature vector*

$$z = E_{w_0}(x(y_{1:n})). \quad (2.43)$$

This feature vector is interpreted as a learned representation of the observed time series.

The feature vector z is then passed to parameter-specific prediction heads. In this context, a *prediction head* is the part of the neural network that maps a shared feature representation to a specified component of the final network output. Thus, the encoder produces the common representation z , while each head produces the mean and variance for one marginal posterior approximation.

Since the transformed SV parameter vector has three components, the network has three prediction heads. For $j = 1, 2, 3$, the j th head is a parametric map

$$h_{w_j} : \mathbb{R}^{d_E} \rightarrow \mathbb{R} \times (0, \infty),$$

where w_j denotes the head weights. The first output component is the predicted posterior mean, while the second output component is the predicted posterior variance for the j th transformed parameter. Recalling that the neural network has outputs of the form (2.37), the complete network output can be written as

$$T_W(y_{1:n}) = (h_{w_1}(z), h_{w_2}(z), h_{w_3}(z)).$$

The full set of trainable weights is therefore

$$W = (w_0, w_1, w_2, w_3),$$

where w_0 contains the encoder weights and w_1, w_2, w_3 contain the weights of the three prediction heads. The encoder is shared across all transformed parameters, while the final maps from the shared feature vector to posterior means and variances are parameter-specific.

The two architectures considered in this thesis differ only in the encoder. In the summary-statistic architecture, $x(y_{1:n})$ is a vector of summary statistics and E_{w_0} is a multilayer perceptron. In the temporal convolutional architecture, $x(y_{1:n})$ is the transformed sequence $\text{logsq}(y_{1:n})$ defined in (2.41), and E_{w_0} is a temporal convolutional encoder with residual connections. The prediction heads have the same structure in both architectures.

Each prediction head is implemented as a multilayer perceptron. A MLP is a neural network formed by composing fully connected affine transformations with nonlinear activation functions. Fully connected means that each layer receives all outputs from the previous layer. The nonlinear activation functions are essential: without nonlinearities, a composition of affine transformations would still be affine.

For an input vector $x \in \mathbb{R}^{d_0}$, an MLP with L hidden layers and one output layer can be written recursively as

$$h^{(0)} = x, \quad h^{(\ell)} = \Phi_\ell \left(A^{(\ell)} h^{(\ell-1)} + b^{(\ell)} \right), \quad \ell = 1, \dots, L+1, \quad (2.44)$$

where

$$A^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad b^{(\ell)} \in \mathbb{R}^{d_\ell}.$$

Here, $A^{(\ell)}$ and $b^{(\ell)}$ are learnable parameters, Φ_ℓ is the activation function in layer ℓ , and $h^{(L+1)}$ is the output. Thus, the MLP defines a parametric function

$$h(\cdot; \{\Phi_\ell\}_{\ell=1}^{L+1}, \omega), \quad \omega = \{(A^{(\ell)}, b^{(\ell)})\}_{\ell=1}^{L+1}.$$

For the hidden layers $\ell = 1, \dots, L$, we use the *rectified linear unit activation function*

$$\text{ReLU}(x) = \max\{0, x\}, \quad x \in \mathbb{R},$$

applied elementwise for vector-valued inputs. For the prediction heads, the output layer must produce one unconstrained posterior mean and one positive posterior variance. The final activation function is therefore chosen to be

$$\Phi_{\text{out}}(u_1, u_2) = (u_1, \kappa_v + \text{softplus}(u_2)), \quad (2.45)$$

where

$$\text{softplus}(x) = \log(1 + \exp(x)), \quad x \in \mathbb{R}.$$

The first component is left unchanged because posterior means are unconstrained. The second component is transformed to ensure a positive posterior variance. The constant $\kappa_v \geq 0$ is used as a numerical lower bound for the predicted variance.

The j th prediction head is therefore the instance of the MLP parametric family given by

$$h_{w_j}(z) = h(z; \{\Phi_\ell\}_{\ell=1}^{L+1}, w_j), \quad (2.46)$$

with

$$\Phi_\ell = \text{ReLU}, \quad \ell = 1, \dots, L,$$

and

$$\Phi_{L+1} = \Phi_{\text{out}}.$$

Since each head outputs two quantities, the final affine transformation

$$(A_{w_j}^{(L+1)}, b_{w_j}^{(L+1)}) \in w_j$$

has

$$A_{w_j}^{(L+1)} \in \mathbb{R}^{2 \times d_L}, \quad b_{w_j}^{(L+1)} \in \mathbb{R}^2, \quad j = 1, \dots, 3.$$

2.6.2 Summary-statistic MLP encoder

The first architecture represents the observed time series through a fixed vector of summary statistics. The purpose of this representation is to provide the network with features that are directly related to the location, variance and dependence structure of the transformed observations. The encoder is then an MLP applied to this summary vector.

Let

$$x_{1:n} = \text{logsq}(y_{1:n})$$

denote the transformed sequence defined in (2.41). This transform is motivated by the approximate estimation methods discussed in Section 2.4.3, where the idealized transform $\log(Y_t^2)$ is used to obtain a noisy observation of the latent log-volatility process. A $\kappa > 0$ is added before taking the logarithm to ensure numerical stability.

Let $0 < p_1 < \dots < p_{n_Q} < 1$ denote a chosen collection of quantile levels, and let n_γ denote the number of autocovariance ratios included. We also define

$$g(u) = 2 \operatorname{arctanh}(u), \quad u \in (-1, 1),$$

which is the same transformation as the one used for the persistence parameter ϕ . In implementation, empirical inputs to g are regularized when necessary to ensure that they lie in its domain.

The summary statistic map

$$S : \mathbb{R}^n \rightarrow \mathbb{R}^{d_S} \tag{2.47}$$

is defined by

$$S(y_{1:n}) = \left(\bar{x}, \widehat{Q}_x(p_1), \dots, \widehat{Q}_x(p_{n_Q}), \tilde{r}_1, \dots, \tilde{r}_{n_\gamma}, \right. \\ \left. g(\widehat{\phi}_A), g(\widehat{\theta}_A), \log \widehat{\sigma}_A, \frac{1}{2} \log \widehat{v}_x, \log \widehat{\text{MAD}}_x, \widehat{\rho}_{\text{plug}} \right),$$

where $d_S = n_Q + n_\gamma + 7$. The individual components are defined and motivated below.

Sample mean $\bar{x}$: The sample mean of the transformed observations is defined by

$$\bar{x} = \frac{1}{n} \sum_{t=1}^n x_t.$$

It captures the location of the transformed observations. As earlier mentioned in (2.17), the idealized log-square process satisfies

$$\mathbb{E}[X_t] = \mu - \gamma - \log 2.$$

Hence the mean of the transformed observations contains direct information about the level parameter μ .

Empirical quantiles $\widehat{Q}_x(p_1), \dots, \widehat{Q}_x(p_{n_Q})$: For $0 < p < 1$, let $\widehat{Q}_x(p)$ denote the empirical p -quantile of $x_{1:n}$. The empirical quantiles summarize the marginal distribution of the transformed observations. They provide information about spread, skewness and tail behavior that may not be captured well by the mean and variance alone.

Autocovariance ratios $\tilde{r}_1, \dots, \tilde{r}_{n_\gamma}$: The empirical autocovariance of $x_{1:n}$ at lag k is defined by

$$\widehat{\gamma}_x(k) = \frac{1}{n} \sum_{t=k+1}^n (x_t - \bar{x})(x_{t-k} - \bar{x}), \quad k = 0, 1, \dots, n_\gamma.$$

The autocovariance-ratio features are motivated by (2.18). For $k = 1, \dots, K$, define

$$\widehat{r}_k = \frac{\widehat{\gamma}_x(k)}{\widehat{\gamma}_x(k-1)}$$

and

$$\tilde{r}_k = g(\widehat{r}_k).$$

For the idealized log-square process, the positive-lag autocovariances satisfy

$$\frac{\gamma_X(k+1)}{\gamma_X(k)} = \phi, \quad k \geq 1.$$

Hence, autocovariance ratios can be viewed as rough persistence summaries. However, the first ratio is not generally equal to ϕ , since $\gamma_X(0)$ is inflated by the $\log\chi_1^2$ observation noise. Indeed,

$$\frac{\gamma_X(1)}{\gamma_X(0)} = \frac{\frac{\sigma^2}{1-\phi^2} \phi}{\frac{\sigma^2}{1-\phi^2} + \frac{\pi^2}{2}},$$

which is attenuated toward zero compared with ϕ . Thus, $\widehat{r}_1$ should not be interpreted as a direct estimate of ϕ , although it can still contain useful information about the relative strength of the latent volatility signal and the observation noise.

Auxiliary ARMA summaries $g(\widehat{\phi}_A)$, $g(\widehat{\vartheta}_A)$ and $\log \widehat{\sigma}_A$: The next group of features is obtained by fitting the auxiliary ARMA(1, 1) model in (2.19) to the transformed observations. The fitted autoregressive coefficient $\widehat{\phi}_A$ is included because it is directly related to the SV persistence parameter ϕ . It is transformed as $g(\widehat{\phi}_A)$ to place it on the same unconstrained scale as the target parameter ψ .

The fitted moving-average coefficient $\widehat{\vartheta}_A$ and innovation standard deviation $\widehat{\sigma}_A$ do not have the same direct relation to the SV parameters. They are nevertheless informative, since they help describe how the auxiliary ARMA(1, 1) model represents $\log(Y_t^2)$. In particular, $\widehat{\vartheta}_A$ captures part of the short-lag autocovariance structure produced by the combination of latent log-volatility variation and $\log\chi_1^2$ measurement noise. Since the relative contribution of these two sources of variation depends on the SV parameters, especially ϕ and σ , $g(\widehat{\vartheta}_A)$ can provide

useful information to the neural network. Similarly, $\log \hat{\sigma}_A$ summarizes the innovation scale in the fitted auxiliary model, and can be informative about the overall variation in $\log(Y_t^2)$.

The ARMA-based summaries use more of the dependence structure of the transformed time series than individual autocovariance ratios, and may therefore provide more stable auxiliary information. The drawback is computational, since an ARMA(1,1) model must be fitted separately for each simulated time series when constructing the summary-statistic data set.

Scale summaries $\frac{1}{2} \log \hat{v}_x$ and $\log \widehat{\text{MAD}}_x$: The empirical variance of the transformed observations is defined by

$$\hat{v}_x = \frac{1}{n} \sum_{t=1}^n (x_t - \bar{x})^2.$$

Thus,

$$\frac{1}{2} \log \hat{v}_x$$

is the logarithm of the empirical standard deviation of $x_{1:n}$. This feature summarizes the overall variation in the transformed observations, but it is sensitive to extreme values because large deviations from the sample mean enter quadratically.

The empirical median absolute deviation is defined by

$$\widehat{\text{MAD}}_x = \text{median}_{1 \leq t \leq n} |x_t - \hat{Q}_x(1/2)|.$$

The feature

$$\log \widehat{\text{MAD}}_x$$

therefore provides a complementary scale summary based on the central part of the empirical distribution. Since it is computed from medians rather than squared deviations from the mean, it is less affected by a small number of very large or very small transformed observations. This is useful for $\log(Y_t^2)$, where extreme values can arise from both the latent log-volatility process and the $\log-\chi_1^2$ observation noise. In implementation, the arguments of these logarithms are regularized to avoid invalid values.

Plug-in volatility-of-volatility proxy $\hat{\rho}_{\text{plug}}$: The plug-in feature $\hat{\rho}_{\text{plug}}$ is motivated by rearranging (2.18). Since

$$\gamma_X(0) = \frac{\sigma^2}{1 - \phi^2} + \frac{\pi^2}{2},$$

we have formally

$$\log \sigma = \frac{1}{2} \left[\log \left(\gamma_X(0) - \frac{\pi^2}{2} \right) + \log(1 - \phi^2) \right].$$

This motivates the plug-in estimate

$$\hat{\rho}_{\text{plug}} = \frac{1}{2} \left[\log \left(\hat{v}_x - \frac{\pi^2}{2} \right) + \log(1 - \hat{\phi}_{\text{proxy}}^2) \right],$$

where $\hat{\phi}_{\text{proxy}}$ is a chosen persistence proxy, for example the fitted ARMA autoregressive coefficient or an autocovariance ratio. This feature is intended as a rough direct proxy for the transformed volatility-of-volatility parameter $\rho = \log \sigma$. In implementation, the terms inside the logarithms are regularized to avoid invalid values.

For this architecture, the encoder input in the general encoder–head notation is

$$x(y_{1:n}) = S(y_{1:n}),$$

so that $\mathcal{X} = \mathbb{R}^{d_S}$. As stated above, the encoder E_{w_0} is an MLP. Thus, the network learns from a fixed collection of features designed to summarize the location, scale and dependence structure of the transformed observations, rather than directly from the full sequence.

2.6.3 Transformed log-return TCN encoder

The second architecture uses a standardized version of the transformed sequence $\text{logsq}(y_{1:n})$ from (2.41) as input to a temporal convolutional encoder. A TCN is a convolutional neural network designed for sequence data. It applies one-dimensional filters along the time dimension, so that local temporal patterns can be learned directly from the ordered sequence. This is well suited to the transformed log-return sequence because information about the SV parameters is contained not only in the marginal distribution of the transformed observations, but also in their dependence across time.

The architecture used here is motivated by temporal convolutional networks for sequence modelling, as studied by Bai et al. [20], and by the residual-network construction of He et al. [19]. The TCN architecture is useful because dilated convolutions allow local filters to cover increasingly large temporal ranges, while residual connections stabilize the learning process of deeper convolutional stacks. In contrast to the causal TCNs used for autoregressive sequence prediction by Bai et al. [20], the encoder used in this thesis is non-causal. Here, causal means that the representation at time t is only allowed to depend on inputs at times $s \leq t$. This restriction is unnecessary in the present setting, since the posterior approximation is conditioned on the full observed time series. We therefore use symmetric padding, allowing the representation at a time point to use neighbouring observations on both sides.

We first describe the convolutional operations used in the encoder. For

$$x \in \mathbb{R}^{C_{\text{in}} \times n},$$

the first index denotes the channel and the second denotes time. Thus, $x_{c,t}$ is the value of channel c at time t , and each channel is a scalar sequence of length n .

A one-dimensional convolutional layer maps x to a new sequence according to

$$\mathbb{R}^{C_{\text{in}} \times n} \longrightarrow \mathbb{R}^{C_{\text{out}} \times n}$$

by applying C_{out} filters to local windows around each time point $1 \leq t \leq n$.

For $j \in \mathbb{N} \cup \{0\}$, let $k = 2j + 1$ be an odd kernel size, and let $d \in \mathbb{N}$ be a dilation factor. The kernel weights and biases are

$$A \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k}, \quad b \in \mathbb{R}^{C_{\text{out}}},$$

where the three indices of A correspond respectively to the output channel, input channel and kernel position.

It is convenient to index the final dimension of A by offsets $r = -j, \dots, j$. Define the zero-padded extension of x by

$$x_{c,t}^{\text{pad}} = \begin{cases} x_{c,t}, & 1 \leq t \leq n, \\ 0, & \text{otherwise.} \end{cases}$$

The length-preserving dilated convolution operator

$$\mathcal{K}_{A,b}^{(k,d)} : \mathbb{R}^{C_{\text{in}} \times n} \rightarrow \mathbb{R}^{C_{\text{out}} \times n}$$

is defined by

$$\left[\mathcal{K}_{A,b}^{(k,d)}(x) \right]_{c,t} = b_c + \sum_{c'=1}^{C_{\text{in}}} \sum_{r=-j}^j A_{c,c',r} x_{c',t+dr}^{\text{pad}}, \quad (2.48)$$

for $c = 1, \dots, C_{\text{out}}$ and $t = 1, \dots, n$. The odd kernel size gives a symmetric set of offsets around t . With padding of dj values on each side, the output has the same time dimension as the input.

The kernel size k determines the number of coefficients in each local filter, while the dilation factor d determines the spacing between the input time points used by these coefficients. When $d = 1$, the operator reduces to an ordinary one-dimensional convolution. Larger dilations allow the same number of weights to cover a wider temporal range. The parameters of the convolutional layer are the filter weights A and the biases b . These weights are shared across time, meaning that the same local pattern is searched for at every position in the sequence. For time series this is a natural inductive bias, since local dependence patterns are not assumed to depend on the absolute time index.

A *temporal residual block* is a length-preserving map with two terms. The main term is a convolutional residual function, which applies dilated convolutions and nonlinearities to the input. The second term is the shortcut branch, which either passes the input through unchanged or transforms it to the required channel dimension. The two terms are then added before the block activation is applied. This is the residual-learning idea of He et al. [19], where the convolutional path

learns a correction to the shortcut branch, rather than an entire transformation from scratch.

In this thesis, as in the baseline TCN block of Bai et al. [20], each residual function contains two dilated convolutional layers. Let

$$x \in \mathbb{R}^{C_{\ell-1} \times n}$$

be the input to block ℓ , and let C_ℓ , $k_\ell = 2j_\ell + 1$, and d_ℓ denote the output channel dimension, kernel size and dilation factor of the block. Write Φ for the hidden-layer activation function. The *residual function* is

$$F_{\xi_\ell}(x) = \mathcal{K}_{A_{\ell,2}, b_{\ell,2}}^{(k_\ell, d_\ell)} \left(\Phi \left(\mathcal{K}_{A_{\ell,1}, b_{\ell,1}}^{(k_\ell, d_\ell)}(x) \right) \right),$$

where

$$A_{\ell,1} \in \mathbb{R}^{C_\ell \times C_{\ell-1} \times k_\ell}, \quad b_{\ell,1} \in \mathbb{R}^{C_\ell},$$

and

$$A_{\ell,2} \in \mathbb{R}^{C_\ell \times C_\ell \times k_\ell}, \quad b_{\ell,2} \in \mathbb{R}^{C_\ell}.$$

The *shortcut branch* is

$$R_\ell(x) = \begin{cases} x, & C_{\ell-1} = C_\ell, \\ \mathcal{K}_{P_\ell, a_\ell}^{(1,1)}(x), & C_{\ell-1} \neq C_\ell, \end{cases}$$

where, in the second case,

$$P_\ell \in \mathbb{R}^{C_\ell \times C_{\ell-1} \times 1}, \quad a_\ell \in \mathbb{R}^{C_\ell}.$$

The 1×1 convolution changes only the channel dimension and does not mix neighbouring time points. It is included so that the residual function and shortcut branch have matching dimensions before they are added.

Thus, the output of the temporal residual block is

$$B_{\xi_\ell}(x) = \Phi(F_{\xi_\ell}(x) + R_\ell(x)), \quad (2.49)$$

where the addition is elementwise over channels and time. The block weights are collected as

$$\xi_\ell = \{(A_{\ell,1}, b_{\ell,1}), (A_{\ell,2}, b_{\ell,2})\},$$

together with (P_ℓ, a_ℓ) when $C_{\ell-1} \neq C_\ell$.

A *temporal residual stack* is obtained by composing such blocks. Let L denote the number of temporal residual blocks, and let

$$x^{(0)} \in \mathbb{R}^{C_0 \times n}$$

be the input to the stack. The hidden feature maps are defined recursively by

$$x^{(\ell)} = B_{\xi_\ell}(x^{(\ell-1)}), \quad \ell = 1, \dots, L. \quad (2.50)$$

This gives

$$x^{(L)} = (B_{\xi_L} \circ \dots \circ B_{\xi_1})(x^{(0)}) \in \mathbb{R}^{C_L \times n}.$$

Since each block is length-preserving, the time dimension remains n throughout the stack.

The final *temporal feature map* $x^{(L)}$ is converted to a fixed-dimensional feature vector by pooling over time. In this architecture, average pooling, maximum pooling and top- k mean pooling are used. For $x \in \mathbb{R}^{C_L \times n}$, define

$$u_c^{\text{avg}} = \frac{1}{n} \sum_{t=1}^n x_{c,t}, \quad u_c^{\text{max}} = \max_{1 \leq t \leq n} x_{c,t}.$$

Let $\alpha \in (0, 1]$ be the top- k pooling fraction, and set

$$m_\alpha = \max\{1, \lfloor \alpha n \rfloor\}.$$

For each channel c , let $I_c^{(\alpha)}$ be a set of indices corresponding to the m_α largest values among $x_{c,1}, \dots, x_{c,n}$. The top- k mean summary is

$$u_c^{\text{top}} = \frac{1}{m_\alpha} \sum_{t \in I_c^{(\alpha)}} x_{c,t}.$$

The pooling map is therefore

$$\Pi_\alpha : \mathbb{R}^{C_L \times n} \rightarrow \mathbb{R}^{3C_L},$$

defined by

$$\Pi_\alpha(x) = (u^{\text{avg}}, u^{\text{max}}, u^{\text{top}}).$$

The average summary captures the typical activation level in each channel, the maximum summary captures the strongest activation, and the top- k mean provides an intermediate summary of large activations that is less sensitive to a single time point than the maximum.

Combining the temporal residual stack and the pooling map gives a parametric family of temporal convolutional encoders. For fixed architecture choices

$$\mathcal{A}_T = (C_0, \dots, C_L, k_1, \dots, k_L, d_1, \dots, d_L),$$

define

$$\Pi_\alpha B^L(\cdot; \mathcal{A}_T, \xi) = \Pi_\alpha \circ B_{\xi_L} \circ \dots \circ B_{\xi_1}, \quad \xi = (\xi_1, \dots, \xi_L).$$

This map sends an input feature map in $\mathbb{R}^{C_0 \times n}$ to a vector in $\mathbb{R}^{3C_L}$. Since each residual function contains two convolutional layers, the *temporal radius* of the receptive field after the full stack is, ignoring boundary effects from padding,

$$R_T = \sum_{\ell=1}^L (k_\ell - 1) d_\ell.$$

Here, the temporal radius means the largest distance, measured in time indices, between a feature in the final temporal feature map and an input value that can affect it. Thus, a feature at time t in $x^{(L)}$ can only depend on entries of the original input $x^{(0)}$ with time indices approximately between $t - R_T$ and $t + R_T$, with the interval truncated near the boundaries of the observed sequence. Larger kernels, larger dilations and deeper stacks therefore increase the temporal range from which the encoder can construct features.

We now specialize this construction to the transformed log-return input. In the general encoder-head notation,

$$x_{\text{TCN}}(y_{1:n}) = x^{(0)}, \quad \mathcal{X} = \mathbb{R}^{1 \times n}, \quad (2.51)$$

where

$$x_{1,t}^{(0)} = \frac{\log(y_t^2 + \kappa) - a_x}{s_x}, \quad t = 1, \dots, n.$$

Here a_x and $s_x > 0$ are fixed centering and scaling constants derived from the mean and standard deviation of $\log(y_t^2)$. The constant $\kappa > 0$ is chosen to be sufficiently small so that its effect on the standardization is negligible. This standardization places the transformed observations on a stable numerical scale before they are passed through the convolutional layers. The required moments of $\log(y_t^2)$ are derived in Appendix A.2.

The input has a single channel, so $C_0 = 1$. The transformed log-return TCN encoder is the map

$$E_{w_0}(x) = \Pi_\alpha B^L(x; \mathcal{A}_T, w_0), \quad (2.52)$$

where

$$w_0 = (\xi_1, \dots, \xi_L)$$

contains the trainable weights of the temporal residual stack. The encoder output dimension is

$$d_E = 3C_L.$$

With

$$x(y_{1:n}) = x_{\text{TCN}}(y_{1:n}),$$

the resulting feature vector

$$z = E_{w_0}(x_{\text{TCN}}(y_{1:n}))$$

is passed to the prediction heads described in Section 2.6.1. Unlike the summary-statistic architecture, this encoder learns its own summaries directly from the transformed sequence. The convolutional filters extract local temporal patterns, the dilations allow information to be combined over increasing temporal ranges, the residual branches stabilize the stacked representation, and the pooling map converts the final temporal feature map into the fixed-dimensional representation used by the posterior heads.

2.7 Stochastic optimization with Adam

The preceding sections defined the joint simulation density p_{sim} , the approximate posterior $q_W(\theta; y_{1:n})$, the marginal empirical losses $L_{N,j}(W)$, their mean $L_N(W)$, and the neural-network architectures used to represent the posterior parameters. It remains to determine weights W for which the predicted means and variances approximate the corresponding posterior moment functions across parameter–data pairs generated by the SV model.

One possibility would be to generate a single large training set and repeatedly minimize its empirical loss. Since parameter–data pairs can be generated directly from the simulator, we instead use *simulation-based training*. In this thesis, this means that an independent mini-batch is generated from p_{sim} at every optimizer iteration, used to evaluate the mini-batch loss and compute its gradient, and then discarded. After T iterations with mini-batch size B , the network has been trained using BT generated pairs, while only the current B pairs need to be retained as training data. Simulation-based training can therefore expose the network to a much larger effective training sample without requiring one comparably large data set to be stored in memory.

More importantly, independently generated mini-batches provide repeated Monte Carlo approximations of the population loss $\mathcal{L}(W)$ and its gradient. Training is therefore directed toward the expected loss under the joint simulation distribution rather than the empirical loss associated with one particular finite sample. As shown in Proposition 1, minimizing the population loss is equivalent to minimizing the prior-predictive average forward KL divergence between the true and approximate posteriors. Proposition 2 further shows that, if the Gaussian mean and variance functions could be chosen freely, this objective would be minimized by the true conditional mean and variance of each transformed marginal posterior for prior-predictive almost every $y_{1:n}$. This conclusion does not require the true marginal posteriors to be Gaussian. The population loss therefore provides a meaningful target for the first two posterior moments even when the Gaussian approximations cannot represent the complete shapes of the marginal posteriors. Within the chosen neural-network family, these moment functions specify the target to be approximated rather than guaranteeing that it is attained exactly.

This procedure follows the stochastic-optimization viewpoint described by Bottou et al. [21], in which repeated updates based on noisy gradient estimates replace repeated evaluation of a large empirical or population objective. The resulting stochastic gradients are used by Adam, introduced by Kingma and Ba [22]. Adam forms exponentially weighted averages of past gradients and squared gradients, which smooth and rescale the parameter updates. We first define the marginal population losses and their mini-batch approximations, then present the Adam update, and finally establish the connection between the population loss, the true posterior and its first two marginal moments.

2.7.1 Population loss, mini-batches and stochastic gradients

The empirical marginal loss $L_{N,j}(W)$ in (2.39) is calculated from a finite collection of simulated pairs. Its population counterpart is the expected marginal Gaussian loss

$$\begin{aligned}\mathcal{L}_j(W) &= \frac{1}{2} \mathbb{E}_{p_{\text{sim}}} \left[\log \widehat{v}_{W,j}(Y_{1:n}) + \frac{(\widetilde{\Theta}_j - \widehat{m}_{W,j}(Y_{1:n}))^2}{\widehat{v}_{W,j}(Y_{1:n})} \right] \\ &= \frac{1}{2} \int \int \left[\log \widehat{v}_{W,j}(y_{1:n}) + \frac{(\widetilde{\theta}_j - \widehat{m}_{W,j}(y_{1:n}))^2}{\widehat{v}_{W,j}(y_{1:n})} \right] p_{\text{sim}}(\widetilde{\theta}, y_{1:n}) d\widetilde{\theta} dy_{1:n}, \quad j = 1, 2, 3.\end{aligned}\tag{2.53}$$

The overall population posterior loss is the mean of these marginal population losses:

$$\mathcal{L}(W) = \frac{1}{3} \sum_{j=1}^3 \mathcal{L}_j(W).\tag{2.54}$$

For a fixed training set $\mathcal{D}_N$, each $L_{N,j}(W)$ is a Monte Carlo approximation of $\mathcal{L}_j(W)$, while $L_N(W)$ is the corresponding approximation of $\mathcal{L}(W)$.

These expectations cannot generally be evaluated exactly. During simulation-based training, they are instead approximated using independently generated mini-batches drawn from p_{sim} . A *batch* is a finite collection of simulated parameter-data pairs used together to evaluate a loss, and its *batch size* is the number of pairs in the collection. A batch used for one optimizer update is referred to as a *mini-batch*. In this thesis, a new mini-batch is generated at every optimizer iteration.

At iteration t , let

$$\mathcal{D}_B^{(t)} = \left\{ (\widetilde{\Theta}^{(t,b)}, Y_{1:n}^{(t,b)}) \right\}_{b=1}^B$$

denote the mini-batch of size B , where

$$(\widetilde{\Theta}^{(t,b)}, Y_{1:n}^{(t,b)}) \stackrel{\text{i.i.d.}}{\sim} p_{\text{sim}}(\widetilde{\theta}, y_{1:n}).$$

For the j th transformed parameter, the corresponding mini-batch loss is

$$L_{B,j}^{(t)}(W) = \frac{1}{2B} \sum_{b=1}^B \left[\log \widehat{v}_{W,j}(Y_{1:n}^{(t,b)}) + \frac{(\widetilde{\Theta}_j^{(t,b)} - \widehat{m}_{W,j}(Y_{1:n}^{(t,b)}))^2}{\widehat{v}_{W,j}(Y_{1:n}^{(t,b)})} \right], \quad j = 1, 2, 3.\tag{2.55}$$

The overall mini-batch loss is the mean of the three marginal mini-batch losses:

$$L_B^{(t)}(W) = \frac{1}{3} \sum_{j=1}^3 L_{B,j}^{(t)}(W).\tag{2.56}$$

Thus, $L_{B,j}^{(t)}(W)$ has exactly the same form as $L_{N,j}(W)$, with the fixed training sample of size N replaced by the mini-batch of size B generated at iteration t .

The marginal stochastic gradients and the gradient supplied to the optimizer are

$$\begin{aligned} g_{t,j} &= \nabla_W L_{B,j}^{(t)}(W_{t-1}), \quad j = 1, 2, 3, \\ g_t &= \nabla_W L_B^{(t)}(W_{t-1}) = \frac{1}{3} \sum_{j=1}^3 g_{t,j}. \end{aligned} \quad (2.57)$$

Although the loss separates into marginal terms, the gradients are taken with respect to the complete weight vector W . In particular, all three marginal losses contribute to the gradient of the shared representation.

For any fixed weight vector w , each marginal mini-batch loss is an unbiased estimator of its population counterpart:

$$\begin{aligned} \mathbb{E}[L_{B,j}^{(t)}(w)] &= \mathcal{L}_j(w), \quad j = 1, 2, 3, \\ \mathbb{E}[L_B^{(t)}(w)] &= \frac{1}{3} \sum_{j=1}^3 \mathbb{E}[L_{B,j}^{(t)}(w)] = \mathcal{L}(w). \end{aligned} \quad (2.58)$$

Provided the one-pair marginal losses have finite variance, independence of the simulated pairs gives

$$\begin{aligned} \text{Var}[L_{B,j}^{(t)}(w)] &= \frac{1}{4B} \text{Var} \left[\log \hat{v}_{w,j}(Y_{1:n}) + \frac{(\tilde{\Theta}_j - \hat{m}_{w,j}(Y_{1:n}))^2}{\hat{v}_{w,j}(Y_{1:n})} \right], \\ &\quad j = 1, 2, 3. \end{aligned} \quad (2.59)$$

The same $1/B$ reduction applies to the variance of the overall mini-batch loss, since $L_B^{(t)}(w)$ is also an average of B independent one-pair losses. Increasing B therefore gives less variable estimates of the marginal and overall population losses, but requires more simulations and computation for each optimizer update.

Under regularity conditions that allow differentiation and expectation to be interchanged, the marginal and overall stochastic gradients are similarly unbiased:

$$\begin{aligned} \mathbb{E}[\nabla_W L_{B,j}^{(t)}(w)] &= \nabla_W \mathcal{L}_j(w), \quad j = 1, 2, 3, \\ \mathbb{E}[\nabla_W L_B^{(t)}(w)] &= \nabla_W \mathcal{L}(w). \end{aligned} \quad (2.60)$$

The current weights W_{t-1} depend on the mini-batches used in the preceding iterations. The new mini-batch $\mathcal{D}_B^{(t)}$, however, is generated independently of these weights. Conditional on W_{t-1} , we therefore have

$$\begin{aligned} \mathbb{E}[L_{B,j}^{(t)}(W_{t-1}) | W_{t-1}] &= \mathcal{L}_j(W_{t-1}), \quad j = 1, 2, 3, \\ \mathbb{E}[L_B^{(t)}(W_{t-1}) | W_{t-1}] &= \mathcal{L}(W_{t-1}), \\ \mathbb{E}[g_{t,j} | W_{t-1}] &= \nabla_W \mathcal{L}_j(W_{t-1}), \quad j = 1, 2, 3, \\ \mathbb{E}[g_t | W_{t-1}] &= \nabla_W \mathcal{L}(W_{t-1}). \end{aligned} \quad (2.61)$$

Thus, every independently generated mini-batch provides unbiased Monte Carlo estimates of the three marginal population losses, their mean, and the corresponding gradients at the current weights. This is an average property over possible mini-batches. An individual stochastic gradient may still differ considerably from the population gradient and need not be a descent direction for $\mathcal{L}$. Adam addresses this variability by combining gradient information across iterations and rescaling the updates using recent squared gradients.

2.7.2 Adam update rule

The simplest update based on the stochastic gradient in (2.57) would be

$$W_t = W_{t-1} - \alpha g_t,$$

where $\alpha > 0$ is the stepsize. Since g_t is computed from a random mini-batch, its direction and magnitude may fluctuate between iterations. The update above passes these fluctuations directly to the weights because it uses only the current stochastic gradient.

Adam, introduced by Kingma and Ba [22], also uses gradients from earlier iterations. It maintains an exponentially weighted average of the gradients to smooth the estimated update direction, together with an exponentially weighted average of the squared gradients to rescale the update coordinatewise. These quantities are treated as estimates of the first and second moments of the stochastic gradients, which gives the method its name: adaptive moment estimation.

Let d_W be the number of trainable weights, so that

$$W_t, g_t, m_t, v_t \in \mathbb{R}^{d_W}.$$

Adam initializes $m_0 = v_0 = 0$ and updates the two moment estimates according to

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (g_t \odot g_t), \end{aligned}$$

where $\beta_1, \beta_2 \in [0, 1)$, and $\odot$ denotes componentwise multiplication. The vector m_t is an exponentially weighted average of the gradients, while v_t is an exponentially weighted average of the squared gradients.

Repeated substitution gives

$$\begin{aligned} m_t &= (1 - \beta_1) \sum_{s=1}^t \beta_1^{t-s} g_s, \\ v_t &= (1 - \beta_2) \sum_{s=1}^t \beta_2^{t-s} (g_s \odot g_s). \end{aligned}$$

The contribution from an earlier gradient decreases geometrically with its age. Values of β_1 and β_2 close to one therefore produce more slowly changing averages, whereas smaller values place more weight on the most recent gradients.

Because $m_0 = v_0 = 0$, the coefficients in these averages sum to $1 - \beta_1^t$ and $1 - \beta_2^t$, respectively, rather than to one. The averages are consequently pulled towards zero during the first iterations. Adam corrects for this initialization by defining

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The weights are then updated according to

$$W_t = W_{t-1} - \alpha_{\text{Adam}} \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon_{\text{Adam}}}, \quad (2.62)$$

where the square root and division are applied componentwise. The constant $\varepsilon_{\text{Adam}} > 0$ prevents division by zero and limits the rescaling when a component of $\widehat{v}_t$ is very small. The complete procedure is summarized in Algorithm 2.

Algorithm 2 Adam update for stochastic minimization

Require: Initial weights W_0 , stepsize $\alpha_{\text{Adam}} > 0$, decay rates $\beta_1, \beta_2 \in [0, 1)$, constant $\varepsilon_{\text{Adam}} > 0$, mini-batch size B , number of optimizer updates T

```

1:  $m_0 \leftarrow 0$ 
2:  $v_0 \leftarrow 0$ 
3: for  $t = 1, \dots, T$  do
4:   Generate an independent mini-batch  $\mathcal{D}_B^{(t)}$  from  $p_{\text{sim}}$ 
5:    $g_t \leftarrow \nabla_W L_B^{(t)}(W_{t-1})$ 
6:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
7:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)(g_t \odot g_t)$ 
8:    $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
9:    $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
10:   $W_t \leftarrow W_{t-1} - \alpha_{\text{Adam}} \widehat{m}_t / (\sqrt{\widehat{v}_t} + \varepsilon_{\text{Adam}})$ 
11: end for
12: return  $W_T$ 

```

The numerator in (2.62) describes the recent average gradient direction. The denominator describes the recent root-mean-square gradient size for each coordinate. A coordinate with consistently large gradients therefore receives a smaller change relative to its raw gradient magnitude than it would under the ordinary stochastic-gradient update. If its gradients frequently change sign, their contributions to $\widehat{m}_t$ may cancel while their squared values continue to contribute positively to $\widehat{v}_t$. The resulting update in that coordinate then becomes small. Adam therefore uses both the consistency and the scale of recent gradients when determining each coordinatewise change.

To bound the size of this change, temporarily set $\varepsilon_{\text{Adam}} = 0$ and define

$$\Delta_t = \alpha_{\text{Adam}} \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}}.$$

The parameter change is then

$$W_t - W_{t-1} = -\Delta_t.$$

Following Kingma and Ba [22], the componentwise magnitude of Δ_t satisfies

$$|\Delta_t| \leq \begin{cases} \alpha_{\text{Adam}} \frac{1 - \beta_1}{\sqrt{1 - \beta_2}}, & 1 - \beta_1 > \sqrt{1 - \beta_2}, \\ \alpha_{\text{Adam}}, & 1 - \beta_1 \leq \sqrt{1 - \beta_2}. \end{cases} \quad (2.63)$$

Here, absolute values and inequalities are interpreted componentwise. When $\varepsilon_{\text{Adam}} > 0$, the denominator in (2.62) is larger, so the magnitude of the corresponding change remains within these bounds. The role of α_{Adam} is then to set the overall scale of these coordinatewise changes, while the two moment estimates, $\widehat{m}_t$ and $\widehat{v}_t$, determine their directions and relative sizes.

2.7.3 Population loss and posterior approximation

We now relate the population loss to the discrepancy between the true and approximate posteriors. We first show that minimizing the population loss is the same as minimizing a prior-predictive average forward Kullback–Leibler divergence. We then use the product structure of the approximate posterior to determine the optimal Gaussian marginals when their parameters can be chosen freely for a fixed sequence. The KL characterization in the first proposition is a direct specialization of the calculation in the proof of Theorem 2 of Maceda et al. [3] to our population loss and product-Gaussian approximation.

Proposition 1 (Population loss and average forward KL divergence). *Assume that the expectations appearing below and the prior-predictive average posterior entropy are finite for all W under consideration. Then the population loss can be written as*

$$\mathcal{L}(W) = C + \frac{1}{3} \int m(y_{1:n}) D_{\text{KL}}(p(\cdot | y_{1:n}) \| q_W(\cdot; y_{1:n})) dy_{1:n}, \quad (2.64)$$

where

$$C = \frac{1}{3} \int m(y_{1:n}) H(p(\cdot | y_{1:n})) dy_{1:n} - \frac{1}{2} \log(2\pi)$$

does not depend on W . Consequently,

$$\arg \min_W \mathcal{L}(W) = \arg \min_W \int m(y_{1:n}) D_{\text{KL}}(p(\cdot | y_{1:n}) \| q_W(\cdot; y_{1:n})) dy_{1:n}. \quad (2.65)$$

Proof. From (2.38),

$$-\log q_W(\tilde{\theta}; y_{1:n}) = \frac{3}{2} \log(2\pi) + \frac{1}{2} \sum_{j=1}^3 \left[\log \widehat{v}_{W,j}(y_{1:n}) + \frac{(\tilde{\theta}_j - \widehat{m}_{W,j}(y_{1:n}))^2}{\widehat{v}_{W,j}(y_{1:n})} \right].$$

Taking expectations under the joint simulation density gives

$$\begin{aligned}\mathbb{E}[-\log q_W(\tilde{\Theta}; Y_{1:n})] &= \frac{3}{2} \log(2\pi) + \sum_{j=1}^3 \mathcal{L}_j(W) \\ &= \frac{3}{2} \log(2\pi) + 3\mathcal{L}(W).\end{aligned}\tag{2.66}$$

Using

$$p_{\text{sim}}(\tilde{\theta}, y_{1:n}) = p(\tilde{\theta} | y_{1:n})m(y_{1:n}),$$

the expectation on the left-hand side becomes

$$\begin{aligned}\mathbb{E}[-\log q_W(\tilde{\Theta}; Y_{1:n})] &= \int m(y_{1:n}) \int p(\tilde{\theta} | y_{1:n}) [-\log q_W(\tilde{\theta}; y_{1:n})] d\tilde{\theta} dy_{1:n}.\end{aligned}$$

For each $y_{1:n}$, the inner integral is the cross-entropy from the true posterior to the approximate posterior. The relation between cross-entropy, entropy and Kullback–Leibler divergence therefore gives

$$\begin{aligned}\mathbb{E}[-\log q_W(\tilde{\Theta}; Y_{1:n})] &= \int m(y_{1:n}) [H(p(\cdot | y_{1:n})) + D_{\text{KL}}(p(\cdot | y_{1:n}) \| q_W(\cdot; y_{1:n}))] dy_{1:n}.\end{aligned}$$

Definitions and further properties of entropy, cross-entropy and Kullback–Leibler divergence are given by Murphy [23, Chapter 6]. Substitution into (2.66) gives (2.64). Since C does not depend on W , the two objectives have the same minimizers. $\square$

The approximate posterior in (2.38) is restricted to a product of marginal Gaussian densities. For fixed $y_{1:n}$, let $p_j(\cdot | y_{1:n})$ denote the j th marginal density of the true posterior. The forward KL divergence then satisfies

$$\begin{aligned}D_{\text{KL}}(p(\tilde{\theta} | y_{1:n}) \| q(\tilde{\theta}; m_{1:3}, v_{1:3})) &= D_{\text{KL}}\left(p(\tilde{\theta} | y_{1:n}) \left\| \prod_{j=1}^3 p_j(\tilde{\theta}_j | y_{1:n})\right\| + \sum_{j=1}^3 D_{\text{KL}}(p_j(\tilde{\theta}_j | y_{1:n}) \| \varphi(\tilde{\theta}_j; m_j, v_j))\right),\end{aligned}\tag{2.67}$$

where $\tilde{\theta}$ is the variable of integration. To obtain this decomposition, use the product form in (2.38) to write

$$\log \frac{p(\tilde{\theta} | y_{1:n})}{q(\tilde{\theta}; m_{1:3}, v_{1:3})} = \log \frac{p(\tilde{\theta} | y_{1:n})}{\prod_{j=1}^3 p_j(\tilde{\theta}_j | y_{1:n})} + \sum_{j=1}^3 \log \frac{p_j(\tilde{\theta}_j | y_{1:n})}{\varphi(\tilde{\theta}_j; m_j, v_j)}.$$

Integrating under the true posterior gives the first divergence on the right-hand side of (2.67). Each term in the sum depends only on $\tilde{\theta}_j$, so its expectation under the joint posterior is the corresponding expectation under the j th marginal.

The first term on the right-hand side of (2.67) does not depend on the Gaussian marginal parameters. For the fixed sequence $y_{1:n}$, it vanishes precisely when

$$p(\tilde{\theta} \mid y_{1:n}) = \prod_{j=1}^3 p_j(\tilde{\theta}_j \mid y_{1:n})$$

almost everywhere. For fixed $y_{1:n}$ and freely chosen marginal parameters, the remaining terms can therefore be minimized separately.

Proposition 2 (Moment-matched product-Gaussian minimizer). *Fix $y_{1:n}$, and suppose that $p(\cdot \mid y_{1:n})$ has finite differential entropy and that, for every j , the marginal posterior of $\tilde{\Theta}_j$ has a finite mean and a finite, strictly positive variance. Define*

$$m_j^*(y_{1:n}) = \mathbb{E}[\tilde{\Theta}_j \mid y_{1:n}], \quad v_j^*(y_{1:n}) = \text{Var}(\tilde{\Theta}_j \mid y_{1:n}).$$

Among all product-Gaussian densities of the form (2.38), the forward KL divergence

$$D_{\text{KL}}(p(\cdot \mid y_{1:n}) \parallel q(\cdot; m_{1:3}, v_{1:3}))$$

is uniquely minimized by

$$m_j = m_j^*(y_{1:n}), \quad v_j = v_j^*(y_{1:n}), \quad j = 1, 2, 3. \quad (2.68)$$

Proof. The differential entropy of the true posterior is finite and does not depend on $m_{1:3}$ or $v_{1:3}$. Minimizing the forward KL divergence is therefore the same as minimizing the cross-entropy

$$-\int p(\tilde{\theta} \mid y_{1:n}) \log q(\tilde{\theta}; m_{1:3}, v_{1:3}) d\tilde{\theta}.$$

Using the product-Gaussian density, this cross-entropy is

$$\frac{3}{2} \log(2\pi) + \frac{1}{2} \sum_{j=1}^3 \left[\log v_j + \frac{v_j^*(y_{1:n}) + (m_j^*(y_{1:n}) - m_j)^2}{v_j} \right].$$

The terms separate over j . For fixed $v_j > 0$, the j th term is uniquely minimized at

$$m_j = m_j^*(y_{1:n}).$$

After this substitution, its derivative with respect to v_j is

$$\frac{v_j - v_j^*(y_{1:n})}{2v_j^2}.$$

Since $v_j^*(y_{1:n}) > 0$, this derivative is negative for $0 < v_j < v_j^*(y_{1:n})$ and positive for $v_j > v_j^*(y_{1:n})$. Hence, the unique minimum occurs at

$$v_j = v_j^*(y_{1:n}).$$

This gives (2.68). □

Propositions 1 and 2 characterize the target of the population loss. If the Gaussian mean and variance functions were unrestricted, the optimal functions would satisfy

$$\begin{aligned} m_j(y_{1:n}) &= \mathbb{E}[\tilde{\Theta}_j \mid y_{1:n}], \\ v_j(y_{1:n}) &= \text{Var}(\tilde{\Theta}_j \mid y_{1:n}) \end{aligned} \quad \text{for prior-predictive almost every } y_{1:n}. \quad (2.69)$$

This conclusion does not require the true posterior marginals to be Gaussian. They may be asymmetric, and the transformed parameters may remain dependent.

Taken together, Propositions 1 and 2 show why the population loss targets the first two moments of the marginal posteriors. The first proposition shows that minimizing the population loss amounts to minimizing the prior-predictive average forward KL divergence. Under the product restriction, the terms that depend on the approximate posterior can be minimized pointwise over $y_{1:n}$ and separately for each marginal. The second proposition shows that each such minimum is attained when the corresponding Gaussian marginal has the true posterior mean and variance. Hence, if the six conditional moment functions were unrestricted, the lowest population loss would be attained by the functions in (2.69). In the neural posterior family, however, a single weight vector W must approximate these functions throughout the prior-predictive distribution, and training minimizes the population loss only indirectly through finite mini-batches and numerical optimization. Equation (2.69) therefore specifies the target of the NPE objective, but does not guarantee that every trained network returns the exact posterior means and variances.

Chapter 3

Model Implementation

The preceding chapters introduced the stochastic-volatility model, the unconstraining transformation of its parameters, the marginal Gaussian posterior family and the neural posterior estimation objective. This chapter presents the computational implementation used to train and evaluate the Summary NN and TCN estimators. The focus is on the choices required to turn these methodological components into reproducible experiments, including data simulation, construction of the transformed targets and network inputs, numerical safeguards, configuration of the MCMC benchmark, early stopping, benchmark evaluation and tuning-parameter selection.

The implementation relies on NumPy for simulation, array operations and the construction of reproducible random-number streams [24]; the `statsmodels.tsa` module for additional time-series calculations [25]; and PyTorch for the neural-network modules, automatic differentiation, GPU execution and the Adam optimizer [26]. The MCMC benchmark is produced using `stochvol` [12].

Section 3.1 describes the two prior configurations, the construction of the transformed targets and architecture-specific inputs, and the numerical and reproducibility measures used in the simulation pipeline. Section 3.2 explains how the MCMC burn-in, retained chain length and observed sequence length were chosen using trace plots, effective sample sizes and an empirical Bernstein–von Mises diagnostic. Section 3.3 presents the PyTorch implementation, including the Gaussian negative log-likelihood, automatic differentiation, GPU computations, numerical diagnostics and deterministic execution. Section 3.4 specifies the early-stopping rule and the metrics used to compare the neural posterior estimators with the MCMC benchmark. Finally, Section 3.5 describes the tuning-parameter selection strategy and compares the selected Summary NN and TCN configurations and their computational requirements.

The complete source code and dependency specification used to produce the results in this thesis are available in the GitHub repository linked in Appendix A.1.

3.1 Data simulation and numerical implementation

The implementation samples from the joint simulation density in (2.31) in four steps. First, an SV parameter vector θ is drawn from the prior in (2.26). Second, an observed sequence $y_{1:n}$ is simulated from (2.12) conditional on θ . The resulting pair is then mapped to the transformed target and input representation,

$$(\theta, y_{1:n}) \mapsto (G(\theta), x(y_{1:n})),$$

using (2.35) and (2.42). Finally, the neural network is evaluated at the constructed input, and its output is scored against the transformed target $\tilde{\theta} = G(\theta)$ using the posterior loss in (2.40).

This section records the numerical choices used to implement this pipeline. We first describe the two prior configurations and clarify the units in which μ is expressed. We then describe how NumPy and statsmodels.tsa are used to construct the transformed targets and the two encoder inputs. Finally, we present the numerical safeguards and reproducibility measures used throughout the implementation, including the parallel simulation procedure and the construction of independent random-number streams.

3.1.1 Prior configurations and the units of μ

The implementation uses the independent prior structure in (2.26),

$$\pi(\theta) = \pi_\mu(\mu)\pi_\phi(\phi)\pi_{\sigma^2}(\sigma^2).$$

Two hyperparameter configurations are considered. The *default prior* is intentionally broad, whereas the *finance prior* concentrates more mass on the log-volatility levels and persistence values expected for daily financial returns. Both configurations use the prior families introduced in (2.26), with the hyperparameters reported in Table 3.1.

The corresponding marginal prior densities are shown in Figure 3.1. The configurations differ only through the priors for μ and ϕ . The finance prior is substantially more concentrated for both parameters, whereas the prior for σ^2 is identical under the two configurations.

The default configuration follows the prior families used by `stochvol` and retains its default choices for the transformed-beta prior on ϕ and the prior on

Table 3.1: Prior configurations used when simulating data from the SV model.

Parameter	Default prior	Finance prior
μ	$\mathcal{N}(0, 10^2)$	$\mathcal{N}(-9, 1)$
$(\phi + 1)/2$	Beta(5, 1.5)	Beta(20, 1.5)
σ^2	χ_1^2	χ_1^2

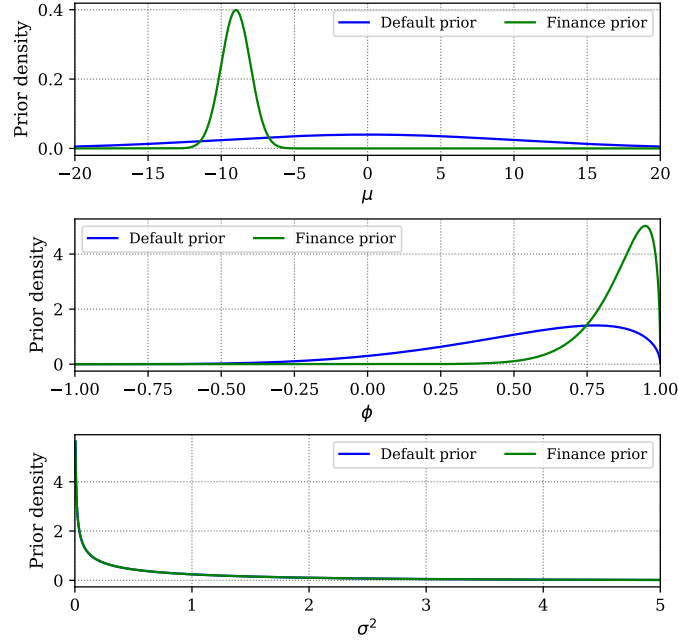


Figure 3.1: Marginal prior densities under the default and finance prior configurations. The finance prior is more concentrated for μ and ϕ , whereas both configurations assign the same prior to σ^2 .

σ^2 ; see Kastner [12]. The standard deviation of the prior for μ is, however, reduced from the `stochvol` default of 100 to 10. A standard deviation of 10 remains extremely broad on the log-variance scale, but reduces the frequency of prior-predictive series with extreme magnitudes. This modification is primarily computational, since a more diffuse prior enlarges the region over which the neural network must learn the posterior map.

The transformed-beta prior implies

$$\mathbb{E}[\phi] = 2 \frac{a_\phi}{a_\phi + b_\phi} - 1 = \begin{cases} 0.538, & \text{under the default prior,} \\ 0.860, & \text{under the finance prior.} \end{cases}$$

The finance prior therefore assigns substantially more mass to highly persistent volatility processes, as also shown in Figure 3.1.

The location $m_\mu = -9$ in the finance prior depends on the units in which returns are recorded. Under (2.12), the conditional standard deviation of Y_t given H_t is $\exp(H_t/2)$. Since the stationary distribution of H_t is symmetric around μ , the quantity $\exp(\mu/2)$ is the stationary median of the conditional volatility for fixed (μ, ϕ, σ) . For daily log returns recorded in decimal units,

$$\exp(-9/2) \approx 0.0111,$$

corresponding to a median daily conditional volatility of approximately 1.11%.

Using 253 trading days per year gives the annualized value

$$\sqrt{253} \exp(-9/2) \approx 0.177,$$

or 17.7%. This is consistent with the volatility levels commonly observed for equity indices. For example, Bollerslev et al. [27] report a median annualized realized volatility of 17.0% and a mean of 20.6% across the equity indices in their data. The finance-prior location therefore represents a plausible central volatility level for daily equity returns recorded in decimal form.

The prior standard deviation $s_\mu = 1$ still allows substantial variation around this location. The one-standard-deviation interval $m_\mu \pm s_\mu = [-10, -8]$ corresponds to median annualized conditional volatility levels between

$$\sqrt{253} \exp(-10/2) \approx 10.7\%$$

and

$$\sqrt{253} \exp(-8/2) \approx 29.1\%.$$

Variation in H_t induced by ϕ and σ further broadens the resulting prior-predictive volatility distribution.

3.1.2 Construction of training inputs and targets

For N independent simulated pairs

$$\left\{ \left(\theta^{(i)}, y_{1:n}^{(i)} \right) \right\}_{i=1}^N$$

drawn from the joint simulation density in (2.31), the implementation constructs one target matrix and one architecture-specific input matrix. NumPy's Generator interface is used to draw the prior variables and Gaussian innovations in (2.12), while vectorized array operations are used to simulate multiple log-volatility processes simultaneously.

The *target matrix* is defined as

$$\tilde{\theta}_{1:N,1:3} = \begin{pmatrix} G(\theta^{(1)})^\top \\ \vdots \\ G(\theta^{(N)})^\top \end{pmatrix} \in \mathbb{R}^{N \times 3}, \quad (3.1)$$

where

$$G(\theta) = (\mu, 2 \operatorname{arctanh}(\phi), \log \sigma)$$

is the unconstraining transformation from (2.35). The finite-precision regularization applied near the boundaries of G is described in Section 3.1.3.

For the temporal convolutional encoder, the corresponding *input-representation matrix* is

$$x_{1:N,1:n}^{\text{TCN}} = \begin{pmatrix} x_{\text{TCN}}(y_{1:n}^{(1)}) \\ \vdots \\ x_{\text{TCN}}(y_{1:n}^{(N)}) \end{pmatrix} \in \mathbb{R}^{N \times n}, \quad (3.2)$$

where $x_{\text{TCN}}(y_{1:n})$ is the standardized log-square representation defined in (2.51).

For the summary-statistic encoder, the input matrix is instead

$$x_{1:N, 1:d_S}^S = \begin{pmatrix} S(y_{1:n}^{(1)})^\top \\ \vdots \\ S(y_{1:n}^{(N)})^\top \end{pmatrix} \in \mathbb{R}^{N \times d_S}, \quad (3.3)$$

where the summary map S is defined in Section 2.6.2.

In this thesis, the term *simulation-based training* refers to generating new training data during optimization rather than repeatedly reading examples from a fixed data set. At the beginning of each training epoch, the implementation generates the examples required for several consecutive mini-batches in one parallel simulation call. The resulting matrices are partitioned into mini-batches, used once and then discarded. The next epoch uses a newly simulated collection of parameter–data pairs. An epoch therefore denotes the optimizer updates performed using one newly generated collection, rather than another pass through a persistent training set. Thus, no simulated example is reused across optimizer updates.

The selected summary representation uses

$$n_Q = 19, \quad n_\gamma = 4.$$

The empirical quantile levels are

$$p_\ell = 0.05 + 0.05(\ell - 1), \quad \ell = 1, \dots, 19,$$

so that the central 90% of the empirical distribution is represented at five-percentage-point intervals. The autocovariance-ratio features are calculated as

$$\hat{r}_k = \frac{\hat{\gamma}_x(k)}{\hat{\gamma}_x(k-1)}, \quad k = 1, \dots, 4.$$

These ratios summarize dependence through lag four. Higher-order ratios were omitted because they depend on increasingly small and noisy empirical autocovariances and were not expected to provide substantial additional information once the first four ratios and the optional ARMA(1, 1)-based summaries were included.

The optional ARMA(1, 1)-based summaries are obtained by fitting the model in (2.19) using the state-space implementation in `statsmodels.tsa`. For a description of the `statsmodels` library and its statistical-modeling capabilities, see Seabold and Perktold [25]. Stationarity and invertibility are enforced, and the optimizer is supplied with the data-dependent initial value described in Appendix A.3. The optimization is limited to 50 iterations so that a small number of difficult fits cannot dominate the simulation time. A deterministic fallback is used if estimation fails, although no failures were observed in the reported experiments.

The ARMA-based summaries can be omitted because fitting a separate model to every simulated sequence is computationally expensive. They were therefore

excluded during the grid search for the summary-network architecture and included when training the final summary-statistic models used in the comparison with MCMC. This allowed the architecture to be selected without repeatedly incurring the cost of the ARMA fits.

With $Q = 19$ and $K = 4$, the summary dimension is

$$d_s = \begin{cases} 1 + 19 + 4 + 3 = 27, & \text{without ARMA-based summaries,} \\ 1 + 19 + 4 + 3 + 3 = 30, & \text{with ARMA-based summaries.} \end{cases} \quad (3.4)$$

Consequently,

$$x_{1:N, 1:d_s}^S \in \begin{cases} \mathbb{R}^{N \times 27}, & \text{without ARMA-based summaries,} \\ \mathbb{R}^{N \times 30}, & \text{with ARMA-based summaries.} \end{cases}$$

Since the N parameter–data pairs are independent draws from (2.31), their construction can be distributed across multiple worker processes. The implementation uses `concurrent.futures.ProcessPoolExecutor` together with Python’s multiprocessing module and the `spawn` process context. Each worker generates a subset of complete training examples, including the prior draws, simulated sequences, transformed targets and architecture-specific input representations. These subsets are then concatenated to form the target and input matrices defined above.

3.1.3 Numerical safeguards and reproducibility

Several constants used by the implementation are introduced solely to prevent overflow, invalid transformations or unstable optimization. They do not represent prior information or architectural capacity choices. Table 3.2 collects these constants and their roles.

The exponent clip is applied to $H_t/2$ and therefore modifies a simulated path only when $|H_t| > 700$. At the upper boundary,

$$\exp(H_t/2) \leq \exp(350),$$

so the observation scale and its square remain representable in double precision for numerically relevant Gaussian innovations. The log-square offset has a noticeable effect only when y_t^2 is comparable to 10^{-12} , or $|y_t|$ is on the order of 10^{-6} or smaller.

The remaining floors and margins similarly act only near the boundaries of their respective transformations. Their use slightly regularizes the extreme tails of the simulated representation and approximate posterior family, but prevents one exceptional value from producing a non-finite input, target or loss. The standardization threshold, Adam denominator and gradient cap affect only the numerical optimization and do not alter the joint simulation distribution.

As a final safeguard, the summary routine replaces a non-finite feature by zero if the preceding regularizations nevertheless fail to produce a finite value.

Table 3.2: Numerical safeguards used in simulation, feature construction and neural-network training.

Quantity	Value	Role
Exponent clip c_{exp}	350	Clips $H_t/2$ before exponentiation when simulating (2.12), preventing floating-point overflow for extreme prior-predictive paths.
Log-square offset κ	10^{-12}	Makes the logarithm in (2.41) finite when an observation is zero or indistinguishable from zero in finite-precision arithmetic.
Summary safe-guard ε_{sum}	10^{-12}	Regularizes logarithm arguments, autocovariance-ratio denominators, ARMA innovation variances and the plug-in variance terms used in $S(y_{1:n})$.
Target-transform margin ε_G	10^{-6}	Clips ϕ away from $\{-1, 1\}$ and σ away from zero before evaluating G in (2.35).
Standardization threshold ε_{std}	10^{-8}	Prevents division by an estimated input standard deviation that is zero or numerically negligible.
Posterior variance floor κ_v	10^{-12}	Defines the network output variance by $v_{W,j} = \text{softplus}(h_{W,j}) + \kappa_v$, ensuring that the Gaussian score has a strictly positive variance.
Adam denomina-tor $\varepsilon_{\text{Adam}}$	10^{-8}	Prevents division by zero in the coordinatewise Adam update (2.62).
Gradient-norm cap c_g	5	Rescales the complete gradient when necessary so that $\ \nabla_W L\ _2 \leq 5$. Since the same factor is applied to every coordinate, the gradient direction is preserved.

Reproducibility is maintained through a hierarchy of NumPy `SeedSequence` objects. Before training begins, the implementation is initialized with a single user-specified *master seed*. This is the root seed from which all random-number streams used during the training run are derived. Fixing the master seed makes the complete seed hierarchy deterministic.

At the training-loop level, an epoch-specific child seed sequence is constructed from

(master seed, stream identifier, epoch index).

Here, a child seed sequence is a `SeedSequence` derived from the master seed together with additional integer labels. Its purpose is to initialize a distinct but reproducible random-number stream for a particular part of the training procedure.

The stream identifier is an integer label specifying the role of the generated data. Distinct identifiers are assigned to training and testing. Including this identifier prevents data sets with different roles but the same master seed and epoch index from being initialized from identical seed information. It also provides a clear namespace for identifying the stream from which each child seed sequence is derived. The epoch index ensures that successive epochs within the same stream receive different seed sequences.

Repeating a run with the same configuration and master seed consequently reproduces the same simulated arrays.

3.2 MCMC configuration and choice of sequence length

The MCMC benchmark is produced using `stochvol`. Its implementation requires choices of burn-in length and the number of retained posterior draws, as well as a suitable length for the observed SV sequences. The sequence length must provide sufficiently informative posteriors for the marginal Gaussian approximation used by the neural estimator to be reasonable.

We first select the burn-in and retained chain lengths by examining convergence and the effective sample size of the resulting MCMC draws. Using this configuration, we then study how the standardized marginal posterior quantiles approach the theoretical Gaussian quantiles as the observed sequence length increases, as suggested by the Bernstein–von Mises theorem. This comparison is used to argue that $n = 253$ provides a sufficiently accurate Gaussian approximation.

3.2.1 Burn-in, retained chain length and effective sample size

Following Algorithm 1, let B denote the number of discarded iterations and M the number of retained parameter draws. Preliminary trace plots indicated that the initial transient had typically disappeared within approximately 200 iterations. To provide a buffer against less favorable initializations, all reported MCMC runs used

$$B = 500.$$

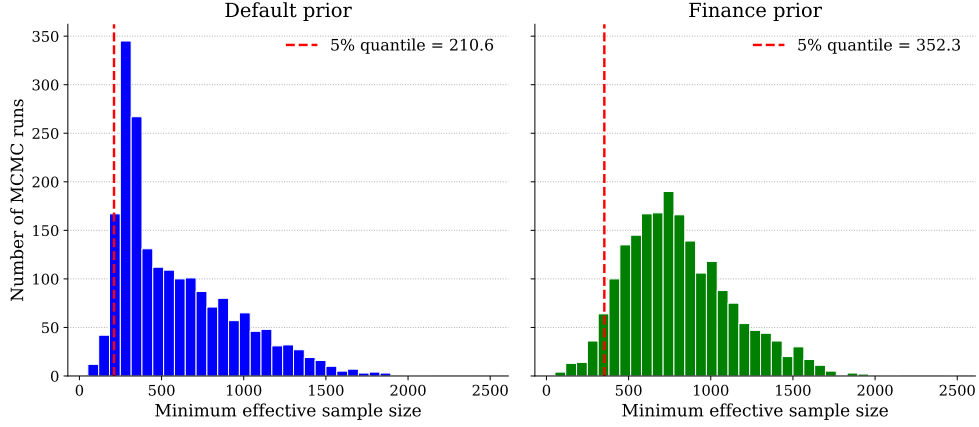


Figure 3.2: Distribution of the minimum ESS across the transformed parameter chains and their squared centered values. For each prior configuration, 2000 independently simulated SV sequences of length $n = 253$ were analyzed using $B = 500$ discarded iterations followed by $M = 20\,000$ retained draws without thinning.

The retained chain length was assessed empirically across independently simulated data sets. For a given data set, let

$$\theta_{1:3,1:M}$$

denote the retained MCMC draws, where $\theta_{1:3,m}$ is the m -th parameter draw. Define the corresponding transformed draws and their componentwise sample means by

$$z_{1:3,m} = G(\theta_{1:3,m}), \quad \bar{z}_j = \frac{1}{M} \sum_{m=1}^M z_{j,m}.$$

For each $j \in \{1, 2, 3\}$, the ESS was computed for both the transformed parameter draws and their squared centered values:

$$E_{j,1} = \text{ESS}(z_{j,1:M}), \quad u_{j,m} = (z_{j,m} - \bar{z}_j)^2, \quad E_{j,2} = \text{ESS}(u_{j,1:M}).$$

The minimum of the resulting six ESS values was recorded as

$$\text{ESS}_{\min} = \min_{j \in \{1,2,3\}} \{E_{j,1}, E_{j,2}\}. \quad (3.5)$$

Monitoring both types of sequences is necessary because the MCMC sample means and variances are later used as reference quantities when evaluating the neural posterior estimator.

A retained length of

$$M = 20\,000$$

was examined using 2000 independently simulated parameter–data pairs under each prior configuration. Each sequence had length $n = 253$ and was analyzed using $B = 500$ discarded iterations followed by $M = 20\,000$ retained draws without thinning. The resulting distributions of $\text{ESS}_{\min}$ are shown in Figure 3.2.

The fifth percentile of $\text{ESS}_{\min}$ was approximately 210.6 under the default prior and 352.3 under the finance prior. Hence, approximately 95% of the examined runs under either prior had an ESS above 200 for all six monitored chains.

An ESS of 200 was adopted as a pragmatic planning threshold. By Equations (2.28) and (2.29), this corresponds to a standardized MCSE of approximately 7.1% for an estimated posterior mean and, under the Gaussian approximation, a relative MCSE of approximately 10% for an estimated posterior variance. Since a retained length of $M = 20\,000$ produced an ESS above this threshold in approximately 95% of the runs under both prior configurations, it was used throughout the benchmark.

3.2.2 Empirical Bernstein–von Mises diagnostic and choice of sequence length

The sequence length must be sufficiently large for the marginal Gaussian posterior family used by the neural estimator to provide a reasonable approximation. The Bernstein–von Mises theorem motivates increasing n as a means of approaching this regime, although it does not provide a finite-sample guarantee.

To state the classical result, let θ_0 be the true value of a fixed-dimensional parameter and let $\hat{\theta}_n$ be an asymptotically efficient estimator, such as the maximum-likelihood estimator in a regular model. Under the conditions of the Bernstein–von Mises theorem,

$$\left\| \Pi(\sqrt{n}(\theta - \hat{\theta}_n) \in \cdot \mid Y_{1:n}) - \mathcal{N}(0, \mathcal{I}(\theta_0)^{-1}) \right\|_{\text{TV}} \xrightarrow{P_{\theta_0}} 0,$$

where $\mathcal{I}(\theta_0)$ is the Fisher information matrix, $\|\cdot\|_{\text{TV}}$ denotes total variation distance and $\xrightarrow{P_{\theta_0}}$ denotes convergence in probability under the data-generating parameter θ_0 . Informally, this means that, for large n ,

$$\theta \mid Y_{1:n} \approx \mathcal{N}\left(\hat{\theta}_n, \frac{1}{n}\mathcal{I}(\theta_0)^{-1}\right).$$

The posterior concentrates within an $n^{-1/2}$ -neighborhood of the true parameter. Within this shrinking neighborhood, the log-likelihood becomes approximately quadratic and the influence of the prior becomes negligible. Readers seeking a fuller explanation of the theorem, its notation and its assumptions are referred to Vaart [28, Section 10.2].

In the classical regular setting, the result requires conditions including that θ_0 is an interior and identifiable point, that the likelihood is sufficiently smooth and locally asymptotically normal, that the Fisher information is finite and non-singular, that the posterior concentrates at the $n^{-1/2}$ rate, and that the prior has a

positive continuous density near θ_0 . For dependent latent-state models, analogous results additionally require conditions such as stationarity, ergodicity or sufficient mixing, together with the integrability and smoothness needed to establish posterior concentration and local asymptotic normality.

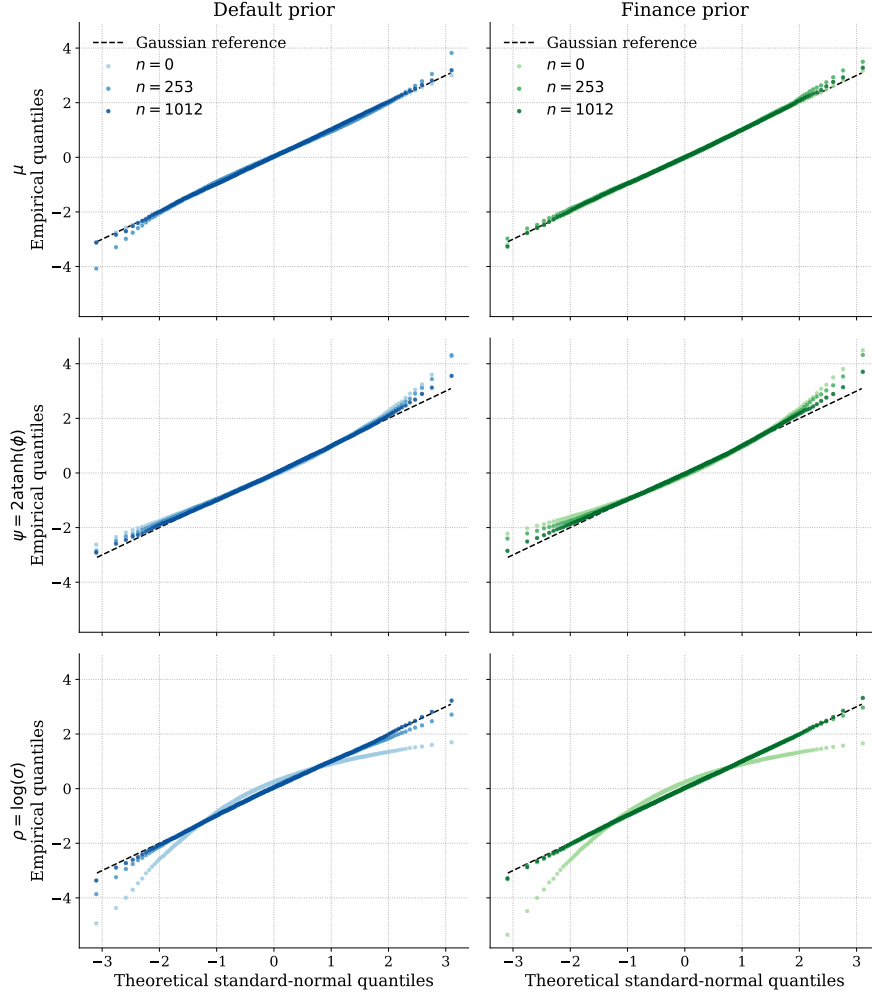


Figure 3.3: Gaussian QQ plots for standardized draws of the transformed SV parameters. The panels compare the prior, denoted by $n = 0$, with posteriors conditioned on sequences of length $n = 253$ and $n = 1012$. The MCMC chains retain 20 000 draws following a burn-in of 500 iterations.

Related Bernstein–von Mises results have been established for regular finite-state hidden Markov models with a known number of states; see Gunst and Shcherbakova [29]. These results do not directly cover the SV model considered here, whose latent process has a continuous state space. Moreover, its likelihood is not available in closed form, making verification of the required regularity conditions beyond the scope of this thesis. The theorem therefore serves mainly as motivation for examining whether the posterior becomes increasingly Gaussian as n grows.

This behavior was examined empirically using Gaussian QQ plots. A single sequence $y_{1:1012}$ was simulated from the standard SV model in (2.12). Two separate MCMC runs were performed, one conditional on $y_{1:253}$ and the other conditional on $y_{1:1012}$, while draws from the prior were included as the reference case $n = 0$. For each value of

$$n \in \{0, 253, 1012\}, \quad 1012 = 4 \cdot 253,$$

the parameter draws were mapped to the unconstrained space using G from (2.35). Each transformed coordinate was standardized using its empirical mean and standard deviation, after which its empirical quantiles were compared with the corresponding quantiles of the standard Gaussian distribution.

Figure 3.3 indicates that the transformed marginal distributions become more Gaussian as the sequence length increases. The marginal posterior of μ is already close to Gaussian at $n = 253$, although this may partly reflect its Gaussian prior. For ψ , the empirical quantiles lie above the Gaussian reference line in both tails, indicating positive skewness: the left tail is shorter and the right tail is longer than under a Gaussian distribution. For ρ , both tails lie below the reference line, indicating negative skewness, with a longer left tail and a shorter right tail. These departures are strongest for the prior at $n = 0$, remain visible at $n = 253$, and are substantially reduced at $n = 1012$. Hence, for the simulated sequence considered here, the marginal skewness decreases in magnitude as the sequence length increases.

At $n = 1012$, all three transformed marginal posteriors closely follow their Gaussian reference lines. The figure therefore provides a marginal diagnostic consistent with Bernstein–von Mises behavior, as the influence of the non-Gaussian prior and the degree of posterior asymmetry diminish with increasing sequence length.

The standard sequence length was therefore fixed at

$$n = 253.$$

At this length, the transformed marginal posteriors are sufficiently close to Gaussian for the intended approximation, despite some remaining positive skewness in ψ and negative skewness in ρ . The value also corresponds to one trading year under the 253-day convention used in this thesis, giving it a natural interpretation with the finance prior. Increasing the length to $n = 1012$ improves the Gaussian approximation further, but represents four trading years and substantially increases the cost of simulation-based training and MCMC estimation. The choice $n = 253$ therefore balances finite-sample Gaussianity, financial interpretation and computational cost.

3.2.3 Fixed MCMC and sequence-length configuration across model simulations

The MCMC baseline and neural posterior estimator are compared across many independently simulated sequences from the prior-predictive distribution (2.33),

Table 3.3: MCMC and sequence-length configuration used in the benchmark experiments.

Quantity	Value	Role
B	500	Discards the initial transient, with the value chosen conservatively from preliminary trace plots.
M	20 000	Provides the retained draws. Approximately 95% of the examined runs attained an ESS above 200 for every parameter chain.
n	253	Balances marginal Gaussianity, computational cost and the interpretation of one trading year.

rather than on a single data set. Evaluating both methods on the same sequences and averaging their scores yields a more stable estimate of their expected performance. To keep this comparison simple and consistent, the burn-in length, retained chain length and sequence length were chosen to perform adequately for a large proportion of sequences under both prior configurations and were applied uniformly.

Every sequence is therefore analyzed using $B = 500$ discarded iterations and $M = 20\,000$ retained draws without thinning, with the sequence length fixed at $n = 253$. This configuration permits efficient evaluation across the prior-predictive distribution while ensuring that both methods are assessed on the same data sets under consistent conditions.

3.3 Neural-network implementation with PyTorch

The summary-statistic and temporal convolutional network posterior estimators were implemented in PyTorch rather than in a lower-level numerical framework or through the R interface to Torch. PyTorch was chosen because it brings together the main tools required for training the posterior estimators within a single, well-supported environment. In particular, it provides CUDA support for executing the networks on GPUs, a built-in Gaussian negative log-likelihood compatible with the marginal posterior approximations, and reverse-mode automatic differentiation through its autograd system. It also supports automatic mixed precision, which can reduce GPU memory use during training, and provides controls for requesting deterministic execution. Relative to Torch for R, PyTorch also offered closer integration with the broader Python ecosystem, including extensive documentation, debugging tools and community resources for developing and diagnosing custom neural-network models. These practical advantages motivated the choice of PyTorch for implementing both architectures as subclasses of `torch.nn.Module`.

For a broader account of the framework’s design and capabilities, see Paszke et al. [26].

The remainder of this section describes the PyTorch tools that are particularly relevant to the implementation. It first considers the built-in Gaussian negative log-likelihood and the use of autograd to obtain the gradients required by the optimizer. It then discusses CUDA automatic mixed precision and gradient scaling, together with the additional detection of NaN and infinite gradients used to diagnose numerical failures during training. Finally, it presents PyTorch code adjustments made to better guarantee reproducibility.

3.3.1 Gaussian negative log-likelihood and graph-based automatic differentiation

For each estimated parameter, the network returns the mean and variance of a Gaussian marginal posterior approximation. Its contribution to the training objective is evaluated using `torch.nn.functional.gaussian_nll_loss`. In PyTorch, a *tensor* is a multidimensional numerical array whose datatype and storage device are tracked by the framework and whose operations can be recorded for automatic differentiation. This loss function therefore provides a tested tensor-based implementation of the Gaussian negative log-likelihood that can be incorporated directly into the complete training loss.

PyTorch obtains the derivatives required for optimization through *automatic differentiation*. Automatic differentiation represents the numerical evaluation of a function as a sequence of elementary operations and applies the chain rule to those operations. When the mini-batch loss $L_B^{(t)}(W_{t-1})$ is evaluated, `torch.autograd` records the differentiable tensor operations and their dependencies in a *computational graph*. This directed graph connects the network weights, through the intermediate tensor calculations, to the resulting scalar loss. Calling `backward()` traverses the graph in reverse and applies the chain rule to compute all components of the stochastic gradient g_t defined in (2.57). The resulting derivatives are stored with the corresponding trainable parameters. This reverse-mode calculation is particularly suitable when a scalar loss depends on many network weights; see Baydin et al. [30] for a review of automatic differentiation.

Automatic differentiation differs from symbolic differentiation, which constructs algebraic expressions for the derivatives, and from finite-difference methods, which approximate them by evaluating the function at perturbed parameter values. PyTorch instead differentiates the numerical operations executed by the program. It thereby avoids both the potentially large expressions produced by symbolic differentiation and the choice of a finite-difference step size with its associated truncation error. The computed gradients nevertheless remain subject to floating-point rounding error.

This graph-based differentiation system is a central reason for choosing PyTorch. Changes to differentiable tensor calculations are automatically reflected in the computational graph and hence in the resulting gradients. Network compo-

nents, transformations and loss constructions can therefore be modified without manually deriving and implementing a new collection of partial derivatives. Such derivations would be cumbersome for networks containing many composed operations and would need to be reconsidered whenever the model were changed. Finite-difference gradients would instead require repeated loss evaluations under separate perturbations of the many network weights. PyTorch therefore provides both the Gaussian loss component and the gradients required by the optimizer within a common framework.

3.3.2 CUDA acceleration, automatic mixed precision and gradient diagnostics

CUDA is NVIDIA's parallel-computing platform and programming model for general-purpose computation on compatible GPU. As described by Nickolls et al. [31], CUDA exposes the many processing cores of a GPU to data-parallel numerical computations. In PyTorch, a CUDA device denotes an available CUDA-compatible GPU, and `torch.cuda` allows tensors and supported operations to be placed on this device. PyTorch thereby integrates GPU execution with its tensor and automatic-differentiation systems, allowing the training calculations to benefit from GPU parallelism without requiring a separate low-level CUDA implementation [26].

PyTorch provides CUDA automatic mixed precision through `torch.amp.autocast` and `torch.amp.GradScaler`. Following the mixed-precision training strategy described by Micikevicius et al. [32], autocasting selects lower-precision datatypes for suitable operations while retaining `float32` where greater numerical range or precision is needed. It therefore does not convert the entire calculation to a single lower-precision datatype. This selective use of lower precision can reduce the memory occupied by intermediate activations and improve throughput on supported GPUs. In the present implementation, autocasting is disabled for variance-sensitive calculations and the Gaussian negative log-likelihood, which are always evaluated in `float32` to improve numerical stability.

TensorFloat-32 is a separate acceleration mode for certain operations with nominally `float32` inputs and is not part of automatic mixed precision. It is disabled so that operations retained in `float32` are not evaluated using reduced internal precision.

Small gradients may underflow when represented in lower precision. PyTorch's adaptive gradient scaler mitigates this problem by multiplying the loss before automatic differentiation and adjusting the scale during training. The gradients are returned to their ordinary scale before the parameter update, and the update is skipped when non-finite gradients are detected. Gradients must likewise be unscaled before they are inspected or capped, otherwise their reported magnitudes and norms include the artificial scale factor.

In addition to the scaler's internal checks, the implementation examines every available parameter gradient after unscaling and reports the epoch and mini-batch whenever a NaN or infinite value is found. Such values indicate numerical insta-

bility, including overflow or invalid arithmetic. This additional diagnostic makes numerical failures and skipped optimizer steps easier to identify during training.

3.3.3 Deterministic execution and reproducibility

Reproducible training requires both consistent random draws and deterministic numerical operations. After the relevant random-number generators have been initialized with fixed seeds, PyTorch can be configured to request deterministic implementations of supported operations. This is particularly important during GPU training, where several algorithms may be available for the same calculation.

For CUDA operations, PyTorch may use the NVIDIA CUDA Deep Neural Network library (cuDNN), which provides optimized GPU implementations of common neural-network operations such as convolutions and pooling. By default, cuDNN may benchmark several algorithms to identify the fastest implementation for a given input shape. Disabling this benchmarking avoids algorithm choices that may vary between runs, while `torch.backends.cudnn.deterministic = True` requests deterministic implementations for the relevant cuDNN operations.

The broader instruction

```
torch.use_deterministic_algorithms(True)
```

extends this requirement beyond cuDNN. PyTorch then selects deterministic implementations where they are available and reports an error when an operation has no deterministic alternative. For operations covered by this mechanism, the absence of a deterministic implementation is therefore made explicit rather than being accepted silently. These controls are useful in scientific work because they make repeated training runs easier to compare and support independent verification of the reported results.

Requesting deterministic algorithms and disabling performance-oriented features may reduce training throughput. These settings nevertheless improve repeatability within a fixed computational environment. They do not guarantee bitwise-identical results across different hardware, PyTorch releases, CUDA versions or supporting libraries. To facilitate reproduction of the software environment, a dependency specification listing the Python packages and versions used for the experiments is included with the source code in the GitHub repository linked in Appendix A.1.

3.4 Early stopping and benchmark evaluation

Algorithm 2 defines the weight sequence $W_0, W_1, \dots, W_T$, where T places an upper limit on the number of Adam updates. This limit prevents training from continuing indefinitely, but does not determine whether further updates continue to improve the posterior approximation. The network is therefore evaluated after each training epoch on an independently simulated validation sample using the marginal

Gaussian loss in (2.40). The validation loss is used to retain the best-performing network weights and to determine when training should be terminated early.

Because parameter–data pairs can be simulated as needed, a new validation sample is generated at every epoch rather than repeatedly evaluating the network on one fixed sample. This reduces the risk that the retained weights and stopping decision become overly adapted to a particular validation set. The validation losses consequently contain additional Monte Carlo variation, which is moderated by using a validation sample larger than the training sample generated during one epoch and by requiring several consecutive epochs without sufficient improvement before stopping.

The first subsection defines this validation-based stopping rule. The second introduces the marginal Gaussian loss difference, root mean squared error and log variance ratio used to compare the trained neural estimator with the MCMC reference.

3.4.1 Simulation-based validation and early stopping

Let b denote the mini-batch size and let J denote the number of mini-batches processed during one epoch. At the beginning of each epoch, the implementation generates Jb new training pairs in one parallel simulation call. These pairs are partitioned into J mini-batches, each used for one Adam update and then discarded. After epoch e , the update index is $t = eJ$, and the current network weights are W_t .

The quantity T from Algorithm 2 is the maximum permitted number of Adam updates. Since J is fixed during training and T is chosen to be divisible by J , the corresponding maximum number of epochs is

$$E_{\max} = \frac{T}{J}. \quad (3.6)$$

This upper limit terminates the optimization if the validation-based stopping rule is not reached earlier.

After each epoch, the current weights are evaluated on a newly simulated validation sample of size N_{val} , generated independently of the training pairs used during that epoch. Let V_e denote the marginal Gaussian loss in (2.40), evaluated at W_{eJ} using this validation sample. The validation pairs are not used to calculate gradients or update the network weights. The value V_e is used only to decide whether the current weights should be retained and whether training should continue.

The validation size is selected separately for the two neural-network implementations rather than being defined as a fixed multiple of Jb . Constructing an input for the summary network requires calculating the selected summary statistics from each simulated sequence, whereas constructing a TCN input requires substantially less additional processing. Generating training and validation inputs is therefore more computationally expensive for the summary network. This difference permits larger mini-batches, more batches per epoch and a larger validation

sample for the TCN. In both implementations, N_{val} is considerably larger than one training mini-batch, which reduces the Monte Carlo variation of the validation loss.

A new validation sample is used at every epoch. The retained weights are therefore not selected through repeated evaluation on the same finite collection of parameter–data pairs. However, successive validation losses are then calculated from different samples, so variation in V_e reflects both changes in the network weights and Monte Carlo variation in the validation calculation.

Let V_{best} be the lowest qualifying validation loss recorded so far, let $\delta > 0$ be the minimum required improvement and let c count the number of consecutive epochs without such an improvement. Epoch e is recorded as an improvement when

$$V_e < V_{\text{best}} - \delta. \quad (3.7)$$

When this condition holds, V_{best} is replaced by V_e , a copy of W_{eJ} is stored as W_{best} , and c is reset to zero. Otherwise, c is increased by one. Training terminates when c reaches the patience P , or when all E_{max} epochs have been completed.

An unusually favorable validation sample may produce a low V_{best} that the following epochs do not immediately improve upon. The patience gives later weight iterates several opportunities to produce a qualifying improvement on newly simulated validation samples before training is terminated. The larger validation samples reduce the underlying Monte Carlo variation. The TCN used a larger validation sample, and its greater computational cost per epoch also favored a somewhat smaller patience.

Table 3.4 gives the training and early-stopping configurations used for the two neural estimators. Each listed quantity was held fixed throughout the corresponding training run.

Table 3.4: Training and early-stopping configurations used for the two neural estimators.

Quantity	Summary network	TCN
Mini-batch size b	1024	4096
Mini-batches per epoch J	10	100
Training simulations per epoch Jb	10240	409600
Validation size N_{val}	20000	204800
Maximum epochs E_{max}	2000	2000
Maximum Adam updates $T = JE_{\text{max}}$	20000	200000
Patience P	100	75
Minimum improvement δ	10^{-5}	10^{-5}

Algorithm 3 summarizes the complete procedure. When training terminates, the stored weights W_{best} are restored rather than retaining the weights produced during the final attempted epoch. The restored network is then evaluated using

the separate final-validation seed stream. These parameter–data pairs were not used for either the Adam updates or the selection of W_{best} .

Algorithm 3 Simulation-based training with early stopping

Require: Initial weights W_0 ; maximum number of Adam updates T

Require: Mini-batch size b ; mini-batches per epoch J

Require: Validation size N_{val}

Require: Minimum improvement δ ; patience P

Require: T divisible by J

```

1:  $E_{\text{max}} \leftarrow T/J$ 
2:  $V_{\text{best}} \leftarrow +\infty$ 
3:  $W_{\text{best}} \leftarrow W_0$ 
4:  $c \leftarrow 0$ 
5:  $t \leftarrow 0$ 
6: for  $e = 1, \dots, E_{\text{max}}$  do
7:   Generate  $Jb$  new training pairs using the training seed stream
8:   Partition the training pairs into  $J$  mini-batches
9:   for  $j = 1, \dots, J$  do
10:     $t \leftarrow t + 1$ 
11:    Evaluate the mini-batch loss and its gradient at  $W_{t-1}$ 
12:    Apply one Adam update to obtain  $W_t$ 
13:   end for
14:   Generate  $N_{\text{val}}$  new pairs using the validation seed stream
15:    $V_e \leftarrow$  marginal Gaussian loss at  $W_t$  on the validation sample
16:   if  $V_e < V_{\text{best}} - \delta$  then
17:      $V_{\text{best}} \leftarrow V_e$ 
18:      $W_{\text{best}} \leftarrow W_t$ 
19:     Store a copy of  $W_{\text{best}}$ 
20:      $c \leftarrow 0$ 
21:   else
22:      $c \leftarrow c + 1$ 
23:   end if
24:   if  $c \geq P$  then
25:     break
26:   end if
27: end for
28: Restore  $W_{\text{best}}$ 
29: Evaluate  $W_{\text{best}}$  using the final-validation seed stream
30: return  $W_{\text{best}}$ 

```

3.4.2 Metrics for comparison with the MCMC reference

The Gaussian loss in (2.40) is used during training and validation because its sample average approximates the population loss in (2.54). The same loss also

provides a natural basis for comparison with the `stochvol` MCMC reference.

For each transformed marginal posterior, the MCMC draws are used to estimate its mean and variance. These estimates define an estimated moment-matched Gaussian reference that can be compared directly with the Gaussian marginal returned by the neural estimator.

Let

$$\left\{ \left(\tilde{\theta}_{1:3}^{(i)}, y_{1:n}^{(i)} \right) \right\}_{i=1}^K$$

be K independently simulated benchmark pairs generated according to (2.31). For the restored network weights W_{best} , define the NPE mean and variance for transformed parameter $j \in \{1, 2, 3\}$ by

$$\begin{aligned} \widehat{m}_{\text{NPE},j}^{(i)} &= \widehat{m}_{W_{\text{best}},j} \left(y_{1:n}^{(i)} \right), \\ \widehat{v}_{\text{NPE},j}^{(i)} &= \widehat{v}_{W_{\text{best}},j} \left(y_{1:n}^{(i)} \right). \end{aligned}$$

For each benchmark sequence $y_{1:n}^{(i)}$, let

$$\theta_{1:3,1:M}^{(i)} \in \mathbb{R}^{3 \times M}$$

denote the retained MCMC draws, where $\theta_{1:3,m}^{(i)}$ is the m th SV parameter draw. Define the transformed draws by

$$z_{1:3,m}^{(i)} = G \left(\theta_{1:3,m}^{(i)} \right), \quad m = 1, \dots, M.$$

The MCMC estimates of the transformed marginal posterior mean and variance are

$$\begin{aligned} \widehat{m}_{\text{MCMC},j}^{(i)} &= \frac{1}{M} \sum_{m=1}^M z_{j,m}^{(i)}, \\ \widehat{v}_{\text{MCMC},j}^{(i)} &= \frac{1}{M} \sum_{m=1}^M \left(z_{j,m}^{(i)} - \widehat{m}_{\text{MCMC},j}^{(i)} \right)^2. \end{aligned}$$

Extending the definitions of the marginal loss (2.39) and overall loss (2.40), let $L_{K,j}^A$ and L_K^A denote the corresponding mean losses for method

$$A \in \{\text{TCN}, \text{Summary NN}, \text{MCMC}\}$$

over the K benchmark pairs, where

$$L_{K,j}^A = \frac{1}{K} \sum_{i=1}^K \left[\frac{\log \widehat{v}_{A,j}^{(i)}}{2} + \frac{\left(\tilde{\theta}_j^{(i)} - \widehat{m}_{A,j}^{(i)} \right)^2}{2 \widehat{v}_{A,j}^{(i)}} \right], \quad L_K^A = \frac{1}{3} \sum_{j=1}^3 L_{K,j}^A. \quad (3.8)$$

For two methods A and B , define their marginal and overall mean loss differences by

$$L_{K,j}^{A-B} = L_{K,j}^A - L_{K,j}^B, \quad L_K^{A-B} = L_K^A - L_K^B. \quad (3.9)$$

The corresponding standard deviations are estimated from the loss differences within each benchmark pair, thereby retaining the paired structure of the comparison.

For a neural method $A \in \{\text{TCN}, \text{SummaryNN}\}$, its estimated marginal and overall loss differences relative to the MCMC reference are written as

$$\widehat{\Delta}_{\text{loss},j}^A = L_{K,j}^{A-\text{MCMC}}, \quad \widehat{\Delta}_{\text{loss}}^A = L_K^{A-\text{MCMC}}. \quad (3.10)$$

Values close to zero indicate similar mean Gaussian losses, whereas positive values indicate a larger mean loss for the neural method. Under the idealized moment-matched reference, the corresponding population loss difference is nonnegative. Its estimate may nevertheless be slightly negative because of finite-sample variation and Monte Carlo error in the MCMC moment estimates.

The loss difference combines discrepancies in posterior location and uncertainty and does not indicate which of these is responsible for the observed difference. It is therefore supplemented with separate measures of posterior-mean accuracy, posterior variance and interval coverage. All metrics are calculated from the same K benchmark pairs and reported separately for μ , ψ and ρ .

To examine posterior location separately, the posterior means are evaluated against the transformed parameters that generated the benchmark sequences. For method A , the root mean squared error is

$$\text{RMSE}_j^A = \left[\frac{1}{K} \sum_{i=1}^K (\tilde{\theta}_j^{(i)} - \widehat{m}_{A,j}^{(i)})^2 \right]^{1/2}. \quad (3.11)$$

A smaller value indicates that the estimated posterior means are, on average, closer to the generating transformed parameter. The MCMC value provides a reference for the estimation error that remains even when the posterior mean is accurately approximated.

Agreement in posterior uncertainty is measured by the per-sequence log variance ratio

$$r_{\text{var},j}^{(i)} = \log \left(\frac{\widehat{v}_{\text{NPE},j}^{(i)}}{\widehat{v}_{\text{MCMC},j}^{(i)}} \right). \quad (3.12)$$

A value of zero indicates equal posterior variances. Negative values indicate that the NPE reports less posterior uncertainty than the MCMC reference and is therefore relatively overconfident, whereas positive values indicate that the NPE posterior is less concentrated.

The average log variance ratio

$$\bar{r}_{\text{var},j} = \frac{1}{K} \sum_{i=1}^K r_{\text{var},j}^{(i)} \quad (3.13)$$

summarizes the variance discrepancy across the benchmark sequences. Exponentiating it gives the geometric mean variance ratio $\exp(\bar{r}_{\text{var},j})$, which may be interpreted as the typical multiplicative factor by which the NPE variance differs from

the MCMC variance. Geometric averaging is natural for ratios because discrepancies by factors c and $1/c$ are treated as equally large in opposite directions. For example, a value of 1.25 indicates that the NPE variance is typically 1.25 times the MCMC variance.

Finally, empirical coverage assesses whether the uncertainty reported by each Gaussian marginal agrees with the frequency at which it contains the generating parameter. For method $A \in \{\text{NPE}, \text{MCMC}\}$ and nominal coverage level $1 - \alpha$, define

$$\hat{C}_{A,j}(1 - \alpha) = \frac{1}{K} \sum_{i=1}^K \mathbf{1} \left\{ \Phi^{-1} \left(\frac{\alpha}{2} \right) \leq \frac{\tilde{\theta}_j^{(i)} - \hat{m}_{A,j}^{(i)}}{\sqrt{\hat{v}_{A,j}^{(i)}}} \leq \Phi^{-1} \left(1 - \frac{\alpha}{2} \right) \right\}, \quad (3.14)$$

where Φ is the cumulative distribution function of the standard Gaussian distribution. For a calibrated Gaussian marginal, the empirical coverage should be close to $1 - \alpha$. Coverage below the nominal level indicates that the intervals are too narrow, displaced, or otherwise misspecified, whereas coverage above it indicates conservative intervals on average.

Coverage is also calculated for the moment-matched MCMC reference. Because these intervals are constructed from the MCMC posterior means and variances, their coverage assesses whether the MCMC marginal posterior is adequately represented by the assumed Gaussian family. Comparing the two methods therefore helps determine whether inaccurate NPE coverage is caused by the neural approximation or by the Gaussian posterior assumption itself.

Together, the marginal Gaussian loss difference evaluates the complete Gaussian approximation, the RMSE isolates the accuracy of its mean, the log variance ratio measures agreement in posterior uncertainty, and empirical coverage assesses the calibration of its credible intervals.

3.5 Tuning-parameter selection

This section describes the selection of the architectural and optimization settings used for the two neural estimators. The term *tuning parameters* refers here to choices fixed before training, including network depth and width, convolutional kernel sizes and dilations, pooling settings, and the Adam step size. These quantities are distinct from the trainable network weights, the parameters of the SV model, and the hyperparameters of its prior distributions.

Evaluating a candidate architecture requires training it on repeatedly simulated parameter–data pairs. A broad joint search over all tuning parameters was therefore computationally infeasible. The summary-statistic architecture was selected using a small, reproducible grid search under the default prior, whereas the TCN was selected through targeted pilot experiments subject to a receptive-field requirement. The final subsection states the common Adam configuration and compares the architectures, parameter counts, and computational requirements of the selected networks.

3.5.1 Tuning the summary-statistic network

Both neural estimators contain a shared encoder followed by three parameter-specific prediction heads. For the summary-statistic network, the encoder and all three heads are multilayer perceptrons (MLP) as defined in 2.6.1. The encoder and each head contain two hidden layers. A width tuple

$$(d_1, d_2)$$

therefore denotes an MLP with d_1 units in its first hidden layer and d_2 units in its second. The same head-width tuple is used for every parameter-specific head.

A Cartesian grid was evaluated over the shared encoder widths

$$(64, 32), \quad (128, 64), \quad (256, 128),$$

and the head widths

$$(16, 16), \quad (32, 32), \quad (64, 64).$$

This gives nine candidate architectures. All trials used the default prior, $Q = 19$ quantiles, a common master seed, and the same training and validation procedure. After training, the restored weights for each candidate was evaluated on a fresh validation sample generated using the same procedure and sample size as the validation samples used during training.

Table 3.5: Post-training validation losses under the default prior for the summary-statistic network. The smallest loss is shown in bold.

Shared encoder widths	Head widths		
	(16, 16)	(32, 32)	(64, 64)
(64, 32)	0.535868	0.554196	0.569154
(128, 64)	0.527872	0.550523	0.556307
(256, 128)	0.579887	0.537528	0.541043

The smallest validation loss was obtained with encoder widths (128, 64) and head widths (16, 16). This configuration was therefore selected for the final summary-statistic estimators under both the default and finance priors.

The grid search was conducted using the 27-dimensional summary vector obtained with the three ARMA(1,1)-based summaries disabled. Estimating an ARMA(1,1) model for every simulated sequence would have substantially increased the cost of evaluating all nine architectures. The selected widths were retained when these summaries were included, giving the 30-dimensional input used by the final estimator in the comparisons with MCMC. They were also retained under the finance prior rather than repeating the grid search under a second prior configuration.

This staged procedure assumes that neither the additional summaries nor the change of prior significantly alters the ideal network widths. It provides a practical architecture under the available computational budget but does not establish a global optimum over the joint choice of architecture, summaries, and prior.

3.5.2 Tuning the temporal convolutional network

For the second neural estimator, the shared encoder is the TCN defined in Section 2.6.3. Its output is passed to three parameter-specific MLP heads, each containing two hidden layers.

A broad Cartesian grid search over the TCN tuning parameters was not computationally feasible. Each candidate processes the complete transformed sequence through several convolutional layers, and shortening the input below $n = 253$ to reduce the computational cost would change the inference problem. The architecture was therefore selected using targeted pilot experiments.

The principal structural requirement was that the receptive field of the final temporal features cover the complete input sequence. The selected stack has $L = 5$ residual blocks with channel widths

$$(C_1, \dots, C_5) = (16, 32, 32, 64, 64)$$

and kernel sizes

$$(k_1, \dots, k_5) = (9, 9, 7, 5, 5).$$

Bai et al. [20] describe an exponentially increasing dilation schedule as a means of expanding the receptive field with network depth. Following this construction, the dilation factors were set to

$$d_\ell = 2^{\ell-1}, \quad \ell = 1, \dots, 5,$$

which gives

$$(d_1, \dots, d_5) = (1, 2, 4, 8, 16).$$

Because each residual function contains two convolutional layers, the temporal radius derived in Section 2.6.3 is

$$R_T = \sum_{\ell=1}^5 (k_\ell - 1)d_\ell = 144. \quad (3.15)$$

The corresponding receptive-field width is

$$1 + 2R_T = 289 > 253.$$

Central features in the final temporal map can consequently depend on the complete input sequence. The selected depth, kernel sizes, and dilation factors satisfy this requirement without an additional residual block.

Each parameter-specific head has hidden widths (32, 32). The complete configuration trained stably and achieved adequate validation performance in the

pilot experiments. It was therefore retained for the final comparisons with Monte Carlo Markov chain (MCMC). Because the choice was based on targeted experiments rather than a broad search, it should be regarded as a practical configuration rather than a demonstrated optimum. Its empirical adequacy is assessed through the moment estimations of the posterior in the results chapter.

3.5.3 Adam configuration and comparison of the two networks

The Adam update was introduced in Section 2.7. Both neural estimators were trained using

$$\alpha_{\text{Adam}} = 5 \times 10^{-4}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \varepsilon_{\text{Adam}} = 10^{-8}. \quad (3.16)$$

The decay rates and stabilizing constant were fixed at the standard values proposed by Kingma and Ba [22]. The step size was selected through pilot training rather than a separate exhaustive search. A value of 5×10^{-4} provided stable optimization while allowing sufficient progress per update when combined with the standardized network inputs and Adam’s adaptive rescaling.

Table 3.6 contains the principal architectural settings selected above, the Adam step size, and the resulting numbers of trainable parameters. The lengths of the encoder-width and channel tuples give the numbers of hidden layers and residual blocks, respectively. The remaining Adam coefficients are common to both estimators and are given in (3.16). The parameter counts refer to the final benchmark configurations, including all three prediction heads and the restored ARMA(1, 1)-based summaries as given by 2.6.2.

Table 3.6: Selected architectural and optimization settings and trainable parameter counts for the two neural estimators.

Quantity	Summary-statistic network	TCN
Input dimension	30	253
Encoder widths or block channels	(128, 64)	(16, 32, 32, 64, 64)
Head hidden widths	(16, 16)	(32, 32)
Kernel sizes	–	(9, 9, 7, 5, 5)
Dilations	–	(1, 2, 4, 8, 16)
Top- K pooling fraction α	–	0.05
Encoder parameters	12 608	105 392
Three-head parameters	4 230	21 894
Total trainable parameters	16 838	127 286
Adam step size α_{Adam}	5×10^{-4}	5×10^{-4}

The TCN contains

$$\frac{127\,286}{16\,838} \approx 7.56$$

times as many trainable parameters as the summary-statistic network. This difference reflects the dimensions and roles of their input representations. The summary-statistic network receives a 30-dimensional vector in which the observed sequence has already been reduced to a fixed collection of features. Information removed by this reduction cannot be recovered by increasing the widths of the encoder or prediction heads. The input representation is therefore assumed to be the principal limitation on how much information the summary-statistic estimator can use.

The TCN, by contrast, receives the complete transformed sequence $x_{\text{TCN}} \in \mathbb{R}^{253}$. Its encoder must extract the relevant temporal features from this larger input, and its depth is tied to the sequence length through the requirement that the receptive field cover all 253 time points. This motivates the five residual blocks and the greater capacity assigned to both the temporal encoder and the prediction heads. The resulting network requires more computation for both evaluation and the gradient calculations performed during training.

The construction of the network inputs gives the opposite computational trade-off. Computing x_{TCN} requires only applying logsq as in (2.41) and standardizing the result using its analytically derived moments. Constructing the final summary vector additionally requires estimating the $\text{ARMA}(1, 1)$ coefficients of $\text{logsq}(y_{1:n})$ using an iterative ARIMA routine with a limit of 50 iterations for each simulated sequence. Input construction is therefore more expensive for the summary-statistic estimator, even though its network evaluation and gradient computation are less expensive. For the TCN, input construction is comparatively simple, while optimization of the larger network requires more computation.

Chapter 4

Results

This chapter evaluates the TCN and Summary NN under the default and finance priors. The main benchmark consists of $K = 5\,000$ independently simulated parameter–data pairs with sequence length $n = 253$. For each sequence, the neural estimates are compared with moment-matched Gaussian marginals constructed from $M = 20\,000$ retained MCMC draws.

For readability, component-indexed quantities in this chapter use the parameter labels $j \in \{\mu, \psi, \rho\}$ rather than the numerical indices $j = 1, 2, 3$ used in Section 3.4.2, with the convention $(x_\mu, x_\psi, x_\rho) = (x_1, x_2, x_3)$.

The chapter first examines the validation-loss histories and the Monte Carlo uncertainty in the benchmark loss estimates. The methods are then compared in terms of posterior-mean RMSE, posterior variance, marginal Gaussian loss, and empirical coverage. A reduced-data MCMC comparison provides an interpretable scale for the TCN loss, while parameter sweeps illustrate the resulting credible intervals on the original parameter scale. Finally, the runtimes of the three posterior-moment estimators are compared.

4.1 Validation loss and Monte Carlo uncertainty

This section considers the marginal Gaussian loss in the two settings in which it is used. First, the validation-loss histories show how the loss changes during training and how the early-stopping rule determines when training is terminated. Second, the Monte Carlo uncertainty in the benchmark mean losses and their paired differences is examined. This establishes whether the finite benchmark sample is sufficiently large to support the loss comparisons used throughout the remainder of the chapter.

4.1.1 Validation loss during training

Figure 4.1 shows the validation-loss histories for both neural estimators under the default and finance priors. The marginal Gaussian loss is plotted on a logarithmic

vertical scale. Each point is calculated from the newly simulated validation sample generated after the corresponding epoch.

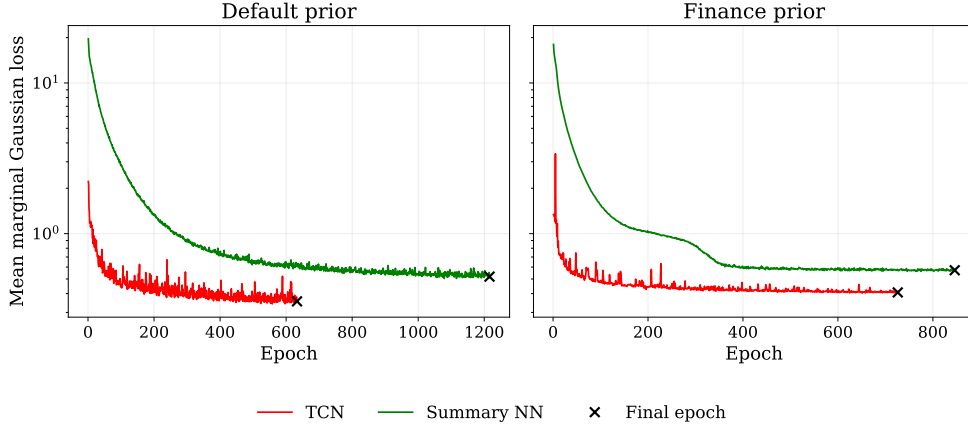


Figure 4.1: Validation-loss histories for the TCN and Summary NN under the default and finance priors. The vertical axis is logarithmic, and the crosses mark the epochs at which training terminated. All four runs were terminated by the early-stopping rule before $E_{\max} = 2000$. The restored weights correspond to the best qualifying checkpoint and therefore need not be those from the marked terminal epoch.

All four histories decrease rapidly during the initial part of training before entering a regime of slower and non-monotone improvement. None shows a sustained increase that would indicate unstable optimization. Under both priors, the TCN attains lower validation losses than the Summary NN.

The TCN histories show greater epoch-to-epoch variation even though each validation estimate is calculated from a larger sample. As shown in (2.59), increasing the validation-sample size reduces the Monte Carlo variance when the distribution of the per-simulation loss is held fixed. It does not ensure less variation across architectures, since the per-simulation loss distributions may differ. Moreover, the frequency of the weights being updated differs where TCN updates its weights 100 times before being evaluated, while summary NN only updates 10 times before evaluation. The TCN history therefore records the accumulated effect of ten times as many updates between successive displayed points, and is likely the main cause of the observed higher variance in the validation loss.

For the Summary NN under the finance prior, the initial decrease is followed by a period of comparatively slow improvement and then a second marked decrease, which persists. This is consistent with an improvement in the fitted network rather than a single unusually favorable validation sample.

This renewed improvement illustrates the purpose of allowing several epochs without a qualifying improvement before terminating training. A patience period allows temporary plateaus and fluctuations to pass without premature termination, at the cost of additional training. The stochasticity of the Adam updates may

also contribute to renewed progress. Under suitable assumptions, stochastic gradient noise can help adaptive methods move away from saddle points [33].

Although the TCN runs terminated at smaller epoch indices, epochs are not comparable units of computational effort for the two estimators. Not only does a TCN epoch contain ten times as many Adam updates, but it also includes forty times as many training simulations and approximately ten times as many validation simulations. Together with the greater cost of each network evaluation and gradient calculation, this made the TCN substantially more computationally expensive to train.

4.1.2 Relative standard deviations of the loss estimates

The mean losses $L_{K,j}^A$ and L_K^A , together with their paired differences $L_{K,j}^{A-B}$ and L_K^{A-B} , were defined in (3.8)–(3.9). All are estimated from the same $K = 5000$ benchmark pairs.

For any nonzero mean loss or mean loss difference Q , define its relative standard deviation by

$$\text{RSD}(Q) = \frac{\widehat{\text{SD}}(Q)}{|Q|}. \quad (4.1)$$

For a difference between methods A and B , the standard deviation is estimated from the loss differences within each benchmark pair.

Table 4.1: Relative standard deviations of the parameter-wise mean losses, overall mean losses, and their paired differences, calculated from $K = 5000$ benchmark pairs. In each column heading, the dot in the superscript denotes the method or method difference given in the first column.

Estimate	$\text{RSD}(L_{K,\mu}^{\cdot})$	$\text{RSD}(L_{K,\psi}^{\cdot})$	$\text{RSD}(L_{K,\rho}^{\cdot})$	$\text{RSD}(L_K^{\cdot})$
MCMC (default)	0.0158	0.1479	0.0278	0.0164
TCN (default)	0.0153	0.3105	0.0316	0.0173
Summary NN (default)	0.0223	0.1292	0.0471	0.0270
TCN – MCMC (default)	0.0957	0.0865	0.0687	0.0539
Summary NN – MCMC (default)	0.0615	0.0437	0.0392	0.0301
Summary NN – TCN (default)	0.0919	0.0573	0.0554	0.0422
MCMC (finance)	0.0320	0.0921	0.0187	0.0176
TCN (finance)	0.0330	0.1010	0.0203	0.0187
Summary NN (finance)	0.0437	0.3683	0.0276	0.0313
TCN – MCMC (finance)	0.2365	0.1660	0.0757	0.0744
Summary NN – MCMC (finance)	0.1146	0.0481	0.0345	0.0338
Summary NN – TCN (finance)	0.1319	0.0486	0.0418	0.0385

Table 4.1 shows that, for the individual methods, the RSDs of the overall mean losses range from 0.0164 to 0.0313. For the paired overall mean loss differences,

the RSDs range from 0.0301 to 0.0744. Since the largest of these values is 0.0744, every paired overall difference is at least

$$\frac{1}{0.0744} \approx 13.4$$

estimated standard deviations from zero. The overall mean loss differences are therefore large relative to their estimated Monte Carlo uncertainty.

The parameter-wise columns in Table 4.1 show greater variation in the RSDs. Among the losses for the individual methods, the two largest values are 0.3105 for the TCN under the default prior and 0.3683 for the Summary NN under the finance prior, both for ψ . Among the paired parameter-wise differences, the largest RSD is 0.2365 for the TCN minus MCMC difference for μ under the finance prior. The next largest is 0.1660 for the corresponding difference for ψ . These parameter-wise estimates have greater relative Monte Carlo uncertainty than the overall loss differences.

Overall, Table 4.1 indicates that the benchmark sample is sufficiently large for the subsequent comparisons of L_K^A and L_K^{A-B} . Their estimated standard deviations are therefore not repeated in the later tables.

4.2 Comparison with the MCMC reference

The benchmark metrics first examine posterior location and uncertainty separately through the posterior-mean RMSE and variance ratio. Their combined effect is then examined through the marginal Gaussian loss in (2.39). Since the TCN has a smaller marginal loss difference than the Summary NN for every transformed parameter under both priors, it is selected for the subsequent comparison with MCMC references conditioned on shorter sequences. All results are calculated from the same $K = 5\,000$ benchmark pairs. See 3.4.2 for the metric definitions.

4.2.1 Posterior-mean RMSE

Table 4.2 reports the RMSE of the estimated posterior means relative to the transformed parameters that generated the benchmark sequences. For every transformed parameter under both priors, the MCMC posterior mean has the smallest RMSE, followed by the TCN and then the Summary NN.

The MCMC posterior mean provides a reference for the estimation error that remains under the posterior. The percentage increase in RMSE relative to MCMC shows how closely each neural estimator preserves this posterior-mean accuracy over the benchmark pairs.

Under the default prior, the TCN RMSE exceeds the MCMC RMSE by 9.6% for μ , 3.2% for ψ and 2.2% for ρ . The corresponding increases for the Summary NN are 42.1%, 15.6% and 10.3%. The TCN therefore remains substantially closer to the MCMC reference.

Table 4.2: Posterior-mean RMSEs relative to the generating transformed parameters, estimated from $K = 5\,000$ benchmark pairs.

Method	RMSE_μ	RMSE_ψ	RMSE_ρ
MCMC (default)	0.5150	0.6513	0.6068
TCN (default)	0.5644	0.6719	0.6204
Summary NN (default)	0.7320	0.7527	0.6694
MCMC (finance)	0.5263	0.5850	0.4831
TCN (finance)	0.5340	0.5958	0.4890
Summary NN (finance)	0.5450	0.6814	0.5506

Under the finance prior, the TCN RMSE is only 1.5%, 1.8% and 1.2% larger than the MCMC RMSE for μ , ψ and ρ , respectively. For the Summary NN, the corresponding increases are 3.6%, 16.5% and 14.0%. The finance prior therefore brings the Summary NN much closer to MCMC for μ , while discrepancies for ψ and ρ are widened.

4.2.2 Posterior variance

Table 4.3 reports the geometric mean variance ratios $\exp(\bar{r}_{\text{var},j})$. A value greater than one indicates that the NPE variance exceeds the corresponding MCMC variance on a geometric average over the benchmark pairs.

Table 4.3: Geometric mean ratios of NPE marginal posterior variance to MCMC posterior variance, estimated from $K = 5\,000$ benchmark pairs and $M = 20\,000$ retained MCMC draws per pair.

Method	$\exp(\bar{r}_{\text{var},\mu})$	$\exp(\bar{r}_{\text{var},\psi})$	$\exp(\bar{r}_{\text{var},\rho})$
TCN (default)	1.1430	1.1856	1.1864
Summary NN (default)	1.4706	1.5956	1.8765
TCN (finance)	1.0095	1.0849	1.1643
Summary NN (finance)	1.1660	1.4411	1.9656

All reported ratios exceed one, so both neural estimators return larger marginal variances than the moment-matched MCMC reference on a geometric average. Under the default prior, the TCN ratios range from 1.1430 to 1.1864, whereas the Summary NN ratios range from 1.4706 to 1.8765.

Under the finance prior, all three TCN ratios move closer to one. The Summary NN ratios also improve for μ and ψ , but its ratio for ρ increases from 1.8765 to 1.9656. The persistent disagreement for ψ and ρ , together with the substantially better TCN results, suggests that the predefined summaries omit sequence information relevant to the posterior uncertainty of these parameters. By operating on

the complete transformed sequence, the TCN can learn additional features of the volatility dynamics that are unavailable to the Summary NN.

4.2.3 Marginal Gaussian loss differences

The RMSE and variance ratio describe separate aspects of the Gaussian approximation. Their combined effect is measured by the loss difference in (3.10).

Table 4.4: Marginal Gaussian loss differences relative to the moment-matched MCMC reference, estimated from $K = 5000$ benchmark pairs and $M = 20000$ retained MCMC draws per pair.

Method	$\widehat{\Delta}_{\text{loss},\mu}$	$\widehat{\Delta}_{\text{loss},\psi}$	$\widehat{\Delta}_{\text{loss},\rho}$
TCN (default)	0.0592	0.0584	0.0937
Summary NN (default)	0.1871	0.2160	0.2889
TCN (finance)	0.0149	0.0233	0.0779
Summary NN (finance)	0.0783	0.1995	0.3488

The loss differences measure how closely each neural estimator reproduces the moment-matched MCMC reference under the marginal Gaussian loss. The TCN has a smaller loss difference than the Summary NN for every transformed parameter under both priors and is therefore selected as the better-performing NPE architecture.

For both neural estimators, the largest loss difference occurs for ρ . The posterior-mean RMSE for ρ does not show a corresponding increase, whereas the variance ratio is largest for ρ under both priors. Its larger loss difference is therefore primarily caused by disagreement in the estimated posterior variance rather than the posterior mean.

Taken together, the metrics indicate that the Summary NN loses more relevant information through its fixed input representation than the TCN does through its learned sequence representation. This difference is most apparent for ψ and ρ , whose posterior estimates depend on temporal features of the volatility process that might not be properly represented by the selected summary statistic (2.47).

4.2.4 Comparison with reduced-data MCMC references

To give the TCN loss a more interpretable scale, the full-sequence TCN was compared with moment-matched MCMC references conditioned on truncated sequences. For

$$\beta \in \{0.80, 0.85, 0.90, 0.95, 1\},$$

MCMC was run conditional on the first

$$\tilde{n}(\beta) = \lceil \beta n \rceil$$

observations of each benchmark sequence, $y_{1:\tilde{n}(\beta)}$. This version of MCMC is denoted by $\text{MCMC}(\beta)$. For each prior, the reported fraction was selected according to

$$\beta^* = \arg \min_{\beta} \left| L_K^{\text{MCMC}(\beta)} - L_K^{\text{TCN}} \right|.$$

Table 4.5: Raw marginal Gaussian losses for the full-sequence TCN and the closest MCMC reference among the considered truncated sequence lengths. Results are based on $K = 5\,000$ benchmark pairs and $M = 20\,000$ retained MCMC draws per pair.

Method	$L_{K,\mu}$	$L_{K,\psi}$	$L_{K,\rho}$	L_K
MCMC (default, $\beta = 0.85$)	-1.0447	-0.0523	-0.6456	-0.5809
TCN (default, full sequence)	-1.0705	-0.0457	-0.6184	-0.5782
MCMC (finance, $\beta = 0.90$)	-0.4877	-0.1188	-0.9604	-0.5223
TCN (finance, full sequence)	-0.5132	-0.1356	-0.9256	-0.5248

Under the default prior, the closest match is obtained from MCMC conditioned on 85% of the sequence, for which

$$L_K^{\text{TCN}} - L_K^{\text{MCMC}(0.85)} = 0.0027.$$

Under the finance prior, the closest match is obtained at 90%, with

$$L_K^{\text{TCN}} - L_K^{\text{MCMC}(0.90)} = -0.0025.$$

Both remaining differences are small relative to the corresponding raw losses.

Under the Gaussian loss, the full-sequence TCN therefore performs similarly to MCMC conditioned on approximately 85% of the observations under the default prior and 90% under the finance prior. This comparison gives a qualitative indication of the degradation introduced by the neural approximation by relating it to the effect of withholding observations from the MCMC reference.

The matched fractions should not be interpreted as showing that the TCN literally discards 15% or 10% of the information in the sequence. They depend on the chosen loss, the discrete grid of candidate fractions and estimates obtained from finite benchmark and MCMC samples. Nevertheless, the larger matched fraction under the finance prior suggests that less information relevant to the Gaussian loss is lost through the neural approximation under this prior than under the default prior.

4.2.5 Empirical coverage

Figure 4.2 compares the empirical coverage defined in (3.14) with the nominal coverage level for each transformed parameter. The dashed diagonal represents exact agreement between nominal and empirical coverage.

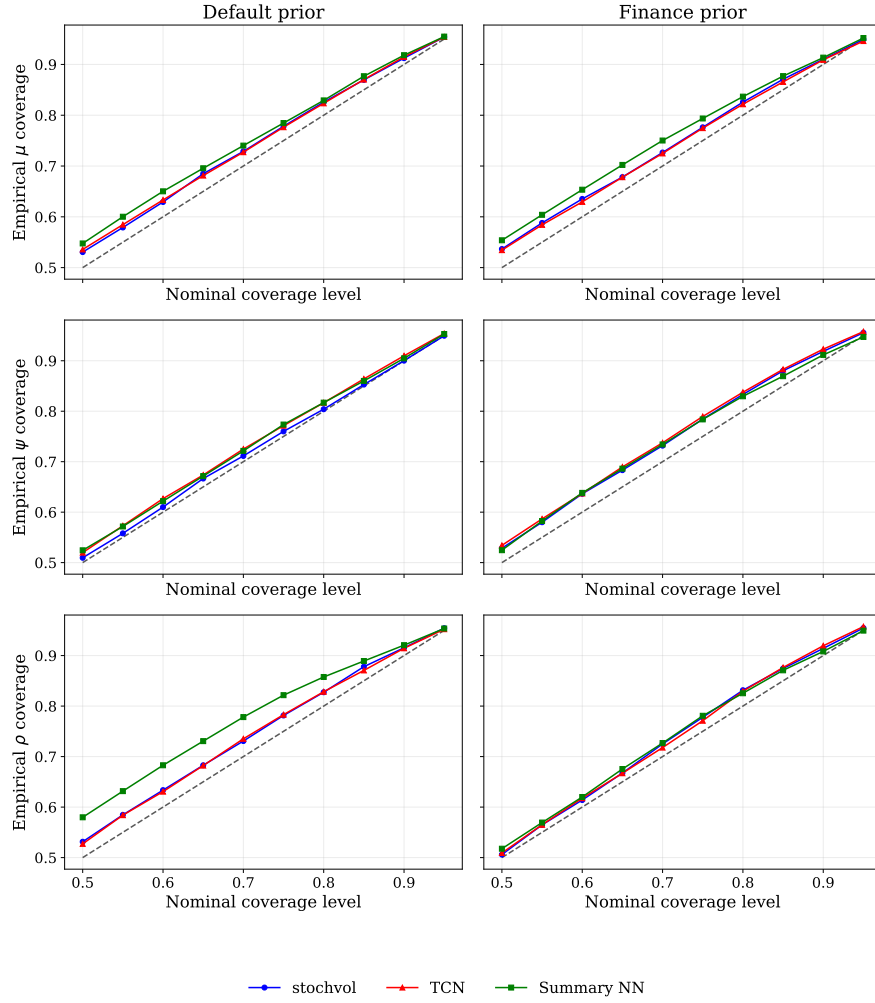


Figure 4.2: Empirical coverage of the central Gaussian credible intervals, estimated from $K = 5000$ benchmark pairs. Rows correspond to μ , ψ , and ρ , while columns correspond to the default and finance priors. The dashed diagonal represents exact nominal coverage. The stochvol curves use Gaussian marginals whose means and variances are estimated from the retained stochvol MCMC draws.

Most curves lie above the diagonal, particularly at the lower and intermediate nominal coverage levels. The Gaussian intervals are therefore predominantly conservative, as they contain the generating transformed parameter more often than prescribed by their nominal levels. The deviations generally decrease toward the highest nominal levels, where the empirical coverages approach the diagonal.

The moment-matched MCMC reference also exhibits overcoverage. The conservative behavior is therefore not caused solely by the neural approximation. The stochvol curves are based on Gaussian intervals constructed from the MCMC posterior means and variances rather than intervals calculated from empirical

MCMC quantiles. The overcoverage of these curves is consequently consistent with the moment-matched Gaussian approximation contributing to the deviation from nominal coverage.

The TCN coverage curves remain close to the moment-matched MCMC reference in all six panels. The Summary NN is also close to the reference for ψ under both priors and for ρ under the finance prior, with the ordering of the methods changing across nominal levels. For μ , the Summary NN generally has somewhat higher coverage than the TCN and MCMC reference under both priors. The largest separation occurs for ρ under the default prior, where the Summary NN lies above the other two curves across all nominal levels.

The marked overcoverage for ρ under the default prior is consistent with the variance results in Table 4.3. The Summary NN has a variance ratio close to 2 relative to MCMC, and therefore produces substantially wider Gaussian intervals. In this case, the increase in interval width appears to outweigh the additional error in the posterior mean, leading to higher empirical coverage.

Under the finance prior, the Summary NN variance ratio for ρ is also close to 2, but its coverage remains close to that of the other methods. Its larger posterior-mean RMSE is consistent with offsetting much of the effect of the increased interval width. This comparison shows that similar empirical coverage does not imply agreement in posterior means or variances, since differences in interval centers and widths can counteract one another.

Overall, the TCN reproduces the coverage of the moment-matched MCMC reference closely. The Summary NN also follows the reference reasonably closely in most panels, but shows marked additional overcoverage for ρ under the default prior. Empirical coverage alone is nevertheless insufficient for assessing the posterior approximation because it does not distinguish between errors in interval location and width. The following section therefore examines the resulting credible intervals directly.

4.3 Marginal credible intervals across parameter sweeps

To illustrate how the posterior estimates vary across selected regions of the parameter space, three one-dimensional parameter sweeps were performed. In each sweep, one SV parameter was assigned ten equally spaced values between the corresponding endpoints in Table 4.6, including both endpoints. The remaining two parameters were held at the baseline values reported in the same table, which describe a persistent volatility process in a plausible financial regime. For each resulting parameter configuration, one sequence of length $n = 253$ was simulated and evaluated by every posterior estimator. The notation used to formulate the credible intervals follows Section 3.4.2.

For neural estimator $A \in \{\text{TCN}, \text{Summary NN}\}$, the 95% marginal credible interval for parameter j on its original scale is

$$\left[G_j^{-1} \left(\widehat{m}_{A,j} + \sqrt{\widehat{v}_{A,j}} \Phi^{-1}(0.025) \right), G_j^{-1} \left(\widehat{m}_{A,j} + \sqrt{\widehat{v}_{A,j}} \Phi^{-1}(0.975) \right) \right], \quad (4.2)$$

Table 4.6: Baseline values and ranges used in the credible-interval sweeps. Each parameter was varied over ten equally spaced values, including the endpoints, while the remaining parameters were held at their baseline values.

Parameter	Baseline value	First sweep value	Last sweep value
μ	−9	−12	−6
ϕ	0.95	0.905	0.995
σ	0.25	0.05	0.45

where Φ is the standard Gaussian CDF and G_j^{-1} is the corresponding component of G^{-1} , defined in (2.36). Each component is strictly increasing, so transforming the endpoints preserves the equal-tailed interpretation. The NPE markers show

$$G_j^{-1}(\widehat{m}_{A,j}),$$

which is the median of the induced marginal posterior on the original scale.

For the `stochvol` reference, the retained MCMC draws were expressed on the original parameter scale (μ, ϕ, σ) . The interval endpoints are their empirical 2.5% and 97.5% quantiles, while the markers show the empirical posterior medians.

In figure 4.3 we see that the posterior medians for μ broadly follow the diagonal under both priors, and the TCN remains close to the `stochvol` reference across most of the sweep. Under the finance prior, shrinkage toward the prior center $\mu = -9$ is most visible at the upper end of the sweep, particularly when the generating value is $\mu = -6$. The intervals also generally widen as the generating value moves away from the region around -9 , with the most pronounced widening occurring at $\mu = -6$.

The estimates of ϕ vary considerably between adjacent sweep points, particularly under the default prior. Since each point is based on a different simulated sequence, this irregularity reflects both the change in the generating parameter and variation between the realized sequences. For the displayed sequences, the wide intervals at several lower and intermediate values of ϕ show substantial posterior uncertainty about persistence. The intervals under the finance prior are generally narrower across this high-persistence sweep, consistent with the greater concentration of that prior in this region. The TCN again follows the MCMC reference more closely than the Summary NN, whose intervals are especially wide at several lower and intermediate values of ϕ .

The intervals for ϕ narrow on the original scale as ϕ approaches one. For the neural estimators, this is partly a direct consequence of the inverse transformation

$$\phi = \tanh\left(\frac{\psi}{2}\right),$$

which compresses changes in ψ near the upper boundary of the stationary region. The narrowing of the NPE intervals near $\phi = 1$ should therefore not be attributed entirely to greater information in the observed sequence.

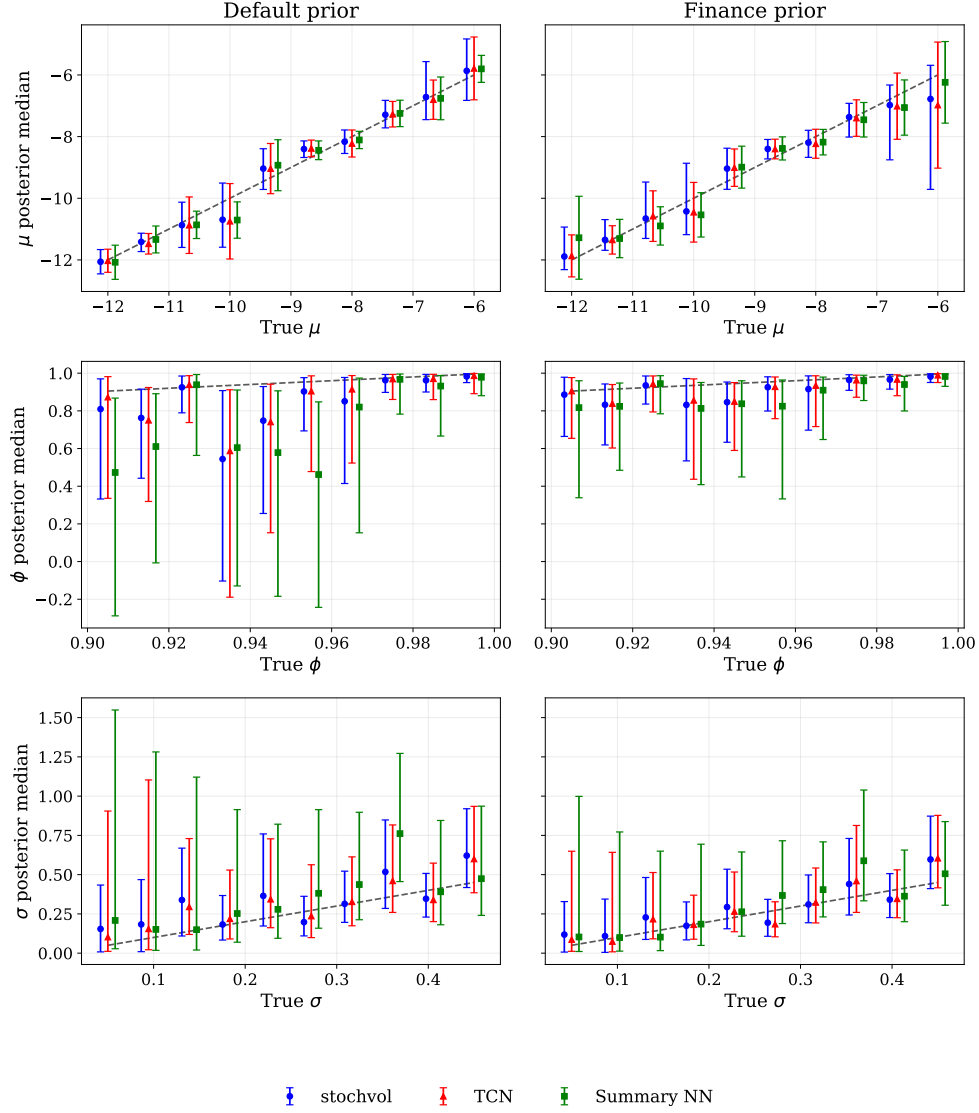


Figure 4.3: Posterior medians and 95% marginal credible intervals across the parameter sweeps. Rows correspond to the parameter being varied, and columns correspond to the default and finance priors. Each point is based on one simulated sequence of length $n = 253$, with the remaining parameters fixed at their baseline values. The dashed line represents equality between the posterior median and the generating parameter. The stochvol intervals are empirical quantile intervals calculated from $M = 20\,000$ retained MCMC draws, whereas the NPE intervals are constructed in the transformed space and mapped back using (4.2).

The σ panels show the clearest differences in posterior uncertainty. The TCN medians and intervals generally follow the MCMC reference, although the TCN intervals are noticeably wider at the two smallest values of σ under both priors and are often moderately wider elsewhere. The Summary NN intervals are considerably wider and vary more between sweep points, particularly under the default prior. This is consistent with the geometric mean variance ratios in Table 4.3, for which the largest variance inflation occurs in the Summary NN approximation of ρ .

For the neural estimators, the asymmetry of the σ intervals follows from the transformation

$$\sigma = \exp(\rho).$$

A symmetric Gaussian interval for ρ becomes a multiplicative interval for σ , with a longer upper side when displayed on the original scale. The `stochvol` intervals are instead empirical quantile intervals, so their asymmetry reflects the corresponding MCMC posterior draws.

In the displayed sweeps, the finance-prior panels contain fewer extreme upper endpoints for σ than the default-prior panels, even though the marginal prior for σ is unchanged. This difference can arise because the posterior uncertainty in σ also depends on the likelihood and on the prior information about μ and ϕ .

Overall, the TCN more closely follows the MCMC posterior medians and interval widths across the displayed sweeps, whereas the Summary NN tends to return wider intervals and more variable estimates. Under the finance prior, the ϕ intervals are generally narrower across the selected high-persistence region, while the μ estimates show clearer shrinkage toward -9 at the upper end of the sweep. Since each point is based on one simulated sequence, these plots illustrate the behavior of the estimators for the displayed realizations rather than their average behavior at each parameter value.

4.4 Runtime of posterior-moment estimation

Table 4.7 reports the time required to estimate the first two moments of the transformed marginal posteriors for one sequence $y_{1:253}^{(i)}$. The results are calculated from the $K = 5\,000$ benchmark pairs. All runtimes were measured on a laptop equipped with a 13th-generation Intel Core i7-13700HX CPU, 32 GB of RAM, and an NVIDIA GeForce RTX 4070 Max-Q GPU with 8 GB of VRAM.

The MCMC reference requires approximately 3.5 seconds per sequence. This includes $B = 500$ burn-in iterations, the generation of $M = 20\,000$ retained posterior draws, and the calculation of their marginal means and variances. The corresponding TCN runtime is 0.0028 seconds, making the TCN approximately 1 250–1 260 times faster. The Summary NN requires approximately 0.08 seconds and is about 44–45 times faster than MCMC.

These factors compare the configurations used in the benchmark and partly reflect the deliberately long MCMC chains. The retained chain length was chosen

Table 4.7: Mean and standard deviation of the runtime required to estimate the first two moments of the transformed marginal posteriors for one sequence of length $n = 253$, calculated from $K = 5\,000$ benchmark pairs.

Method	Mean runtime (s)	Runtime SD (s)
MCMC (default)	3.5289	0.6195
TCN (default)	0.0028	0.0000
Summary NN (default)	0.0808	0.0528
MCMC (finance)	3.5038	0.5990
TCN (finance)	0.0028	0.0000
Summary NN (finance)	0.0776	0.0493

to provide adequate effective sample sizes even for parameter chains with irregularly low ESS, rather than to minimize runtime for a typical sequence. Averaged over the $B + M = 20\,500$ iterations, the measured MCMC runtime corresponds to approximately 1.7×10^{-4} seconds per iteration. A single iteration is inexpensive, but many iterations are required to estimate the posterior moments accurately.

Using this average as a linear approximation, matching the Summary NN runtime would permit only approximately 454–469 MCMC iterations in total. This is fewer than the $B = 500$ burn-in iterations alone, so there is no nonnegative number of retained draws M for which MCMC with the same burn-in has a comparable runtime. Matching the TCN runtime would permit only about a total of 16 iterations. Equivalently, with $M = 0$, the extrapolated runtime of the 500 burn-in iterations is approximately 0.085–0.086 seconds, which is already slightly longer than the Summary NN runtime and about 31 times the TCN runtime.

The Summary NN is approximately 28–29 times slower than the TCN. Its network evaluation is inexpensive, but constructing its input requires fitting an ARMA(1, 1) model using an iterative routine limited to 50 iterations. Variation in this fit across sequences also accounts for the comparatively large runtime standard deviation. In contrast, the TCN input requires only the fixed logsq transformation and analytical standardization, followed by a fixed collection of tensor operations. Its runtime standard deviation consequently rounds to zero at the precision shown in Table 4.7. The similar runtimes under the two priors indicate that computational cost is governed primarily by the inference procedure rather than the prior configuration.

4.5 Discussion of results

The main empirical finding is that the TCN gives the closest neural approximation to the MCMC posterior moments under both priors while requiring the shortest evaluation time. The Summary NN shows larger differences, particularly for the posterior uncertainty of ψ and ρ , whereas MCMC remains more flexible when the

prior or sequence length changes. The following discussion relates these findings to the Gaussian loss, the information available to the two neural encoders and the different costs of adapting each method.

4.5.1 Separating moment accuracy from posterior shape

Propositions 1 and 2 provide a direct interpretation of the benchmark results. Within the selected product-Gaussian family, the population objective targets the means and variances of the transformed marginal posteriors. The close agreement between the TCN and the MCMC moment estimates therefore suggests that the TCN approximates these conditional moment functions well across the benchmark distribution.

The skewness visible for ψ and ρ in Figure 3.3 does not conflict with this conclusion. It describes posterior shape that cannot be retained by a moment-matched Gaussian. The overcoverage of the moment-matched MCMC intervals supports the same distinction: some of the interval behavior is caused by the marginal Gaussian approximation rather than by the neural encoder.

For the sequence examined in the Bernstein–von Mises diagnostic, the transformed marginal posteriors are closer to Gaussian at $n = 1012$ than at $n = 253$. If this behavior extends beyond the displayed sequence, longer sequences would make the selected posterior family more appropriate.

4.5.2 Sequence representation and model extension

The results point toward the input representation as an important limitation of the Summary NN. Its largest differences from MCMC occur for ψ and ρ , particularly in the estimated posterior variances. The relative accuracy for μ is consistent with the relation

$$\log(y_t^2) = h_t + \log(\varepsilon_t^2).$$

Since h_t has stationary mean μ , the level of the log-squared observations changes directly with μ . The mean and quantiles included in S therefore provide direct summaries of features that are informative about this parameter.

The effects of ϕ and σ must also be learned from the temporal dependence and dispersion of the sequence. Although S includes autocovariance ratios, dispersion measures and fitted ARMA(1, 1) features, it reduces these properties to a fixed collection of values. Temporal patterns not represented by this collection are removed before the sequence reaches the network. The TCN instead receives the complete transformed sequence and can learn combinations of temporal features across its full receptive field.

This distinction also changes the population target of the two estimators. A Summary NN receiving only $S(Y_{1:n})$ targets posterior moments conditional on the summary, whereas the TCN can target posterior moments conditional on the complete transformed sequence. Unless S is sufficient for the transformed parameters, these conditional moments need not agree. Since $S(Y_{1:n})$ is a function of $Y_{1:n}$, the

law of total variance also implies that conditioning on the summary cannot reduce the expected posterior variance relative to conditioning on the complete sequence. This provides a possible explanation for the larger Summary NN variance estimates for ψ and ρ , although the reported geometric mean variance ratios do not by themselves establish that this is their cause.

The validation-loss histories give no clear indication that the Summary NN performed worse simply because its training was terminated too early. Both networks were trained until the early-stopping rule recorded a sustained absence of sufficient improvement. The TCN nevertheless contains approximately 7.56 times as many trainable parameters, so the experiments do not separate the effect of the input representation from the effect of network capacity or the optimizer’s ability to fit each architecture. The results show that the complete TCN implementation provides a clear practical advantage, but they do not isolate how much of that advantage comes from each architectural difference.

The choice of representation also matters when extending inference to other SV models. Neural posterior estimation requires the ability to simulate from the model rather than an algorithm for drawing from its posterior. Once a new SV model can be simulated, the existing TCN template can often be retained by replacing the simulation routine and adding prediction heads and loss terms for any additional transformed parameters, followed by retraining. The Summary NN requires the same changes together with a new or revised summary map. Constructing suitable summaries may require detailed knowledge of how the new parameters affect the observed sequence.

Extending MCMC presents a different problem. If the new model is not supported by `stochvol`, a sampler following Algorithm 1 requires new posterior update rules. The benchmark sampler benefits from a model-specific update that samples the complete log-volatility path in one block. If this update is unavailable for the extended model, a simpler implementation could update the latent volatilities individually. Because neighboring latent states are strongly related, such pointwise updates can produce slower mixing and lower effective sample sizes. Deriving an effective block update for the new model may require considerably more work. Among the implementations considered here, the TCN therefore offers the simplest general template for extending inference to a new simulatable SV model.

4.5.3 Speed, flexibility and repeated inference

The runtime comparison is especially strong because the 0.0028-second TCN runtime permits only about 16 MCMC iterations under the linear approximation used in Section 4.4. This is far below the 500 discarded iterations in the benchmark configuration. Within the time required for one TCN evaluation, the benchmark MCMC procedure cannot complete its chosen burn-in, let alone retain enough draws to estimate the posterior moments. Once the weights and input are fixed, the TCN estimate also contains no chain-sampling variation.

The Summary NN evaluation time is governed mainly by the iterative ARMA(1, 1) fit used to construct $S(y_{1:n})$. The larger TCN is therefore both more accurate and faster for the implementations considered here. Its learned representation avoids both the information restriction of the fixed summaries and the most expensive part of their construction.

The TCN is most attractive when many sequences of similar length are to be analyzed under the same model and prior, particularly when the sequences can be evaluated in batches. MCMC is more adaptable when the prior or sequence length changes because the sampler can be rerun directly in the new setting. The Summary NN also has an advantage over the current TCN for varying sequence lengths as $S(y_{1:n})$ has a fixed dimension for every n . Its trained weights would still need to be checked, and possibly retrained, at new lengths, but the network architecture itself does not depend on a receptive-field condition. The present TCN was designed for $n = 253$, and its full-sequence receptive-field argument applies only up to length 289.

4.5.4 Overall comparison of the inference methods

Table 4.8: Main practical trade-offs among the posterior estimators considered in this thesis.

Property	MCMC	TCN	Summary NN
Agreement with MCMC moments	Posterior reference	Closest method under both priors	Larger differences, mainly for the uncertainty of ψ and ρ
Mean evaluation time	Approximately 3.5 s	0.0028 s	Approximately 0.08 s
Training before use	None	Largest training cost	Smaller network, but costly summary construction
New prior	Rerun the sampler	Retrain the network	Retrain the network
Different sequence length	Directly supported	Length-specific training and receptive field	More adaptable because $S(y_{1:n})$ has fixed dimension
New SV model	Construct new posterior updates. An efficient block sampler may be difficult to retain	Replace the simulator, extend the heads and loss, and retrain	Same points as TCN, but also requires a new summary map S
Stored trained model	No trained weights	127 286 parameters	16 838 parameters
Posterior information	Joint draws retaining dependence and non-Gaussian shape	Separate marginals	Separate Gaussian marginals

Table 4.8 summarizes the main strengths and weaknesses of the three implementations.

The TCN has the clearest advantage when many similar-length sequences are analyzed under a stable model and prior and the marginal posterior moments are the quantities of interest. It combines the closest neural agreement with MCMC and the shortest evaluation time, at the cost of the largest trained network and the most expensive training procedure. MCMC is more flexible when priors or sequence lengths change and retains joint posterior information. The Summary NN occupies an intermediate position: it stores fewer weights and handles different input lengths more naturally, but its fixed representation appears to remove information relevant to ψ and ρ , while its ARMA-based features make it slower than the TCN.

4.5.5 Extensions and further evaluation

The most important extension is to estimate the joint posterior. A multivariate Gaussian output could represent posterior correlations, while a more flexible conditional density would also be needed to reproduce the skewness and tail behavior seen in figure 3.3. This would allow joint credible regions and posterior inference for quantities depending on several SV parameters.

A hybrid neural estimator could combine the strengths of the two encoders. The transformed sequence could be processed by a TCN encoder while $S(y_{1:n})$ is processed by a smaller MLP encoder. Their feature vectors could then be concatenated and supplied to the parameter-specific heads. This would combine learned temporal features with the model-based information in the summaries. It would also provide a direct test of whether S contributes information that the TCN does not readily recover.

The TCN output could additionally be used to initialize an MCMC chain or to select proposal locations and scales. This could shorten the initial transient or improve the proposals, while the subsequent MCMC updates would retain the desired posterior as their stationary distribution. A joint neural output would make this approach more useful by also providing estimated parameter dependence.

The TCN architecture was selected through targeted experiments rather than an exhaustive search, and ReLU was the only activation function examined. Future experiments could compare smaller and larger networks, other activation functions and pooling rules, and several sequence lengths with corresponding receptive fields. This would show whether a simpler TCN can retain the current accuracy or whether additional capacity can reduce the remaining differences from MCMC.

Finally, the runtime comparison could be extended from equal computation time to equal estimation error. For each sequence, a long independent MCMC calculation could provide a high-accuracy reference for a posterior second moment or variance. Shorter MCMC runs with several retained lengths M could then be compared with the NPE estimate of the same quantity. Selecting the value of M for

which the two estimates have similar RMSE would give a more direct comparison of the runtime needed to attain the same moment accuracy.

Chapter 5

Time Irreversible SV models

The preceding chapter showed that the TCN estimator can estimate the first two moments of the transformed marginal posteriors for the standard SV model with losses close to those obtained from MCMC, while taking far less time to evaluate. The TCN is also convenient to extend because NPE requires simulated parameter–data pairs rather than a posterior sampling algorithm. For a new simulatable SV model, the existing template can largely be retained by modifying the simulator, adding prediction heads and loss terms, and adjusting the temporal encoder where necessary. The Summary NN would additionally require a new or revised summary map, whereas MCMC would require model-specific posterior update rules. We therefore use the TCN to investigate a more general SV model.

The extension considered here is motivated by sudden changes in financial volatility. The Gaussian log-volatility innovations of the standard SV model are symmetric and light-tailed, which limits the probability of unusually large changes. Previous work has introduced greater flexibility in several ways. Barndorff-Nielsen and Shephard [34] construct continuous-time models based on positive non-Gaussian Ornstein–Uhlenbeck volatility processes. In a discrete-time setting, Chib et al. [35] use heavy-tailed Student’s t observation innovations and a Bernoulli jump component in the observation equation. Eraker et al. [36] instead consider continuous-time models with jumps in both returns and volatility and find evidence for both in equity-index data. These models allow rare changes either through the volatility process or through the observation equation.

Our approach retains the discrete-time autoregressive log-volatility equation and replaces its Gaussian innovation with a standardized generalized hyperbolic skew Student’s t distribution. Its positive skewness and heavy tail allow unusually large positive innovations, and hence sudden increases in volatility, without a separate jump component. Nonzero innovation skewness also makes the latent log-volatility process time irreversible, representing the asymmetry between a sudden increase and its subsequent reversion. The resulting GHST-SV model introduces two parameters governing innovation skewness and tail weight while preserving the interpretations of μ , ϕ , and σ .

The first section defines time irreversibility, compares candidate innovation

distributions, and specifies the GHST-SV model and its priors. The second describes the TCN implementation, the MCMC and transformed-prior references, and simulation-based calibration. The final section presents the results through the validation-loss history, loss comparisons, credible-interval sweeps and simulation-based calibration.

5.1 The GHST-SV model

This section introduces the GHST-SV model, an extension of the standard SV model that allows for time-irreversible processes. This is achieved by replacing the Gaussian log-volatility innovations with a skewed distribution, since nonzero innovation skewness induces time irreversibility in the latent log-volatility process. We first define time irreversibility and compare candidate innovation distributions before motivating the choice of the generalized hyperbolic skew Student's t distribution. We then define the complete GHST-SV model and specify the priors and unconstrained parameterization for the new parameters governing innovation skewness and tail weight.

5.1.1 Time irreversibility

Let $\{X_t\}$ be a strictly stationary process. It is *time reversible* if

$$(X_t, X_{t+1}, \dots, X_{t+k}) \stackrel{d}{=} (X_{t+k}, X_{t+k-1}, \dots, X_t)$$

for every $t \in \mathbb{Z}$ and $k \geq 1$. Otherwise, the process is *time irreversible*. Time irreversibility therefore means that the forward and reversed orderings differ in at least one finite-dimensional distribution.

Under its stationary distribution, the log-volatility process in the Gaussian SV model is a Gaussian AR(1) process and is time reversible. This restricts the model from representing directional asymmetries in volatility dynamics. Such flexibility is relevant for financial time series, where volatility may increase through sudden positive shocks before reverting more gradually. Allowing the log-volatility innovations to be positively skewed provides a direct way of representing this behaviour.

To see the connection with time irreversibility, define $X_t = H_t - \mu$, so that the state equation becomes

$$X_t = \phi X_{t-1} + \sigma \eta_t.$$

Suppose that $|\phi| < 1$, $\phi \neq 0$, and that the i.i.d. innovations have mean zero, a finite third moment and nonzero skewness. One can show that

$$\mathbb{E}[X_{t-1}^2 X_t] - \mathbb{E}[X_t^2 X_{t-1}] = \frac{\phi(1-\phi)\sigma^3}{1-\phi^3} \mathbb{E}[\eta_t^3] \neq 0.$$

Time reversibility would require these two mixed moments to be equal because $(X_{t-1}, X_t) \stackrel{d}{=} (X_t, X_{t-1})$; see Bahr [37, Proposition 2] for an in-depth derivation of

the result. Consequently, nonzero innovation skewness is sufficient for the latent log-volatility process to be time irreversible, although it is not necessary.

5.1.2 Selecting the log-volatility innovation distribution

The preceding result motivates replacing the Gaussian log-volatility innovation by a positively skewed distribution. A suitable replacement should allow skewness and tail weight to be varied while retaining the Gaussian distribution as a limiting or special case. It should also admit a mean-zero, unit-variance parameterization, so that μ , ϕ , and σ retain their interpretations from the standard SV model. Finally, direct simulation should be inexpensive because training the NPE requires repeated simulation from the model. We consider the generalized hyperbolic skew Student's t (GHST) distribution and the Gaussian-generated sinh-arcsinh (SHASH) distribution.

The GHST distribution. Following Aas and Haff [38], let

$$X \sim \text{GHST}(\nu, m, \delta, \beta), \quad \nu > 0, \quad \delta > 0, \quad m, \beta \in \mathbb{R},$$

and define

$$q(x) = \sqrt{\delta^2 + (x - m)^2}.$$

Its density can be written as

$$f_{\text{GHST}}(x) = \begin{cases} \frac{2^{(1-\nu)/2} \delta^\nu |\beta|^{(\nu+1)/2} K_{(\nu+1)/2}(|\beta| q(x))}{\sqrt{\pi} \Gamma(\nu/2) q(x)^{(\nu+1)/2}} \exp\{\beta(x - m)\}, & \beta \neq 0, \\ \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi} \delta \Gamma(\nu/2)} \left[1 + \frac{(x - m)^2}{\delta^2} \right]^{-(\nu+1)/2}, & \beta = 0, \end{cases} \quad (5.1)$$

where K_λ is the modified Bessel function of the second kind. The second case is a Student's t density with location m , scale $\delta/\sqrt{\nu}$, and ν degrees of freedom.

The Gaussian variance-mean mixture in Aas and Haff [38] also gives a direct sampling procedure. In a form convenient for the reparameterization below, draw independently

$$Z \sim \mathcal{N}(0, 1), \quad U \sim \text{InvGamma}\left(\frac{\nu}{2}, \frac{1}{2}\right),$$

where the inverse-gamma distribution is parameterized by shape and scale, so its density is proportional to $u^{-\nu/2-1} \exp\{-1/(2u)\}$, and set

$$X = m + \beta \delta^2 U + \delta \sqrt{U} Z. \quad (5.2)$$

Then $X \sim \text{GHST}(\nu, m, \delta, \beta)$. Its mean and variance are

$$\mathbb{E}[X] = m + \frac{\beta \delta^2}{\nu - 2}, \quad \nu > 2, \quad (5.3)$$

$$\text{Var}(X) = \frac{\delta^2}{\nu - 2} + \frac{2\beta^2 \delta^4}{(\nu - 2)^2(\nu - 4)}, \quad \nu > 4. \quad (5.4)$$

For use as a log-volatility innovation, let $\nu > 4$, impose $\mathbb{E}[X] = 0$ and $\text{Var}(X) = 1$, and let $r \in [0, 1)$ denote the fraction of the variance contributed by the random mean component in (5.2):

$$r = \frac{\beta^2 \delta^4 \text{Var}(U)}{\text{Var}(X)}.$$

Solving for the original GHST parameters and choosing nonnegative β gives

$$\begin{aligned} \delta &= \sqrt{(\nu - 2)(1 - r)}, \\ \beta &= \frac{\sqrt{r(\nu - 4)/2}}{1 - r}, \\ m &= -\sqrt{\frac{r(\nu - 4)}{2}}. \end{aligned} \tag{5.5}$$

Substitution into (5.2) yields the standardized innovation

$$\eta(Z, U; r, \nu) = \sqrt{(\nu - 2)(1 - r)} U Z + \sqrt{\frac{r(\nu - 4)}{2}} ((\nu - 2)U - 1). \tag{5.6}$$

This gives a GHST innovation parameterized directly by (r, ν) , with

$$\mathbb{E}[\eta(Z, U; r, \nu)] = 0, \quad \text{Var}(\eta(Z, U; r, \nu)) = 1.$$

For $\nu > 6$, $r > 0$ gives positive skewness, whereas $r = 0$ gives a symmetric standardized Student's t innovation. Hence, r controls the contribution of the component producing positive skewness, while ν controls the polynomial tail weight. The Gaussian limit is considered in Section 5.1.4.

The SHASH distribution. The SHASH family of Jones and Pewsey [39] is generated by an invertible transformation of a Gaussian random variable and was used for the log-volatility innovations in Bahr [37]. We use the sign convention in which positive ϵ produces positive skewness. If $Z \sim \mathcal{N}(0, 1)$, then

$$X_{\epsilon, \tau} = \sinh\left(\frac{\text{arsinh}(Z) + \epsilon}{\tau}\right), \quad \epsilon \in \mathbb{R}, \quad \tau > 0,$$

has density

$$g_{\epsilon, \tau}(x) = \frac{\tau \cosh(\tau \text{arsinh}(x) - \epsilon)}{\sqrt{2\pi(1 + x^2)}} \exp\left\{-\frac{1}{2} \sinh^2(\tau \text{arsinh}(x) - \epsilon)\right\}.$$

To state its first two moments, define

$$P_q = \frac{e^{1/4}}{\sqrt{8\pi}} \left[K_{(q+1)/2}\left(\frac{1}{4}\right) + K_{(q-1)/2}\left(\frac{1}{4}\right) \right].$$

Then

$$\mathbb{E}[X_{\epsilon,\tau}] = \sinh\left(\frac{\epsilon}{\tau}\right) P_{1/\tau}, \quad (5.7)$$

$$\text{Var}(X_{\epsilon,\tau}) = \frac{1}{2} \left[\cosh\left(\frac{2\epsilon}{\tau}\right) P_{2/\tau} - 1 \right] - \mathbb{E}[X_{\epsilon,\tau}]^2. \quad (5.8)$$

The mean-zero, unit-variance SHASH variable can therefore be generated as

$$\eta_{\text{SHASH}}(Z; \epsilon, \tau) = \frac{\sinh([\text{arsinh}(Z) + \epsilon]/\tau) - \mathbb{E}[X_{\epsilon,\tau}]}{\sqrt{\text{Var}(X_{\epsilon,\tau})}}. \quad (5.9)$$

Thus, SHASH can also be standardized for every (ϵ, τ) . Its simulation requires only one Gaussian draw and an elementary transformation, but its standardization constants involve modified Bessel functions. The Gaussian distribution is obtained exactly when $(\epsilon, \tau) = (0, 1)$. Positive ϵ gives positive skewness, and $\tau < 1$ gives tails heavier than Gaussian tails.

Figure 5.1 illustrates that both standardized families recover the Gaussian density and can produce positive skewness with increasingly heavy right tails.

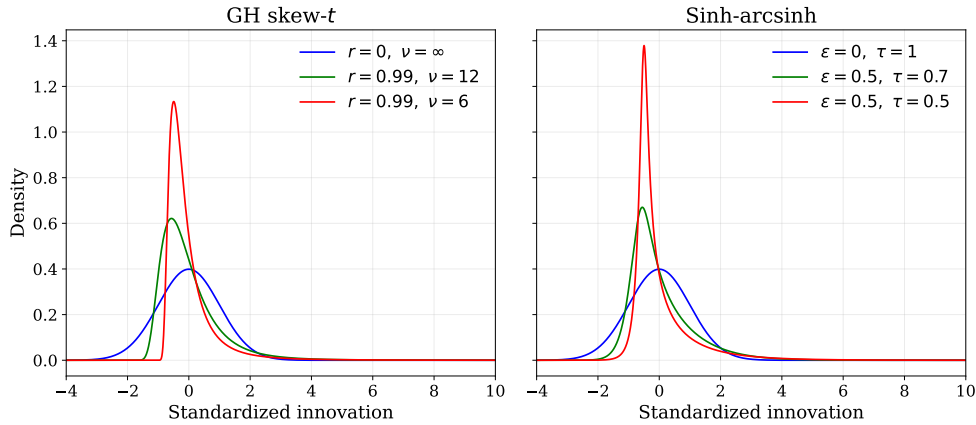


Figure 5.1: Standardized GHST (left) and SHASH (right) densities. The blue curves show the Gaussian case, while the green and red curves show positive skewness with increasing tail weight.

Comparison and selection. The main distinction is the tail behavior. For $r > 0$, Aas and Haff [38] show that the density of the standardized GHST innovation satisfies

$$f_{\text{GHST}}(x) \sim c_+ x^{-\nu/2-1}, \quad x \rightarrow +\infty, \quad (5.10)$$

$$f_{\text{GHST}}(x) \sim c_- |x|^{-\nu/2-1} \exp\left\{-\frac{\sqrt{2r(\nu-4)}}{1-r} |x|\right\}, \quad x \rightarrow -\infty, \quad (5.11)$$

for constants $c_+, c_- > 0$. Its positive tail is polynomial, whereas its negative tail is exponentially damped. By contrast, the tail expansion in Jones and Pewsey [39] gives, after standardization,

$$f_{\text{SHASH}}(x; \epsilon, \tau) \sim C_{\pm} |x|^{\tau-1} \exp\{-c_{\pm} |x|^{2\tau}\}, \quad x \rightarrow \pm\infty, \quad (5.12)$$

where $C_{\pm}, c_{\pm} > 0$. The two SHASH tails can have different scales, but both remain lighter than every polynomial tail and all polynomial moments exist.

This distinction is useful for the intended volatility behavior. Relative to the value predicted by the autoregressive component, an innovation of size $x > 0$ multiplies the conditional standard deviation of Y_t by $\exp(\sigma x/2)$. The GHST distribution can therefore assign appreciable probability to unusually large increases in volatility while making equally large one-step decreases less frequent. This is consistent with the empirical motivation for allowing abrupt changes in volatility; for example, Eraker et al. [36] find evidence for jumps in volatility in equity-index data.

There is also direct, although not fully comparable, financial evidence for the GHST distribution. Aas and Haff [38] fit it to daily log returns from equity, bond, exchange-rate, and interest-rate series and report a better overall fit than the normal inverse Gaussian distribution and Azzalini's skew Student's t distribution. However, their study concerns observed returns rather than latent log-volatility innovations.

SHASH is slightly simpler to simulate, but GHST has elementary expressions for its mean and variance and the polynomial-positive, exponentially-damped-negative tail structure sought here. We therefore use the standardized GHST distribution for the log-volatility innovations.

5.1.3 GHST-SV model definition

Figure 5.2 illustrates the effect of replacing the Gaussian log-volatility innovations of the standard SV model by positively skewed and heavy-tailed GHST innovations. With Gaussian innovations, changes in log volatility are symmetric in distribution, and the simulated path fluctuates around its mean without a preferred direction. The GHST-SV path instead contains occasional sharp increases followed by more gradual mean reversion, giving it a saw-tooth-like appearance. This directional asymmetry is characteristic of time-irreversible behavior. The corresponding return path also contains clusters of larger absolute observations during periods of high log volatility.

Let

$$Z_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1), \quad U_t \stackrel{\text{i.i.d.}}{\sim} \text{InvGamma}\left(\frac{\nu}{2}, \frac{1}{2}\right).$$

The GHST-SV model retains the observation equation of the standard SV model and replaces its Gaussian log-volatility innovations by the standardized GHST in-

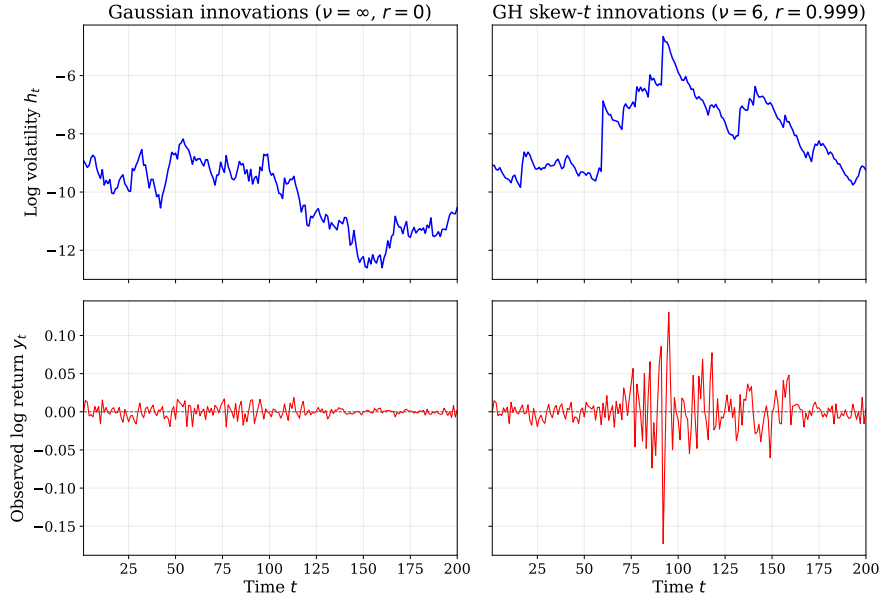


Figure 5.2: Simulated paths from the standard SV model with Gaussian log-volatility innovations (left) and the GHST-SV model with positively skewed and heavy-tailed innovations (right). The upper panels show the latent log volatilities h_t , while the lower panels show the corresponding observed log returns y_t .

novation in (5.6):

$$Y_t = \exp(H_t/2)\varepsilon_t, \quad t = 1, \dots, n, \quad (5.13)$$

$$H_t - \mu = \phi(H_{t-1} - \mu) + \sigma\eta(Z_t, U_t; r, \nu), \quad t = 2, \dots, n, \quad (5.14)$$

where

$$\varepsilon_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

The sequences $\{\varepsilon_t\}$, $\{Z_t\}$, and $\{U_t\}$ are mutually independent, and

$$\mu \in \mathbb{R}, \quad \phi \in (-1, 1), \quad \sigma > 0, \quad r \in [0, 1), \quad \nu > 6.$$

Because the GHST innovation is centered and has unit variance,

$$\mathbb{E}[H_t | H_{t-1}] = \mu + \phi(H_{t-1} - \mu), \quad \text{Var}(H_t | H_{t-1}) = \sigma^2.$$

Hence, μ , ϕ , and σ retain their interpretations from the standard SV model, and the stationary moments remain those in (2.5). Moreover, Bahr [37, Proposition 1] shows that, under strict stationarity, an AR(1) process with i.i.d. non-Gaussian innovations has a non-Gaussian stationary distribution. Denote the resulting stationary density by

$$f_{\text{stat}}(h | \mu, \phi, \sigma, r, \nu).$$

For a stationary finite-sample model, the initial state is assigned

$$H_1 \sim f_{\text{stat}}(\cdot | \mu, \phi, \sigma, r, \nu),$$

and the remaining states and observations follow (5.14) and (5.13). The approximation of f_{stat} is considered later in the model implementation.

The parameters of the extended model are collected in

$$\theta_{1:5} = (\mu, \phi, \sigma, r, \nu). \quad (5.15)$$

5.1.4 Priors and unconstrained parameterization

The five prior components are taken to be mutually independent. For the original SV parameters (μ, ϕ, σ) , we use the default prior configuration in Table 3.1. The two additional innovation parameters are assigned the priors

$$r \sim \text{Uniform}(0, 1), \quad \nu - 6 \sim \text{Exp}(\lambda_\nu), \quad \lambda_\nu = \frac{\log(10)}{24}, \quad (5.16)$$

where λ_ν denotes the exponential rate.

The restriction $\nu > 6$ ensures that the GHST innovation has a finite third moment for every $r \in (0, 1)$, so its standardized skewness is well defined. The shifted-exponential form is motivated by the prior used in `stochvol` for an SV model with Student's t conditional observation innovations. In that setting, Hosszejni and Kastner [7] assign an exponential prior to $\nu - 2$, where the shift ensures $\nu > 2$, so the conditional observation variance is finite. Here, ν instead governs the GHST log-volatility innovations, and the shift by 6 imposes the stronger third-moment requirement.

As shown in Appendix A.4, for every fixed $r \in [0, 1)$,

$$\eta(Z_t, U_t; r, \nu) \xrightarrow{d} \eta_t, \quad \eta_t \sim \mathcal{N}(0, 1), \quad \text{as } \nu \rightarrow \infty.$$

Consequently, the standard SV model is recovered in the limit $\nu \rightarrow \infty$. Since the exponential density decreases with ν , the prior assigns greater density to lower admissible values, corresponding to heavier-tailed innovations, while retaining support for values approaching the Gaussian limit. The rate in (5.16) is chosen so that

$$\mathbb{P}(\nu > 30) = \exp\{-\lambda_\nu(30 - 6)\} = 0.1.$$

Hence, the prior places probability 0.9 on $6 < \nu \leq 30$ and probability 0.1 on the region $\nu > 30$, which we treat as approximately Gaussian. The parameter r controls the allocation of variance between the two components in (5.6). Since no particular allocation is preferred, r is assigned a uniform prior.

Because $r \in (0, 1)$ and $\nu > 6$ almost surely under (5.16), an unconstrained target for the TCN-based NPE is obtained by extending the transformation G in (2.35):

$$\begin{aligned} G(\theta_{1:5}) = \tilde{\theta}_{1:5} &= (\mu, \psi, \rho, \tilde{r}, \tilde{\nu}) \in \mathbb{R}^5, \\ \psi &= 2 \operatorname{arctanh}(\phi), \quad \rho = \log(\sigma), \\ \tilde{r} &= \operatorname{logit}(r) = \log\left(\frac{r}{1-r}\right), \quad \tilde{\nu} = \log(\nu - 6). \end{aligned} \quad (5.17)$$

The NPE is trained on the resulting unconstrained parameter vector $\tilde{\theta}_{1:5}$.

5.2 Model implementation

This section describes the implementation and evaluation procedures used to extend TCN-based neural posterior estimation to the five-parameter GHST-SV model. Only the TCN estimator is extended, since its use of the complete transformed observation sequence avoids the need to construct new summary statistics tailored to r and ν . The main changes therefore concern the simulator, the dimension of the target, the number of prediction heads and the receptive field of the temporal encoder.

The sequence length is increased from 253 to

$$n = 10 \cdot 253 = 2530.$$

Since the GHST innovations are standardized, r and ν do not affect the conditional mean or variance of H_t . Information about these parameters must instead be obtained from asymmetry and tail behavior in the observed sequence. The longer sequence is used to provide more information about these distributional features.

The mini-batch size, number of mini-batches per epoch and complete early-stopping configuration are retained from the TCN column of Table 3.4. A newly simulated validation sample is therefore still used after every epoch, and the network weights producing the lowest qualifying validation loss are restored when training terminates.

The following subsections describe the extended target, input construction and TCN configuration, define transformed-prior references, introduce simulation-based calibration and the Kolmogorov–Smirnov test, and explain how the initial distribution used by the simulator is approximated.

5.2.1 Target and input construction and tuning parameters

For N independently simulated parameter–data pairs

$$\left\{ \left(\theta_{1:5}^{(i)}, y_{1:n}^{(i)} \right) \right\}_{i=1}^N$$

from the GHST-SV model, the target matrix becomes

$$\tilde{\theta}_{1:N,1:5} = \begin{pmatrix} G(\theta_{1:5}^{(1)})^\top \\ \vdots \\ G(\theta_{1:5}^{(N)})^\top \end{pmatrix} \in \mathbb{R}^{N \times 5}, \quad (5.18)$$

where G is the extended transformation in (5.17). No new input representation is required. The TCN continues to receive the standardized log-square representation from (2.51), so the input matrix remains that in (3.2), now with $n = 2530$ columns.

The numerical safeguards in Table 3.2 are retained where applicable. The target-transform margin ε_G is extended to the two new transformations. Before evaluating G , r is clipped to

$$[\varepsilon_G, 1 - \varepsilon_G],$$

and $\nu - 6$ is replaced by

$$\max\{\nu - 6, \varepsilon_G\}.$$

This ensures that both $\text{logit}(r)$ and $\log(\nu - 6)$ remain finite in finite-precision arithmetic.

The network output is extended from three to five parameter-specific heads, one for each coordinate of

$$\tilde{\theta}_{1:5} = (\mu, \psi, \rho, \tilde{r}, \tilde{\nu}).$$

Each head retains the hidden widths (32, 32). The posterior loss in (2.40) is modified by summing the five marginal Gaussian losses and replacing the factor $1/3$ by $1/5$.

Table 5.1 reports the final TCN configurations used for NPE training. The standard-SV configuration is included to show which settings are retained and which are changed for the GHST-SV estimator.

Table 5.1: Selected tuning-parameter configurations used to train the standard SV and GHST-SV TCN posterior estimators, together with the resulting receptive fields and trainable parameter counts.

Quantity	Standard-SV TCN	GHST-SV TCN
Sequence length n	253	2 530
Number of residual blocks	5	6
Block channels	(16, 32, 32, 64, 64)	(16, 32, 32, 64, 64, 64)
Kernel sizes	(9, 9, 7, 5, 5)	(9, 9, 7, 5, 5, 5)
Dilations	(1, 2, 4, 8, 16)	(1, 2, 4, 16, 64, 256)
Pooling operations	Mean, maximum, top- K	Mean, maximum, top- K
Top- K pooling fraction α_{pool}	0.05	0.05
Head hidden widths	(32, 32)	(32, 32)
Parameter-specific heads	3	5
Temporal radius R_T	144	1 392
Receptive-field width $1 + 2R_T$	289	2 785
Encoder parameters	105 392	146 480
Parameters across all heads	21 894	36 490
Total trainable parameters	127 286	182 970
Adam step size α_{Adam}	5×10^{-4}	5×10^{-4}

The pooling map is retained without modification. It combines mean and maximum pooling with top- K pooling using $\alpha_{\text{pool}} = 0.05$. The latter gives explicit weight to the largest temporal activations and can preserve information from rare or extreme responses that may be diluted by mean pooling.

The standard-SV receptive-field width of 289 is insufficient for an input of length 2530. Under the GHST-SV configuration in Table 5.1, the temporal radius

is

$$\begin{aligned}
 R_T &= \sum_{\ell=1}^6 (k_\ell - 1)d_\ell \\
 &= 8 + 16 + 24 + 64 + 256 + 1\,024 \\
 &= 1\,392.
 \end{aligned} \tag{5.19}$$

The corresponding receptive-field width is

$$1 + 2R_T = 2\,785 > 2\,530.$$

Central features in the final temporal map can consequently depend on the complete input sequence. The larger dilation factors in the later blocks increase the spacing between the values used by each convolution without increasing its number of trainable parameters.

Because the pooling dimension and head widths remain unchanged, each parameter-specific head still contains 7 298 trainable parameters. The five heads contain 36 490 parameters, which, together with the 146 480 encoder parameters, gives 182 970 trainable parameters in total. Hence,

$$\frac{182\,970}{127\,286} \approx 1.44,$$

so the GHST-SV TCN contains approximately 43.7% more trainable parameters than the standard-SV TCN. Processing a sequence ten times as long nevertheless increases the memory and computation required for both training and evaluation.

5.2.2 MCMC and transformed-prior reference

The `stochvol` MCMC implementation is retained as an additional comparison for the three standard-SV parameters. It uses the burn-in and retained chain lengths from Table 3.3,

$$B = 500, \quad M = 20\,000,$$

and is conditioned on sequences of length $n = 2\,530$. The sample means and variances of the transformed MCMC draws are inserted into the marginal Gaussian loss defined in (3.8).

The likelihood used by `stochvol` assumes Gaussian log-volatility innovations. When applied to data simulated from the GHST-SV model, its draws therefore represent the posterior under a misspecified standard-SV model rather than the corresponding GHST-SV posterior. The resulting losses nevertheless provide empirical reference values for μ , ψ , and ρ . No corresponding MCMC reference is available for $\tilde{r}$ or $\tilde{v}$.

A second comparison is obtained from the unconditional transformed prior and is available for all five parameters. Let

$$\widehat{m}_{\text{prior},j} = \mathbb{E}[\tilde{\Theta}_j], \quad \widehat{v}_{\text{prior},j} = \text{Var}(\tilde{\Theta}_j).$$

Table 5.2: Means and variances of the transformed prior distributions.

Parameter j	$\widehat{m}_{\text{prior},j}$	$\widehat{v}_{\text{prior},j}$
μ	0.0000	100.0000
ψ	1.4696	1.1561
ρ	-0.6352	1.2337
$\tilde{r}$	0.0000	3.2899
$\tilde{v}$	1.7668	1.6449

These moments can be calculated analytically from the priors in Table 3.1 and (5.16). Their numerical values are reported in Table 5.2.

Using the same convention as in Section 3.4.2, these moments are inserted into (3.8) to obtain $L_{K,j}^{\text{prior}}$. This provides a no-data reference evaluated on the same K benchmark parameters as the other methods. Consequently, $L_{K,j}^{\text{prior}}$ varies across independently generated benchmark sets and is correlated with the corresponding TCN and MCMC loss estimates.

The evaluation therefore reports the TCN, MCMC, and prior losses for $j \in \{\mu, \psi, \rho\}$, and the TCN and prior losses for $j \in \{\tilde{r}, \tilde{v}\}$. The standard-SV MCMC loss is only an empirical reference for the first three parameters and is not obtained from a GHST-SV posterior reference method. Since the TCN is trained under the GHST-SV model whereas the MCMC likelihood assumes Gaussian innovations, no fixed ordering between their losses is guaranteed.

5.2.3 Simulation-based calibration and the Kolmogorov–Smirnov test

Because no posterior reference is available for the complete GHST-SV model, the TCN estimator is also examined using simulation-based calibration (SBC). The procedure follows the posterior-quantile construction of Cook et al. [40]. In their implementation, posterior CDF values are estimated from posterior draws. Here, the TCN approximation has Gaussian marginals with analytically available CDFs, so these values can be evaluated directly without posterior sampling.

For each benchmark sample $i = 1, \dots, K$, a parameter vector and an observed sequence are simulated from the GHST-SV joint distribution:

$$\Theta^{(i)} \sim \pi(\theta_{1:5}), \quad Y_{1:n}^{(i)} | \Theta^{(i)} \sim f(y_{1:n}; \Theta^{(i)}), \quad \tilde{\Theta}^{(i)} = G(\Theta^{(i)}). \quad (5.20)$$

For $y_{1:n}^{(i)}$, the TCN returns the estimated Gaussian marginal densities

$$\varphi(\tilde{\theta}_j; \widehat{m}_j^{(i)}, \widehat{v}_j^{(i)}), \quad j = 1, \dots, 5.$$

Let $\widehat{F}_j^{(i)}$ denote the corresponding estimated marginal CDF. The *SBC value* for pa-

parameter j in benchmark sample i is defined as

$$c_j^{(i)} = \widehat{F}_j^{(i)}(\widetilde{\theta}_j^{(i)}) = \Phi\left(\frac{\widetilde{\theta}_j^{(i)} - \widehat{m}_j^{(i)}}{\sqrt{\widehat{v}_j^{(i)}}}\right). \quad (5.21)$$

Thus, $c_j^{(i)}$ is the posterior probability assigned by the TCN to values no greater than the transformed parameter value that generated the sequence.

Let $F_j(\cdot | y_{1:n})$ denote the exact GHST-SV posterior marginal CDF. If

$$\widehat{F}_j(\cdot | y_{1:n}) = F_j(\cdot | y_{1:n})$$

for almost every $y_{1:n}$, then, conditional on $Y_{1:n}^{(i)} = y_{1:n}$, the generating parameter value follows the corresponding posterior marginal. It follows that

$$\Pr(c_j^{(i)} \leq u | Y_{1:n}^{(i)} = y_{1:n}) = \Pr(F_j(\widetilde{\theta}_j^{(i)} | y_{1:n}) \leq u | Y_{1:n}^{(i)} = y_{1:n}) = u, \quad 0 \leq u \leq 1,$$

provided that the posterior marginal is continuous. Consequently,

$$c_j^{(i)} \sim \text{Uniform}(0, 1) \quad (5.22)$$

across independently simulated parameter–data pairs. Uniformity is examined separately for each transformed parameter using histograms of $\{c_j^{(i)}\}_{i=1}^K$.

Following the Kolmogorov–Smirnov goodness-of-fit statistic described by Massey [41], let

$$\widehat{H}_j(c) = \frac{1}{K} \sum_{i=1}^K \mathbf{1}\{c_j^{(i)} \leq c\}$$

be the empirical CDF of the SBC values. The statistic reported in the later SBC plots is

$$\text{KS}_j = \sup_{0 \leq c \leq 1} |\widehat{H}_j(c) - c|. \quad (5.23)$$

For a given c , $\widehat{H}_j(c)$ is the proportion of benchmark samples in which the generating parameter value lies at or below the estimated c -quantile. Hence, KS_j is the largest absolute difference, over all $c \in [0, 1]$, between this empirical proportion and its nominal value c .

Let $\text{KS}_K^{(0)}$ denote the same statistic calculated from K independent $\text{Uniform}(0, 1)$ values. The KS p -value for parameter j is

$$p_{\text{KS},j} = \Pr(\text{KS}_K^{(0)} \geq \text{KS}_j). \quad (5.24)$$

The limiting distribution derived by Kolmogorov [42] gives the large-sample approximation

$$p_{\text{KS},j} \approx 2 \sum_{\ell=1}^{\infty} (-1)^{\ell-1} \exp\{-2 \cdot \ell^2 \cdot K \cdot \text{KS}_j^2\}. \quad (5.25)$$

The qualitative interpretation of the histogram shapes follows Talts et al. [43]. A $\cup$ -shaped histogram has taller bars near 0 and 1 and shorter bars around $1/2$. This pattern is consistent with estimated marginals that are too narrow, since the generating values then fall too frequently in their extreme quantiles. Conversely, a $\cap$ -shaped histogram has a central hump and is consistent with estimated marginals that are too wide. An asymmetric concentration toward 0 or 1 is consistent with systematic error in posterior location: mass near 0 suggests that the estimated marginal is shifted toward larger values, whereas mass near 1 suggests a shift toward smaller values. These patterns are diagnostic rather than conclusive, and several forms of error may occur simultaneously.

At the population level, marginal uniformity is necessary but not sufficient for exact posterior recovery. Systematic non-uniformity therefore provides evidence that an estimated marginal differs from the corresponding GHST-SV posterior marginal, whereas an approximately uniform finite-sample histogram only means that this check did not reveal a discrepancy. Moreover, an estimator that ignores the observations and returns the prior can pass these marginal checks because the prior CDF evaluated at a draw from the prior is itself uniform. SBC is therefore used as a necessary diagnostic of the TCN approximation, not as proof that the complete posterior has been recovered.

5.2.4 Approximating the initial distribution

Simulating $y_{1:n}$ from the GHST-SV model requires an initial log-volatility H_1 . Although its stationary mean and variance are available from (2.5), the stationary density

$$f_{\text{stat}}(\cdot \mid \mu, \phi, \sigma, r, \nu)$$

is generally not available in closed form. We approximate it by starting from a moment-matched Gaussian distribution and applying a latent burn-in under the GHST-SV transition equation. The justification follows the analysis in Bahr [37, Section 3.1.1], whose main results are summarized here.

For any random variable X with the required finite moments, define

$$\text{skew}(X) = \mathbb{E} \left[\left(\frac{X - \mathbb{E}[X]}{\sqrt{\text{Var}(X)}} \right)^3 \right]$$

and

$$\text{excess kurt}(X) = \mathbb{E} \left[\left(\frac{X - \mathbb{E}[X]}{\sqrt{\text{Var}(X)}} \right)^4 \right] - 3.$$

Since the GHST innovation is centered and standardized, let

$$\gamma_1 = \text{skew}(\eta(Z_t, U_t; r, \nu)) = \mathbb{E}[\eta(Z_t, U_t; r, \nu)^3],$$

and, when its fourth moment is finite, let

$$\gamma_2 = \text{excess kurt}(\eta(Z_t, U_t; r, \nu)) = \mathbb{E}[\eta(Z_t, U_t; r, \nu)^4] - 3.$$

The standardized stationary log-volatility has the infinite-sum representation

$$\frac{H_t - \mu}{\sigma / \sqrt{1 - \phi^2}} = \sqrt{1 - \phi^2} \sum_{j=0}^{\infty} \phi^j \eta(Z_{t-j}, U_{t-j}; r, \nu).$$

Independence of the innovations gives

$$\text{skew} \left(\frac{H_t - \mu}{\sigma / \sqrt{1 - \phi^2}} \right) = \gamma_1 \frac{(1 - \phi^2)^{3/2}}{1 - \phi^3},$$

and, when γ_2 is finite,

$$\text{excess kurt} \left(\frac{H_t - \mu}{\sigma / \sqrt{1 - \phi^2}} \right) = \gamma_2 \frac{(1 - \phi^2)^2}{1 - \phi^4} = \gamma_2 \frac{1 - \phi^2}{1 + \phi^2}.$$

These expressions follow from the stationary AR(1) representation and the independence of the innovations, rather than from the particular innovation distribution considered by Bahr [37]. They therefore also apply to the GHST-SV model whenever the required moments are finite. The skewness expression is well defined under the restriction $\nu > 6$. In the skewed case $r > 0$, the excess-kurtosis expression additionally requires $\nu > 8$.

Both damping factors converge to zero as $|\phi| \rightarrow 1$. Hence, although the stationary distribution remains non-Gaussian, its skewness and, when defined, excess kurtosis are reduced for large $|\phi|$. This supports the use of a moment-matched Gaussian distribution as the starting point of the latent burn-in when the process is highly persistent.

For a latent burn-in length m_B , the simulation is initialized by drawing

$$H_{1-m_B} \sim \mathcal{N} \left(\mu, \frac{\sigma^2}{1 - \phi^2} \right). \quad (5.26)$$

The GHST-SV transition is then applied for $t = 2 - m_B, \dots, 1$:

$$H_t - \mu = \phi(H_{t-1} - \mu) + \sigma \eta(Z_t, U_t; r, \nu).$$

Because the Gaussian starting distribution in (5.26) has the stationary mean and variance, these moments are preserved throughout the burn-in:

$$\mathbb{E}[H_t] = \mu, \quad \text{Var}(H_t) = \frac{\sigma^2}{1 - \phi^2}, \quad t = 1 - m_B, \dots, 1.$$

The resulting distribution of H_1 therefore matches the first two moments of f_{stat} for every m_B .

Expanding the recursion gives

$$H_1 - \mu = \phi^{m_B}(H_{1-m_B} - \mu) + \sigma \sum_{j=0}^{m_B-1} \phi^j \eta(Z_{1-j}, U_{1-j}; r, \nu).$$

The only term determined by the Gaussian starting distribution is multiplied by ϕ^{m_B} . When $|\phi|$ is small, the burn-in therefore rapidly reduces sensitivity to its form. When $|\phi|$ is large, this reduction is slower, but the stationary skewness and finite excess kurtosis are more strongly damped. These complementary properties motivate combining the moment-matched Gaussian starting distribution with a latent burn-in.

The burn-in length used in the results was selected by comparing candidate values of m_B with a reference distribution generated using a much longer burn-in. Their empirical quantiles were compared using QQ plots, with the long-burn-in distribution serving as a numerical approximation to f_{stat} . Among the values considered, $m_B = 20$ was selected as the optimal burn-in length. The diagnostic and its implementation are available in the GitHub repository linked in Appendix A.1.

Denote the distribution of H_1 obtained after these $m_B = 20$ GHST-SV transitions by

$$\hat{f}_{\text{stat}}(\cdot | \mu, \phi, \sigma, r, \nu).$$

Thus, $\hat{f}_{\text{stat}}$ is the generally non-Gaussian distribution obtained after the latent burn-in. The simulated sequence is initialized with

$$H_1 \sim \hat{f}_{\text{stat}}(\cdot | \mu, \phi, \sigma, r, \nu),$$

after which $H_2, \dots, H_n$ and $Y_1, \dots, Y_n$ are generated from (5.14) and (5.13).

We therefore have all components required to simulate from the GHST-SV model, train the TCN estimator, and evaluate it in the following results section.

5.3 Results

This section evaluates the TCN estimator for the five-parameter GHST-SV model. We first examine its validation-loss history, then compare the marginal Gaussian losses for MCMC, the TCN and the prior, and finally assess the estimated marginal posteriors using credible-interval sweeps and simulation-based calibration.

5.3.1 Validation loss history

Figure 5.3 shows the validation-loss history for the TCN under the GHST-SV model.

The curve exhibits greater epoch-to-epoch variation than the TCN histories for the standard SV model, including several isolated upward spikes. These spikes are short-lived and are not followed by a sustained increase, so the history does not indicate unstable optimization. Instead, the loss remains within its lowest observed range for an extended period before training terminates at approximately epoch 460, suggesting that it had reached a stable plateau. The more pronounced fluctuations may reflect either a more challenging optimization problem for the five-parameter GHST-SV model or greater variance in the validation-loss estimates due to the additional parameters, or both.

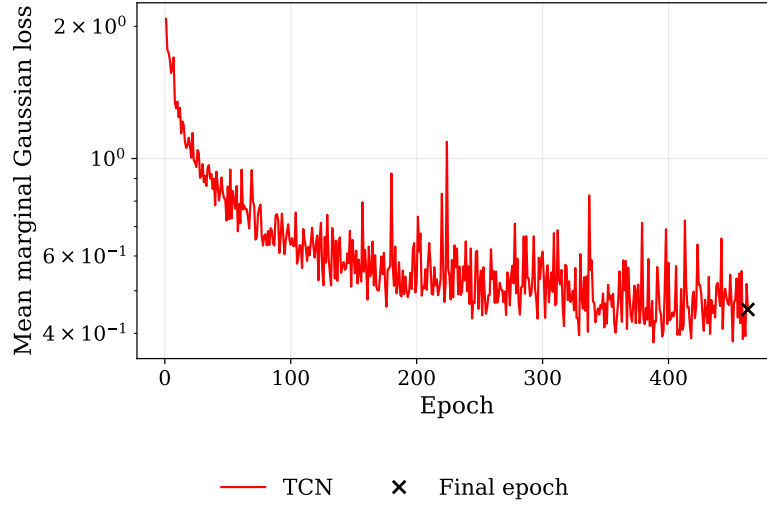


Figure 5.3: Validation-loss history for the TCN under the GHST-SV model. The vertical axis is logarithmic, and the cross marks the terminal epoch. The restored weights correspond to the best qualifying checkpoint.

5.3.2 Loss comparison among MCMC, TCN and prior

Table 5.3: Marginal Gaussian losses for MCMC, TCN and prior, calculated from $K = 5\,000$ benchmark pairs with sequence length $n = 2\,530$. The MCMC moments are calculated from $M = 20\,000$ retained draws per pair after $B = 500$ burn-in iterations.

Method	$L_{K,\mu}$	$L_{K,\psi}$	$L_{K,\rho}$	$L_{K,\tilde{r}}$	$L_{K,\tilde{v}}$	L_K
MCMC	−1.2392	−0.0951	1.7260	—	—	—
TCN	−1.0757	0.0817	−0.5139	1.9254	1.5135	0.3862
Prior	3.7271	1.4798	1.5389	2.0471	1.6740	2.0934

Since all three rows in Table 5.3 are evaluated on the same $K = 5\,000$ benchmark pairs, the empirical prior losses provide direct no-data reference values for the two inference methods. The TCN attains a lower loss than the prior for all five parameters. Its average loss decreases from 2.0934 to 0.3862, corresponding to a reduction of 81.6%. This improvement is driven primarily by the three standard-SV parameters. Relative to the prior, the reductions are only 5.9% for $\tilde{r}$ and 9.6% for $\tilde{v}$. The TCN therefore improves on the fixed prior moments for both parameters, but by considerably less than for μ , ψ , and ρ . The results do not determine whether this reflects limited information about r and v in the observations, limitations in what the TCN has learned, or both.

For the three parameters available from both inference methods, MCMC attains lower losses than the TCN for μ and ψ , by 0.1635 and 0.1768, respec-

tively. The ordering reverses for ρ , where the TCN loss is -0.5139 , compared with 1.7260 for MCMC. The MCMC loss is also 0.1871 , or 12.2% , higher than the prior loss. Hence, for ρ , the posterior moments from the misspecified standard-SV model perform worse than the fixed transformed-prior moments.

This poor MCMC loss is not explained by its posterior mean. The posterior-mean root mean squared error was shown to be 0.4364 for MCMC and 0.4458 for the TCN. Instead, the mean MCMC loss is dominated by a small number of posteriors with excessively small variances. Its median ρ -loss is -0.0439 , compared with a mean of 1.7260 , while the largest 5% of the losses account for approximately 79.1% of their total sum. Moreover, the median estimated posterior variance is 0.0040 for MCMC, compared with 0.0085 for the TCN. The unfavorable mean loss therefore reflects rare but severe overconfidence rather than consistently inaccurate point estimates.

This behavior is consistent with the model misspecification. The `stochvol` implementation assumes Gaussian log-volatility innovations, whereas the benchmark sequences are generated with skewed and heavy-tailed GHST innovations. The roles of μ and ϕ as the long-run level and persistence remain unchanged, while σ directly scales the innovation term. Inference for $\rho = \log \sigma$ is therefore particularly sensitive to the assumed innovation distribution.

The sequence length is $n = 2530$, ten times that used in the earlier standard-SV comparison. Under misspecification, the MCMC posterior can concentrate around the value of ρ preferred by the Gaussian model without approaching the generating value. The longer sequence can therefore amplify the consequences of the misspecification and provides a plausible explanation for the extreme MCMC losses.

Overall, the TCN provides large improvements over the prior reference for the three standard-SV parameters and smaller improvements for the two GHST-SV shape parameters. The MCMC loss remains useful as an empirical comparison for μ and ψ , but its misspecified likelihood prevents it from serving as a GHST-SV posterior reference for ρ .

5.3.3 Credible intervals and simulation-based calibration

Table 5.4: Baseline values and ranges used in the GHST-SV credible-interval sweeps.

Parameter	Baseline value	First sweep value	Last sweep value
μ	-9	-12	-6
ϕ	0.95	0.905	0.995
σ	0.25	0.05	0.45
r	0.50	0.20	0.80
ν	15	8	22

To examine the TCN estimates across the GHST-SV parameter space, we re-

peat the one-dimensional parameter sweeps from Section 4.3. In each of the five sweeps, one parameter was varied over ten equally spaced values, including both endpoints, while the remaining four parameters were held at their baseline values. The baseline values and sweep endpoints are given in Table 5.4.

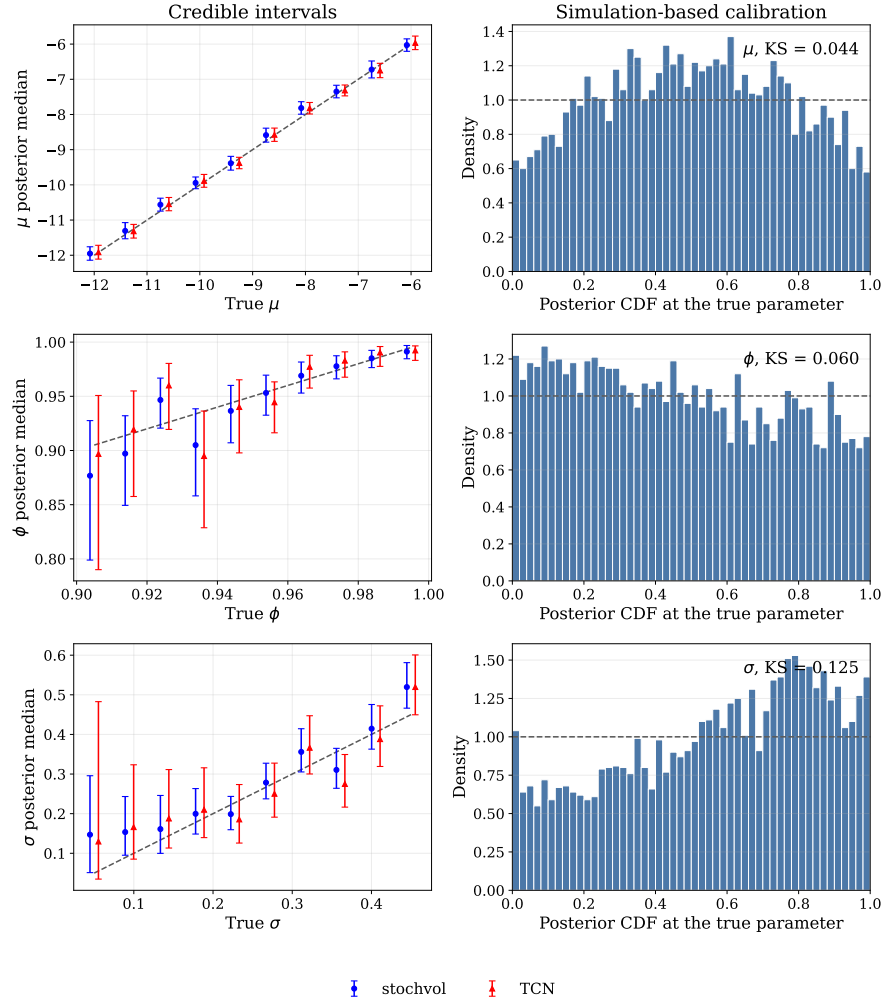


Figure 5.4: Posterior medians and 95% marginal credible intervals across the parameter sweeps (left), and TCN SBC histograms calculated from $K = 5000$ benchmark pairs (right). The rows correspond to μ , ϕ , and σ . In the left column, the dashed lines mark equality between the generating parameter and posterior median. In the right column, the dashed lines show the density of the uniform distribution.

For each parameter configuration, one sequence was simulated and used to construct a 95% marginal posterior credible interval. The TCN intervals on the original parameter scales were constructed as in (4.2). For the three standard-SV parameters, intervals obtained with *stochvol* are included as empirical compar-

ison values. The SBC histograms in the right columns of Figures 5.4 and 5.5 are instead calculated from the $K = 5000$ benchmark pairs. Because each componentwise transformation in G is strictly increasing, the SBC values are unchanged when expressed on the original parameter scales.

For all observed KS values, the terms with $\ell \geq 2$ in (5.25) are negligible, giving

$$p_{\text{KS},j} \approx 2 \exp \left\{ -2K \text{KS}_j^2 \right\}.$$

Calculated from the unrounded KS values, the largest of the five p -values is $7.8719 \cdot 10^{-9}$, so they all round to 0.0000 at four decimal places.

The small p -values provide evidence against exact uniformity, but should be interpreted in light of the large benchmark sample. With $K = 5000$, the test is sensitive to relatively small systematic differences. Even perfect recovery of the exact posterior means and variances would not ensure uniformity when the exact posterior marginals are non-Gaussian, because SBC examines the complete marginal CDF rather than only its first two moments. The histogram shapes and KS values are therefore more informative about the form and magnitude of the differences than the p -values alone.

The credible intervals in the left column closely reproduce the main patterns observed under the default prior in Section 4.3. The TCN and `stochvol` results are broadly similar for μ and ϕ , including the narrowing of the ϕ intervals near one. For σ , the TCN intervals are wider at the smaller sweep values and become more similar to the `stochvol` intervals as σ increases. The explanations given in the earlier section therefore also apply here.

For μ , the SBC histogram has a broad central hump and lower density near both endpoints. This is consistent with the TCN marginals being too wide on average, and hence with $\widehat{v}_\mu$ tending to exceed the corresponding posterior variance. The ϕ histogram instead places more mass near zero than near one, which is consistent with the estimated posterior location, and hence $\widehat{m}_\psi$, being too large on average. The stronger concentration toward one for σ is consistent with $\widehat{m}_\rho$ being too small on average.

These interpretations describe aggregate behavior across the benchmark pairs and need not hold for every individual sequence. There is therefore no contradiction between the close agreement in many of the displayed credible intervals and the rejection of exact SBC uniformity.

The credible intervals for r and ν show that the TCN estimates are strongly influenced by the prior. Across the r sweep, the intervals cover almost the entire parameter range and the posterior medians respond only weakly to changes in the generating value. The ν medians similarly remain within a comparatively narrow range, while the intervals are extremely wide and show no clear monotone change across the sweep. This prior-dominated behavior agrees with Table 5.3, where the TCN losses for $\tilde{r}$ and $\tilde{\nu}$ are only slightly smaller than the corresponding prior losses.

The SBC histograms nevertheless show structured departures from uniformity. For r , the broad interior hump is consistent with excessive posterior dispersion.

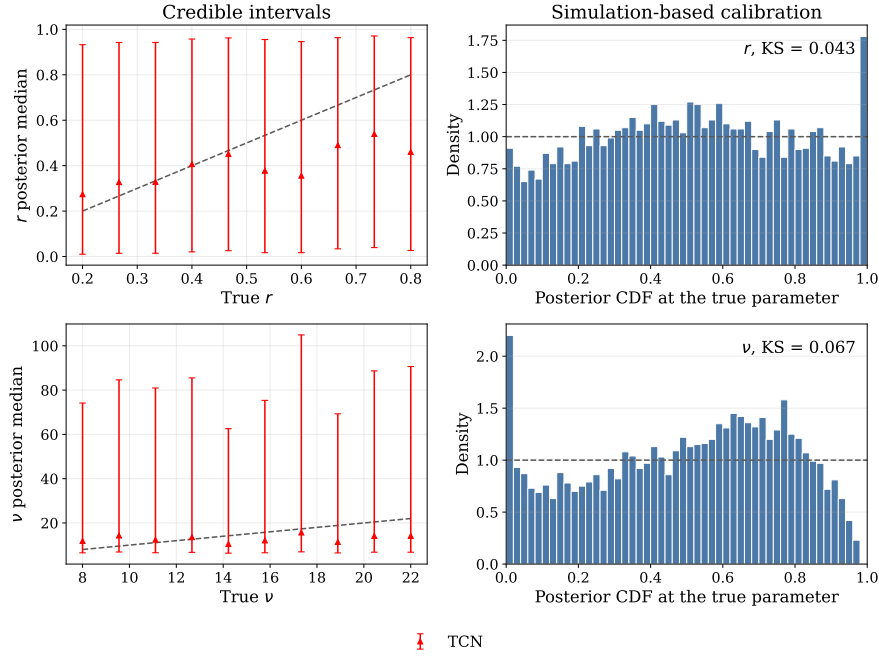


Figure 5.5: TCN posterior medians and 95% marginal credible intervals across the r and ν sweeps (left), and the corresponding SBC histograms calculated from $K = 5000$ benchmark pairs (right). In the left column, the dashed lines mark equality between the generating parameter and posterior median. In the right column, the dashed lines show the density of the uniform distribution.

The sharp increase near one means that, for a subset of the benchmark pairs, the generating value lies unusually far in the upper tail of the estimated marginal. This is consistent with a posterior location that is too small, insufficient dispersion, or both, in those cases.

The ν histogram is more asymmetric. Its pronounced spike near zero means that, for some benchmark pairs, the generating value lies far in the lower tail of the estimated marginal. This is consistent with a posterior location that is too large, insufficient dispersion, or both. The displaced interior hump and subsequent decrease toward one further suggest that the form and direction of the approximation error vary across the benchmark pairs.

Taken together, the credible-interval sweeps and prior-loss comparison show that the TCN extracts only limited information about r and ν , although it performs slightly better than returning the prior moments alone.

Chapter 6

Discussion

This chapter considers the findings of the thesis as a whole and continues the discussion in Section 4.5. Rather than repeating the parameter-wise comparisons from the two results sections, the purpose is to assess what the experiments establish about amortized posterior-moment estimation, how the main methodological choices affect that interpretation, and where the evidence remains inconclusive.

6.1 Overall assessment

The results give a qualified positive answer to the main research question. For the standard SV model, the TCN estimated the first two moments of the transformed marginal posteriors with losses close to those obtained from the moment-matched MCMC reference. Its overall loss was comparable to MCMC conditioned on 85% of the observations under the default prior and 90% under the finance prior. At the same time, evaluating the trained TCN required approximately 0.0028 seconds per sequence, compared with about 3.5 seconds for the benchmark MCMC configuration. The TCN therefore provides an effective amortized estimator when many sequences of the same length are to be analyzed under a fixed model and prior.

The Summary NN did not attain the same accuracy. Its largest differences from MCMC occurred for the uncertainty of ψ and ρ , which depends on temporal dependence and dispersion that may not be retained by the selected summary map. The TCN instead received the complete transformed sequence and could learn its own temporal features. Although the TCN contained more trainable parameters and was more expensive to train, it was also faster to evaluate because its input did not require a separate ARMA(1, 1) fit. Among the implementations examined in this thesis, the TCN consequently gives the best combination of posterior-moment accuracy and evaluation time for the standard SV model.

This conclusion is specific to the amortized setting. The TCN incurs a large initial training cost, and its learned map is tied to the model, prior, parameter transformations and sequence length used during training. MCMC does not require this training stage and can be rerun directly when these choices change.

The computational advantage of the TCN is therefore strongest when the trained estimator can be reused across a large number of comparable sequences.

6.2 Consequences of the marginal Gaussian approximation

The theoretical results in Section 2.7.3 show that the population loss targets the conditional means and variances of the transformed marginal posteriors. The standard-SV comparison supports the conclusion that the TCN approximates these moment functions well. It does not establish that the complete posterior distribution is accurately represented.

This distinction is visible in the empirical coverage results. Gaussian intervals constructed from both the TCN and the moment-matched MCMC reference were generally conservative. Since the MCMC intervals were also affected, the deviation from nominal coverage cannot be attributed only to the neural encoder. The remaining skewness of the transformed marginal posteriors and the restriction to Gaussian marginals also contribute. Moreover, the product approximation omits posterior dependence between parameters. Close agreement in the reported losses should therefore be interpreted as agreement in the first two marginal moments, not as recovery of non-Gaussian marginal shape or the joint posterior.

The comparison between the TCN and Summary NN also does not isolate the effect of the input representation. The architectures differed in parameter count, training simulations per epoch and optimization cost, in addition to their inputs. The results show that the complete TCN implementation is better for the present problem, but they do not determine how much of this advantage is caused by the learned sequence representation rather than the larger network or the different training configuration. A controlled comparison using matched capacities and training budgets would be needed to separate these effects.

6.3 Interpretation of the GHST-SV extension

The GHST-SV experiment confirms one practical advantage of simulation-based inference: the TCN template could be extended without constructing a new posterior sampler. The simulator was changed, two prediction heads were added, and the temporal encoder was enlarged so that its receptive field covered the longer sequence. This was enough to obtain a trainable estimator for all five parameters.

The strongest result from this extension concerns the three standard-SV parameters. The TCN retained large improvements over the prior for μ , ψ and ρ , even though the data were generated with skewed and heavy-tailed log-volatility innovations. For ρ , the misspecified standard-SV MCMC reference occasionally returned very small posterior variances and consequently had a larger mean loss than both the TCN and the prior. The TCN did not show the same failure. This indicates that the network adjusted to the GHST-SV simulation distribution rather than

merely reproducing the relation between observations and parameters learned under the standard SV model.

The results are much weaker for r and ν . Relative to the transformed-prior reference, the TCN loss was reduced by only 5.9% for $\tilde{r}$ and 9.6% for $\tilde{\nu}$. The credible-interval sweeps showed little movement in the posterior medians and very wide intervals, while the SBC histograms displayed clear deviations from uniformity. The TCN therefore extracted some information beyond the prior moments, but not enough to give accurate or sharply data-dependent Gaussian marginals for these parameters.

For r , the broad central concentration in the SBC histogram is consistent with excessive posterior dispersion on average. For ν , the intervals are also wide, but the asymmetric SBC histogram and its spike near zero point to errors in location or marginal shape in addition to possible overdispersion. The results therefore do not support explaining both parameters solely through overestimated posterior variances.

A plausible explanation is that the observed returns contain limited finite-sample information about the two shape parameters. The GHST innovations are standardized, so r and ν do not change the conditional mean or variance of the latent transition. Their effects must be learned from asymmetry and tail behavior after passing through the persistent latent process and the Gaussian observation equation. Different combinations of r and ν may also produce similar observable behavior over a finite sequence. The findings are therefore consistent with weak practical identifiability, meaning broad and strongly prior-dependent finite-sample posteriors. They do not establish structural non-identifiability of the GHST-SV model.

The successful estimation of ρ provides limited evidence against a complete failure of the temporal encoder. It shows that the TCN can adapt its shared representation when the innovation distribution changes. It does not, however, imply that the same representation must recover r and ν . The information relevant to these two heads differs from the information relevant to μ , ψ and ρ , and the available experiments do not separate limited information in the observations from limitations of the network or optimization.

The selected transformations provide another possible explanation for the SBC results. When the observations contain little information, the transformed posteriors remain close to the transformed priors. However, $\tilde{r} = \text{logit}(r)$ has a logistic prior distribution, while $\tilde{\nu} = \log(\nu - 6)$ has a skewed non-Gaussian prior distribution. A Gaussian marginal can match their first two moments but not their complete shapes. Consequently, even exact recovery of the posterior means and variances would not produce uniform SBC values when the true posteriors remain close to these priors. The poor SBC results for r and ν may therefore combine weak information, imperfect moment estimation and posterior-family misspecification.

6.4 Further work

The first priority is to determine why the TCN extracts only limited information about r and v . One useful experiment would be to fix one of these parameters while estimating the other. This would show whether their simultaneous variation makes their effects on the observed sequence difficult to distinguish. The estimator could also be trained and evaluated at several sequence lengths to examine whether the posterior estimates become more responsive as additional observations are provided.

An oracle experiment could further separate limited information in the observations from limitations of the neural estimator. The TCN could be trained using the latent log-volatility sequence $H_{1:n}$, or even the standardized innovations, instead of the observed returns. If r and v could then be estimated accurately, this would indicate that much of their information is obscured by the Gaussian observation equation. Poor estimates even in this setting would instead point more strongly toward confounding between the two parameters or limitations of the posterior approximation and training procedure.

A related model extension would place skewness in the observation innovation ε_t rather than in the log-volatility innovation. Such a parameter affects the observed return distribution more directly and may therefore be easier to estimate. The current input

$$\log(y_t^2 + \kappa)$$

would not be suitable on its own because squaring removes the sign of the return. A TCN for this model should instead receive the signed returns, possibly together with the log-squared sequence as a second input channel. This would provide a useful comparison with the weak recovery of the latent innovation skewness parameter in the GHST-SV model.

The posterior family should also be extended beyond separate Gaussian marginals. A multivariate Gaussian approximation would retain posterior correlations, while mixtures or conditional normalizing flows could represent non-Gaussian marginal shapes. This may be particularly important for r and v , whose posteriors can remain strongly influenced by their non-Gaussian transformed priors. Estimating the complete joint posterior would also permit joint credible regions and posterior inference for quantities depending on several SV parameters.

Finally, the method could be extended from parameter inference to volatility forecasting. This would require information about the terminal latent state, either through a separate approximation of its posterior or through a network trained directly to estimate the posterior predictive distribution of future volatility. Applying the resulting estimator to observed financial returns and examining posterior predictive behavior would provide a more practical test of whether the computational advantages found for simulated parameter inference also carry over to volatility analysis and forecasting.

Conclusion

This thesis examined whether neural posterior estimation can provide accurate and computationally efficient estimates of the first two moments of marginal parameter posteriors in stochastic-volatility models. The approach was motivated by settings in which the same Bayesian inference problem must be solved repeatedly for many return sequences. Rather than approximating the complete posterior, the parameters were transformed to an unconstrained space and each transformed marginal posterior was represented by a Gaussian distribution whose mean and variance were estimated by a neural network.

Two neural estimators were implemented for the standard SV model. The Summary NN used a fixed collection of location, dispersion and temporal-dependence features, while the TCN learned directly from the complete transformed return sequence. Both estimators were trained on newly simulated parameter–data pairs and evaluated against moment-matched marginals obtained from `stochvol` MCMC.

The TCN gave the closest neural approximation to the MCMC posterior moments under both prior configurations. Its overall marginal Gaussian loss was similar to MCMC conditioned on 85% of the observations under the default prior and 90% under the finance prior. Its mean evaluation time was approximately 0.0028 seconds per sequence, compared with about 3.5 seconds for the benchmark MCMC calculation. The Summary NN produced larger differences, especially in the posterior variances of ψ and ρ , and its ARMA-based input construction also made it slower to evaluate than the TCN.

The TCN was then extended to the GHST-SV model with skewed and heavy-tailed log-volatility innovations. The extension required a new simulator, two additional posterior heads and a larger receptive field, but no model-specific posterior sampler. The trained estimator continued to extract considerable information about μ , ψ and ρ . In particular, it avoided the severe overconfidence in ρ observed for the misspecified standard-SV MCMC reference. The results for r and ν were less successful. Their losses improved only slightly over the transformed-prior reference, their credible intervals remained strongly prior-dependent, and their SBC histograms showed clear discrepancies.

The main research question therefore receives a qualified answer. For the standard SV model, a TCN-based NPE can estimate transformed marginal posterior means and variances with losses close to a strong MCMC reference while reducing the cost of repeated evaluation by several orders of magnitude. The same

construction can be adapted to a richer simulatable model, but ease of implementation does not guarantee that every new parameter is well informed by the observations or well represented by Gaussian marginals. Among the models examined here, the clearest use case for the TCN is repeated inference for the standard SV model under a fixed prior and sequence length.

Future work should estimate the joint posterior, use more flexible marginal families or prior-Gaussianizing transformations, and examine the information available about r and ν through targeted ablations and longer sequences. Extending the method to signed observation inputs, latent-state inference and posterior predictive volatility forecasting would move the approach beyond parameter moments and toward a more complete amortized inference procedure.

Bibliography

- [1] Norges Bank Investment Management. “Investments,” Accessed: Jul. 29, 2026. [Online]. Available: <https://www.nbim.no/en/investments/>.
- [2] Norges Bank Investment Management. “Risk management,” Accessed: Jul. 29, 2026. [Online]. Available: <https://www.nbim.no/en/investments/risk-management/>.
- [3] E. Maceda, E. C. Hector, A. Lenzi, and B. J. Reich, “Demonstrating the power and flexibility of variational assumptions for amortized neural posterior estimation in environmental applications,” *Environmetrics*, vol. 37, no. 1, 2026, Art. no. e70074. DOI: 10.1002/env.70074.
- [4] United Nations. “Goal 8: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all,” Department of Economic and Social Affairs, Accessed: Jul. 29, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal8>.
- [5] P. J. Brockwell and R. A. Davis, *Introduction to Time Series and Forecasting* (Springer Texts in Statistics), 3rd ed. Cham: Springer, 2016, ISBN: 978-3-319-29854-2. DOI: 10.1007/978-3-319-29854-2.
- [6] S. J. Taylor, “Financial returns modelled by the product of two stochastic processes—a study of daily sugar prices, 1961–79,” in *Stochastic Volatility: Selected Readings*, N. Shephard, Ed., Originally published in *Time Series Analysis: Theory and Practice 1*, edited by O. D. Anderson, North-Holland, Amsterdam, 1982, pp. 203–226, Oxford: Oxford University Press, 2005, pp. 60–82, ISBN: 978-0-19-925719-5. DOI: 10.1093/oso/9780199257195.003.0003.
- [7] D. Hosszejni and G. Kastner, “Modeling univariate and multivariate stochastic volatility in R with stochvol and factorstochvol,” *Journal of Statistical Software*, vol. 100, no. 12, pp. 1–34, 2021. DOI: 10.18637/jss.v100.i12.
- [8] E. Ruiz, “Quasi-maximum likelihood estimation of stochastic volatility models,” *Journal of Econometrics*, vol. 63, no. 1, pp. 289–306, 1994. DOI: 10.1016/0304-4076(93)01569-8.
- [9] K. Kristensen, A. Nielsen, C. W. Berg, H. Skaug, and B. M. Bell, “TMB: Automatic differentiation and Laplace approximation,” *Journal of Statistical Software*, vol. 70, no. 5, pp. 1–21, 2016. DOI: 10.18637/jss.v070.i05.

- [10] J. Wahl, *stochvolTMB: Likelihood estimation of stochastic volatility models*, R package version 0.3.0, 2026. [Online]. Available: <https://github.com/jenswahl/stochvolTMB>.
- [11] C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed. New York: Springer, 2004. DOI: 10.1007/978-1-4757-4145-2.
- [12] G. Kastner, “Dealing with stochastic volatility in time series using the R package stochvol,” *Journal of Statistical Software*, vol. 69, no. 5, pp. 1–30, 2016. DOI: 10.18637/jss.v069.i05.
- [13] J. M. Flegal and G. L. Jones, “Batch means and spectral variance estimators in markov chain monte carlo,” *The Annals of Statistics*, vol. 38, no. 2, pp. 1034–1070, Apr. 2010. DOI: 10.1214/09-A05735.
- [14] K. Cranmer, J. Brehmer, and G. Louppe, “The frontier of simulation-based inference,” *Proceedings of the National Academy of Sciences*, vol. 117, no. 48, pp. 30 055–30 062, 2020. DOI: 10.1073/pnas.1912789117.
- [15] G. Papamakarios and I. Murray, “Fast ϵ -free inference of simulation models with bayesian conditional density estimation,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 1028–1036. DOI: 10.5555/3157096.3157212. Accessed: Jun. 23, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2016/hash/6aca97005c68f1206823815f66102863-Abstract.html>.
- [16] J.-M. Lueckmann, P. J. Gonçalves, G. Bassetto, K. Öcal, M. Nonnenmacher, and J. H. Macke, “Flexible statistical inference for mechanistic models of neural dynamics,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 1289–1299. Accessed: Jun. 23, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/addfa9b7e234254d26e9c7f2af1005cb-Abstract.html>.
- [17] D. Greenberg, M. Nonnenmacher, and J. Macke, “Automatic posterior transformation for likelihood-free inference,” in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 2404–2414. Accessed: Jun. 23, 2026. [Online]. Available: <https://proceedings.mlr.press/v97/greenberg19a.html>.
- [18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016, ISBN: 9780262035613. [Online]. Available: <https://www.deeplearningbook.org>.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.
- [20] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *CoRR*, vol. abs/1803.01271, 2018. arXiv: 1803.01271 [cs.LG].

- [21] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. DOI: 10.1137/16M1080173. arXiv: 1606.04838 [stat.ML]. [Online]. Available: <https://arxiv.org/abs/1606.04838>.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. DOI: 10.48550/arXiv.1412.6980. arXiv: 1412.6980 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1412.6980>.
- [23] K. P. Murphy, *Probabilistic Machine Learning: An Introduction* (Adaptive Computation and Machine Learning). The MIT Press, 2022, ISBN: 978-0-262-04682-4. [Online]. Available: <https://probml.github.io/pml-book/book1.html>.
- [24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2.
- [25] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with python," in *Proceedings of the 9th Python in Science Conference*, 2010. DOI: 10.25080/Majora-92bf1922-011.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035. DOI: 10.5555/3454287.3455008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- [27] T. Bollerslev, B. Hood, J. Huss, and L. H. Pedersen, "Risk everywhere: Modeling and managing volatility," *The Review of Financial Studies*, vol. 31, no. 7, pp. 2729–2773, 2018. DOI: 10.1093/rfs/hhy041.
- [28] A. W. van der Vaart, *Asymptotic Statistics* (Cambridge Series in Statistical and Probabilistic Mathematics). Cambridge: Cambridge University Press, 1998, vol. 3. DOI: 10.1017/CB09780511802256.
- [29] M. C. M. de Gunst and O. V. Shcherbakova, "Asymptotic behavior of bayes estimators for hidden markov models with application to ion channels," *Mathematical Methods of Statistics*, vol. 17, no. 4, pp. 342–356, 2008. DOI: 10.3103/S1066530708040054.

- [30] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: A survey,” *Journal of Machine Learning Research*, vol. 18, pp. 1–43, 2018, Art. no. 153. [Online]. Available: <https://jmlr.org/papers/v18/17-468.html>.
- [31] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” *Queue*, vol. 6, no. 2, pp. 40–53, 2008. DOI: 10.1145/1365490.1365500.
- [32] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in *International Conference on Learning Representations*, 2018, ISBN: 978-1-7138-7272-6. [Online]. Available: <https://openreview.net/forum?id=r1gs9JgRZ>.
- [33] M. Staib, S. J. Reddi, S. Kale, S. Kumar, and S. Sra, “Escaping saddle points with adaptive gradient methods,” in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 5956–5965. Accessed: Jul. 29, 2026. [Online]. Available: <https://proceedings.mlr.press/v97/staib19a.html>.
- [34] O. E. Barndorff-Nielsen and N. Shephard, “Non-gaussian Ornstein–Uhlenbeck-based models and some of their uses in financial economics,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 63, no. 2, pp. 167–241, 2001. DOI: 10.1111/1467-9868.00282.
- [35] S. Chib, F. Nardari, and N. Shephard, “Markov chain monte carlo methods for stochastic volatility models,” *Journal of Econometrics*, vol. 108, no. 2, pp. 281–316, 2002. DOI: 10.1016/S0304-4076(01)00137-3.
- [36] B. Eraker, M. Johannes, and N. Polson, “The impact of jumps in volatility and returns,” *The Journal of Finance*, vol. 58, no. 3, pp. 1269–1300, 2003. DOI: 10.1111/1540-6261.00566.
- [37] E. K. Bahr, “Parameter estimation of time-irreversible stochastic volatility models with tmb,” Unpublished project thesis. Included as an attachment to the present master’s thesis, Project thesis, Norwegian University of Science and Technology, Trondheim, 2026.
- [38] K. Aas and I. H. Haff, “The generalized hyperbolic skew Student’s t -distribution,” *Journal of Financial Econometrics*, vol. 4, no. 2, pp. 275–309, 2006. DOI: 10.1093/jjfinec/nbj006.
- [39] M. C. Jones and A. Pewsey, “Sinh–arcsinh distributions,” *Biometrika*, vol. 96, no. 4, pp. 761–780, 2009. DOI: 10.1093/biomet/asp053.
- [40] S. R. Cook, A. Gelman, and D. B. Rubin, “Validation of software for Bayesian models using posterior quantiles,” *Journal of Computational and Graphical Statistics*, vol. 15, no. 3, pp. 675–692, 2006. DOI: 10.1198/106186006X136976.

- [41] F. J. Massey Jr., “The Kolmogorov–Smirnov test for goodness of fit,” *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951. DOI: 10.1080/01621459.1951.10500769.
- [42] A. N. Kolmogorov, “Sulla determinazione empirica di una legge di distribuzione,” *Giornale dell’Istituto Italiano degli Attuari*, vol. 4, no. 1, pp. 83–91, 1933. Accessed: Jul. 29, 2026. [Online]. Available: <https://digitale.bnc.roma.sbn.it/tecadigitale/giornale/CFI0353791/1933/unico>.
- [43] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman. “Validating Bayesian inference algorithms with simulation-based calibration.” arXiv: 1804.06788 [stat.ME], Accessed: Jul. 27, 2026. [Online]. Available: <https://arxiv.org/abs/1804.06788>.

Appendix A

Additional Material

A.1 GitHub project link

The code used in this thesis is available at
<https://github.com/edvardbahr/Master-thesis>.

A.2 Moments of the log-squared observations

The centering and scaling constants used for the transformed TCN input can be obtained analytically under the parameter prior in (2.26). Recall that the SV observation equation is

$$y_t = \exp(h_t/2)\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1).$$

Conditional on the SV parameters, the log-volatility has moments

$$\mathbb{E}[h_t \mid \mu, \phi, \sigma] = \mu, \quad \text{Var}(h_t \mid \mu, \phi, \sigma) = \frac{\sigma^2}{1 - \phi^2}.$$

Since

$$\log(y_t^2) = h_t + \log(\varepsilon_t^2),$$

and h_t and ε_t are conditionally independent, the moments of $\log(y_t^2)$ can be obtained from the moments of these two terms.

Because $\varepsilon_t^2 \sim \chi_1^2$,

$$\mathbb{E}[\log(\varepsilon_t^2)] = -\gamma - \log 2, \quad \text{Var}(\log(\varepsilon_t^2)) = \frac{\pi^2}{2},$$

where γ denotes the Euler–Mascheroni constant. It follows that

$$\mathbb{E}[\log(y_t^2) \mid \mu, \phi, \sigma] = \mu - \gamma - \log 2$$

and

$$\text{Var}(\log(y_t^2) \mid \mu, \phi, \sigma) = \frac{\sigma^2}{1 - \phi^2} + \frac{\pi^2}{2}.$$

The law of total expectation and the prior for μ in (2.26) give

$$\mathbb{E}[\log(y_t^2)] = m_\mu - \gamma - \log 2.$$

The law of total variance similarly gives

$$\text{Var}(\log(y_t^2)) = \mathbb{E}\left[\frac{\sigma^2}{1-\phi^2}\right] + \frac{\pi^2}{2} + s_\mu^2.$$

The final term is the variance of the conditional mean $\mu - \gamma - \log 2$.

Under the independent priors in (2.26),

$$\mathbb{E}\left[\frac{\sigma^2}{1-\phi^2}\right] = \mathbb{E}[\sigma^2] \mathbb{E}\left[\frac{1}{1-\phi^2}\right].$$

The scaled chi-squared prior gives

$$\mathbb{E}[\sigma^2] = B_\sigma.$$

To evaluate the remaining expectation, let

$$U = \frac{\phi + 1}{2} \sim \text{Beta}(a_\phi, b_\phi).$$

Since

$$1 - \phi^2 = 4U(1 - U)$$

and

$$\frac{1}{U(1-U)} = \frac{1}{U} + \frac{1}{1-U},$$

we obtain

$$\mathbb{E}\left[\frac{1}{1-\phi^2}\right] = \frac{1}{4} \left\{ \mathbb{E}\left[\frac{1}{U}\right] + \mathbb{E}\left[\frac{1}{1-U}\right] \right\}.$$

The inverse moments follow from ratios of beta functions. The beta function is defined by

$$B(x, y) = \int_0^1 u^{x-1} (1-u)^{y-1} du = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}, \quad x, y > 0.$$

For $U \sim \text{Beta}(a_\phi, b_\phi)$, this gives the general identity

$$\mathbb{E}[U^r (1-U)^s] = \frac{B(a_\phi + r, b_\phi + s)}{B(a_\phi, b_\phi)},$$

whenever the corresponding integral is finite. In particular,

$$\mathbb{E}\left[\frac{1}{U}\right] = \frac{B(a_\phi - 1, b_\phi)}{B(a_\phi, b_\phi)} = \frac{a_\phi + b_\phi - 1}{a_\phi - 1}$$

and

$$\mathbb{E}\left[\frac{1}{1-U}\right] = \frac{B(a_\phi, b_\phi - 1)}{B(a_\phi, b_\phi)} = \frac{a_\phi + b_\phi - 1}{b_\phi - 1}.$$

Consequently,

$$\mathbb{E}\left[\frac{1}{1-\phi^2}\right] = \frac{a_\phi + b_\phi - 1}{4} \left(\frac{1}{a_\phi - 1} + \frac{1}{b_\phi - 1} \right),$$

provided $a_\phi > 1$ and $b_\phi > 1$. If either condition fails, this expectation, and hence the prior-predictive variance of $\log(y_t^2)$, is infinite.

Combining these results, the constants used to standardize the TCN input for a given prior configuration are

$$\begin{aligned} a_x &= m_\mu - \gamma - \log 2, \\ s_x &= \left[s_\mu^2 + \frac{\pi^2}{2} + B_\sigma \frac{a_\phi + b_\phi - 1}{4} \left(\frac{1}{a_\phi - 1} + \frac{1}{b_\phi - 1} \right) \right]^{1/2}. \end{aligned} \quad (\text{A.1})$$

These are the exact prior-predictive mean and standard deviation of $\log(y_t^2)$. The stabilizing constant κ is not included in these analytic moments and serves only as a numerical safeguard in the implemented transformation $\log(y_t^2 + \kappa)$.

A.3 ARIMA initialization

Let $\hat{r}_1, \dots, \hat{r}_{n_\gamma}$ denote the untransformed autocovariance ratios. The preliminary persistence estimate is

$$\widehat{\phi}_0 = \begin{cases} \text{median}(\hat{r}_2, \dots, \hat{r}_{n_\gamma}), & n_\gamma \geq 2, \\ \hat{r}_1, & n_\gamma = 1, \end{cases} \quad (\text{A.2})$$

with clipping to the interior of $(-1, 1)$. Excluding the first ratio from the median when possible reduces sensitivity to the distinctive first-lag effect of the observation noise in the log-square representation. The optimizer is initialized at

$$(\bar{x}, \widehat{\phi}_0, 0, \widehat{\sigma}_{A,0}^2), \quad \widehat{\sigma}_{A,0}^2 = \max \{ \widehat{\text{Var}}(x) (1 - \widehat{\phi}_0^2), \varepsilon_{\text{sum}} \}. \quad (\text{A.3})$$

The four entries are the constant, autoregressive coefficient, moving-average coefficient and innovation variance.

A.4 Gaussian limit as $\nu \rightarrow \infty$

Fix $r \in [0, 1)$. For each $\nu > 4$, let

$$Z \sim \mathcal{N}(0, 1), \quad U_\nu \sim \text{InvGamma}\left(\frac{\nu}{2}, \frac{1}{2}\right),$$

independently, and write

$$\eta_\nu = \sqrt{(\nu-2)(1-r)U_\nu} Z + \sqrt{\frac{r(\nu-4)}{2}} ((\nu-2)U_\nu - 1).$$

Since

$$Q_\nu = U_\nu^{-1} \sim \chi_\nu^2,$$

the standard large-degrees-of-freedom limit for a chi-squared variable gives

$$\frac{Q_\nu}{\nu} \xrightarrow{p} 1, \quad G_\nu := \frac{Q_\nu - \nu}{\sqrt{2\nu}} \xrightarrow{d} G, \quad G \sim \mathcal{N}(0, 1).$$

In other words, Q_ν is approximately

$$Q_\nu \approx \nu + \sqrt{2\nu} G_\nu$$

when ν is large.

Substituting $U_\nu = 1/Q_\nu$ directly into η_ν gives

$$\eta_\nu = \sqrt{(1-r)\frac{\nu-2}{Q_\nu}} Z + \sqrt{\frac{r(\nu-4)}{2}} \left(\frac{\nu-2}{Q_\nu} - 1 \right).$$

Using

$$\frac{\nu-2}{Q_\nu} - 1 = -\frac{Q_\nu - \nu}{Q_\nu} - \frac{2}{Q_\nu},$$

we can write this as

$$\begin{aligned} \eta_\nu &= \sqrt{(1-r)\frac{\nu-2}{Q_\nu}} Z \\ &\quad - \sqrt{r} \frac{\sqrt{\nu(\nu-4)}}{Q_\nu} G_\nu - \sqrt{r} \frac{\sqrt{2(\nu-4)}}{Q_\nu}. \end{aligned}$$

The three terms now have simple limits. First,

$$\sqrt{(1-r)\frac{\nu-2}{Q_\nu}} = \sqrt{(1-r)\frac{1-2/\nu}{Q_\nu/\nu}} \xrightarrow{p} \sqrt{1-r}.$$

Second,

$$\frac{\sqrt{\nu(\nu-4)}}{Q_\nu} = \frac{\sqrt{1-4/\nu}}{Q_\nu/\nu} \xrightarrow{p} 1.$$

Finally,

$$\frac{\sqrt{2(\nu-4)}}{Q_\nu} \xrightarrow{p} 0.$$

Consequently,

$$\eta_\nu \xrightarrow{d} \sqrt{1-r} Z - \sqrt{r} G.$$

Because Q_ν , and hence G_ν , is independent of Z , the limiting variables G and Z are independent. Therefore,

$$\sqrt{1-r} Z - \sqrt{r} G \sim \mathcal{N}(0, (1-r) + r) = \mathcal{N}(0, 1).$$

We have therefore shown that

$$\eta(Z, U_\nu; r, \nu) \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{as } \nu \rightarrow \infty$$

for every fixed $r \in [0, 1)$.

